TI TOTAL

ÁREA FISCAL E CONTROLE

Professor
Ramon Souza

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Tecnologia da Informação

TEORIA

Engenharia de Software

SUMÁRIO

1. ENGENHARIA DE SOFTWARE 5
1.1 O que é Engenharia de Software? 5
1.2 Princípios de Engenharia de Software 8
1.3 Relação da engenharia de software com outras disciplinas 12
2. CICLO DE VIDA E METODOLOGIAS DE SOFTWARE 15
2.1 Introdução ao ciclo de vida e metodologias 15
2.1.1 Conceitos iniciais 15
2.1.2 Modelos de processos de negócios 16
2.1.3 Tipos de fluxos de processos 19
2.2 Modelos lineares ou sequenciais 21
2.2.1 Modelo cascata 21
2.2.2 Modelo formal 26
2.2.3 Modelo V 27
2.3 Modelos incrementais 30
2.3.1 Desenvolvimento incremental 30
2.3.2 Processo Unificado 35
2.3.3 Rapid Application Development - RAD 43
2.4 Modelos evolucionários 44
2.4.1 Prototipação 44
2.4.2 Modelo espiral 52
2.4.3 Modelo concorrente 56
2.5 Modelos especializados 57
2.5.1 Desenvolvimento baseado em componentes 57
2.5.2 Desenvolvimento orientado a aspectos 60

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

3. METODOLOGIAS ÁGEIS 61
3.1 Modelos Ágeis 61
3.2 Extreme Programming - XP 64
3.2.1 Valores do XP 64
3.2.2 Atividades do XP 66
3.2.3 Práticas do XP 68
3.3 Scrum 75
3.3.1 Pilares do Scrum 76
3.3.2 Valores do Scrum 78
3.3.3 Time Scrum 79
3.3.4 Estrutura do Scrum 81
3.3.5 Eventos do Scrum 82
3.3.6 Artefatos do Scrum 85
3.4 DSDM 87
3.5 FDD 90
3.6 TDD 93
3.7 ATDD 96
3.8 ASD 100
3.9 Kanban 102
3.10 Especificação por exemplo 104
3.11 DDD 107
4. ESQUEMAS DE AULA 111
5. REFERÊNCIAS 142

A nossa aula é bem esquematizada, então para facilitar o seu acesso aos **esquemas**, você pode usar o seguinte índice:

Esquema 1 – Engenharia de Software. 6
Esquema 2 – Engenharia de Software em Camadas. 7
Esquema 3 – Princípios da Engenharia de Software. 10
Esquema 4 – Engenharia de Software e outras disciplinas. 12
Esquema 5 – Processo de Software. 15
Esquema 6 – Modelo de Processo Genérico do Pressman. 16
Esquema 7 – Modelo de Processo Genérico de Sommerville. 17
Esquema 8 – Tipos de Fluxos de Processo. 19
Esquema 9 – Modelo Cascata. 22
Esquema 10 – Vantagens e desvantagens do Modelo Cascata. 22

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Esquema 11 – Estágios do modelo Cascata de Sommerville. 23
Esquema 12 – Fases do modelo Cascata de Pressman. 23
Esquema 13 – Modelos formais. 26
Esquema 14 – Modelo V segundo Pressman. 27
Esquema 15 – Desenvolvimento Incremental segundo Sommerville. 30
Esquema 16 – Modelo Incremental segundo Pressman. 30
Esquema 17 – Características do Desenvolvimento Incremental. 31
Esquema 18 – Vantagens e desvantagens do desenvolvimento incremental. 32
Esquema 19 – Iterativo x Incremental. 33
Esquema 20 – Rational Unified Processo – RUP. 35
Esquema 21 – Perspectiva Dinâmica ou fases do RUP. 36
Esquema 22 – Perspectiva estática ou disciplinas do RUP. 37
Esquema 23 – Fases e Disciplinas do RUP. 38
Esquema 24 – Perspectiva prática ou boas práticas do RUP. 39
Esquema 25 – Rapid Application Development (RAD). 43
Esquema 26 – Prototipação. 45
Esquema 27 – Tipos de prototipação. 46
Esquema 28 – Vantagens e desvantagens da prototipação. 48
Esquema 29 – Modelo de Processo para Desenvolvimento de Protótipos de Sommerville. 49
Esquema 30 – Paradigma de prototipação segundo Pressman. 49
Esquema 31 – Modelo Espiral. 52
Esquema 32 – Modelo espiral de Boehm segundo Sommerville. 53
Esquema 33 – Modelo espiral proposto por Pressman. 54
Esquema 34 – Vantagens e desvantagens do modelo espiral. 54
Esquema 35 – Modelo de Desenvolvimento Concorrente ou Engenharia Concorrente. 56
Esquema 36 – Desenvolvimento baseado em componentes ou orientado a reuso. 57
Esquema 37 – Desenvolvimento orientado a reuso de Sommerville. 58
Esquema 38 – Desenvolvimento baseado em componentes de Pressman. 59
Esquema 39 – Desenvolvimento orientado a aspectos. 60
Esquema 40 – Métodos ágeis. 61
Esquema 41 – Valores do XP. 65
Esquema 42 – Processo de Extremming Programming segundo Pressman. 67
Esquema 43 – Práticas do XP. 72
Esquema 44 – Scrum. 75
Esquema 45 – Pilares do Scrum. 76
Esquema 46 – Valores do Scrum. 78
Esquema 47 – Time Scrum. 79
Esquema 48 – Processo do Scrum. 81
Esquema 49 – Eventos Scrum. 83
Esquema 50 – Artefatos do Scrum. 85
Esquema 51 – Método de Desenvolvimento de Sistemas Dinâmicos (DSDM). 87
Esquema 52 – Ciclo de vida DSDM. 88
Esquema 53 – Feature Driven Development – FDD. 91
Esquema 54 – Ciclo TDD. 93
Esquema 55 – Ciclo TDD Simplificado. 94
Esquema 56 – Características dos testes. 94
Esquema 57 – Ciclo de Implementação do ATDD. 97
Esquema 58 – Etapas do ATDD. 98
Esquema 59 – Ciclo do ASD. 100
Esquema 60 – Kanban. 102
Esquema 61 – Especificação por exemplo. 106
Esquema 62 – DDD. 108

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

1. ENGENHARIA DE SOFTWARE

Hoje, o **software** assume um papel duplo. É **um produto e, ao mesmo tempo, o veículo para entregar um produto**. Como produto, ele fornece o potencial computacional incorporado pelo hardware do computador ou, de forma mais ampla, por uma rede de computadores acessíveis por hardware local. Independentemente de ele residir em um telefone celular ou operar dentro de um computador mainframe, o software é um transformador de informações - produzindo, gerenciando, adquirindo, modificando, exibindo ou transmitindo informações que podem ser simples como um único bit ou tão complexas quanto uma apresentação multimídia derivada de dados adquiridos de dezenas de fontes independentes. Como o veículo usado para entregar o produto, o software atua como base para o controle do computador (sistemas operacionais), a comunicação de informações (redes) e a criação e controle de outros programas (ferramentas de software e ambientes).

O mundo moderno não poderia existir sem o software. Infraestruturas e serviços nacionais são controlados por sistemas computacionais, e a maioria dos produtos elétricos inclui um computador e um software que o controla. A manufatura e a distribuição industriais são totalmente informatizadas, assim como o sistema financeiro. A área de entretenimento, incluindo a indústria da música, jogos de computador, cinema e televisão, faz uso intensivo de software. Portanto, a **engenharia de software** é **essencial para o funcionamento das sociedades nacionais e internacionais**.

1.1 O que é Engenharia de Software?

A **engenharia de software** é uma **disciplina de engenharia que se preocupa com todos os aspectos da produção de software**, desde os estágios iniciais da especificação do sistema até a manutenção do sistema após sua utilização (SOMMERVILLE, 2011).

Sendo uma **disciplina de engenharia**, aplicam-se **processos, teorias, métodos e ferramentas onde apropriados**. Esses instrumentos são usados seletivamente para resolver os problemas, mas os engenheiros buscam as soluções mesmo quando não há teorias e métodos aplicáveis. Outra característica da engenharia é a de realizar os trabalhos com **restrições organizacionais e financeiras**.

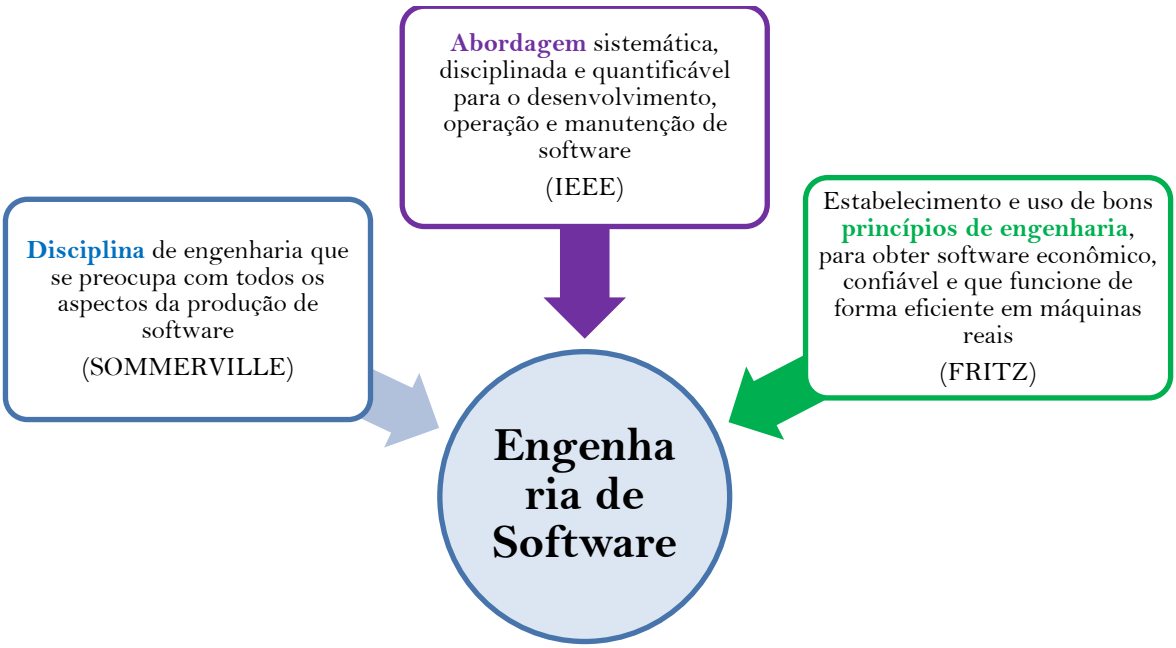
A **engenharia de software** trata de **todos os aspectos de software**, isto é, **não se preocupa apenas com os processos técnicos** de desenvolvimento de software. Também inclui atividades como gerenciamento de projetos de software e o desenvolvimento de ferramentas, métodos e teorias para suportar a produção de software.

A IEEE resume **engenharia de software** como a **aplicação de uma abordagem sistemática, disciplinada e quantificável para o desenvolvimento, operação e manutenção de software**.

Fritz Bauer destaca que a **engenharia de software** é o **estabelecimento e uso de bons princípios de engenharia**, a fim **de obter software de maneira econômica**, que seja **confiável** e que funcione de forma **eficiente em máquinas reais**.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Podemos então esquematizar a engenharia de software como:



Esquema 1 – Engenharia de Software.

Pressman destaca que a **engenharia de software** é **baseada em camadas**:

- O **alicerce** que suporta a engenharia de software é o **foco na qualidade**.
- A base para a engenharia de software é a camada de **processo**. O **processo** é a cola que une as camadas de tecnologia e permite o desenvolvimento racional e oportuno de software. O **processo** define a **estrutura que deve ser estabelecida para a entrega efetiva** dos produtos da engenharia de software.
- Os **métodos** **fornecem as instruções técnicas para a criação de software**. Os **métodos** englobam uma ampla gama de tarefas que incluem comunicação, análise de requisitos, modelagem de projetos, construção de programas, testes e suporte.
- As **ferramentas** fornecem **suporte automatizado ou semiautomatizado para o processo e os métodos**.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Vamos ver um esquema sintetizando os componentes da engenharia de software.

Esquema 2 – Engenharia de Software em Camadas.

1- (CESPE - 2018 - EBSERH - Técnico em Informática) Em relação aos conceitos de engenharia de software, julgue o item subsecutivo.
A engenharia de software se concentra nos aspectos práticos da produção de um sistema de software.

Resolução:

A **engenharia de software** é uma **disciplina de engenharia que se preocupa com todos os aspectos da produção de software**, desde os estágios iniciais da especificação do sistema até a manutenção do sistema após sua utilização (SOMMERVILLE, 2011).

A IEEE resume **engenharia de software** como a **aplicação de uma abordagem sistemática, disciplinada e quantificável para o desenvolvimento, operação e manutenção de software**.

Gabarito: Certo.

2- (CESPE - 2015 - STJ - Técnico Judiciário - Tecnologia da Informação) Com referência aos conceitos básicos de engenharia de software, julgue o item seguinte.

Os procedimentos de engenharia de software constituem o elo entre os métodos e as ferramentas, o que possibilita o desenvolvimento racional e oportuno de software.

Resolução:

A **engenharia de software** é uma **disciplina de engenharia que se preocupa com todos os aspectos da produção de software**, desde os estágios iniciais da especificação do sistema até a manutenção do sistema após sua utilização.

Sendo uma **disciplina de engenharia**, são aplicadas **teorias, métodos e ferramentas onde são apropriados**. Esses instrumentos são usados seletivamente para resolver os problemas, mas os engenheiros buscam as soluções mesmo quando não há teorias e métodos aplicáveis. Outra característica da engenharia é a de realizar os trabalhos com **restrições organizacionais e financeiras**.

Gabarito: Certo.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

1.2 Princípios de Engenharia de Software

David Hooker propôs **sete princípios** que enfocam a prática da engenharia de software como um todo:

1. **A razão para tudo existir**: um **software existe para fornecer valor a seus usuários**. Todas as decisões devem ser tomadas com isso em mente. Antes de especificar um requisito do sistema, antes de observar uma funcionalidade do sistema, antes de determinar as plataformas de hardware ou processos de desenvolvimento, faça perguntas como: "Isso agrega valor real ao sistema?" Se a resposta for "não", não faça isso. Todos os outros princípios apóiam este.

2. **KISS (Keep It Simple, Stupid!)**: o design de software não é um processo casual. Há muitos fatores a serem considerados em qualquer esforço de design. Todo **o design deve ser o mais simples possível**, mas não simplista. Isso torna sistema mais facilmente compreendido e de fácil manutenção. Isso não quer dizer que recursos, até mesmo recursos internos, devam ser descartados em nome da simplicidade. De fato, os designs mais elegantes são geralmente os mais simples. Simples também não significa "rápido e malfeito". Na verdade, muitas vezes é preciso pensar muito e trabalhar em várias iterações para simplificar. A recompensa é um software mais sustentável e menos propenso a erros.

3. **Mantenha a visão**: uma **visão clara é essencial para o sucesso** de um projeto de software. Sem integridade conceitual, um sistema ameaça tornar-se uma colcha de retalhos de projetos incompatíveis, unidos pelo tipo errado de parafusos. Comprometer a visão arquitetônica de um sistema de software enfraquece e acabará por quebrar mesmo os sistemas bem projetados. Ter um arquiteto capacitado que possa **manter a visão e impor a conformidade** ajuda a garantir um projeto de software muito bem-sucedido.

4. **O que você produz, outros irão consumir**: de uma forma ou de outra, alguém usará, manterá, documentará ou dependerá do entendimento do seu sistema. Então, sempre **especifique, projete e implemente, sabendo que alguém terá que entender o que você está fazendo**. Especifique com um olho para os usuários. Projete, mantendo os implementadores em mente. Codifique com preocupação aqueles que devem manter e estender o sistema. Alguém pode ter que depurar o código que você escreve, e isso faz deles um usuário do seu código. Facilitar seu trabalho agrega valor ao sistema.

5. **Esteja aberto ao futuro**: **um sistema com uma longa vida útil tem mais valor**. Nos ambientes de computação de hoje, onde as especificações mudam em um piscar de olhos e as plataformas de hardware são obsoletas com apenas alguns meses, as vidas úteis dos softwares costumam ser medidas em meses em vez de anos. No entanto, os verdadeiros sistemas de software de "força industrial" devem durar muito mais tempo. Para fazer isso com sucesso, esses sistemas devem estar prontos para se adaptar a essas e outras mudanças.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Sistemas que fazem isso com sucesso são aqueles que foram projetados dessa maneira desde o início. Sempre pergunte "e se" e prepare-se para todas as respostas possíveis criando sistemas que resolvam o problema geral, não apenas o específico. Isso poderia levar à reutilização de todo um sistema.

6. **Planeje-se para o reuso**: a **reutilização economiza tempo e esforço**. Obter um alto nível de reutilização é, sem dúvida, o objetivo mais difícil de se conseguir no desenvolvimento de um sistema de software. A reutilização de códigos e designs foi considerada como um dos principais benefícios do uso de tecnologias orientadas a objetos. Existem muitas técnicas para realizar a reutilização em todos os níveis do processo de desenvolvimento do sistema. O planejamento antecipado para reutilização reduz o custo e aumenta o valor dos componentes reutilizáveis e dos sistemas nos quais eles são incorporados.

7. **Pense**: colocar o **pensamento claro e completo antes da ação quase sempre produz melhores resultados**. Quando você pensa em algo, é mais provável que você faça o certo. Você também ganha conhecimento sobre como fazer isso direito novamente. Se você pensa em algo e ainda faz errado, torna-se uma experiência valiosa. Um efeito colateral do pensamento é aprender a reconhecer quando você não sabe alguma coisa, e nesse ponto você pode pesquisar a resposta. Quando um pensamento claro entra em um sistema, o valor sai. A aplicação dos seis primeiros princípios requer pensamento intenso, para o qual as recompensas potenciais são enormes.

Outros princípios da engenharia de software são:

- **Rigor e Formalidade**: a engenharia de software é uma atividade criativa, mas que deve ser **realizada de maneira sistemática**, o Rigor e a Formalidade é um complemento necessário a criatividade que visa **aumentar a confiança dos desenvolvimentos de software**.

- **Separação de Interesses**: envolve **dominar a complexidade, separando os problemas principais e concentrando-se em um de cada vez** (dividir para conquistar), suporte a paralelização de atividades e separação das responsabilidades.

- **Modularidade ou Decomposição**: considera que um **sistema complexo pode ser divido em peças mais simples**, chamadas de módulos.

- **Abstração**: conceito que visa a **identificação de aspectos importantes de um fenômeno, ignorando os seus detalhes**. O tipo de abstração a ser aplicado depende do propósito. A abstração pode ser útil para realizar uma estimativa de custos de um projeto de software através de analise de similaridade com projetos passados.

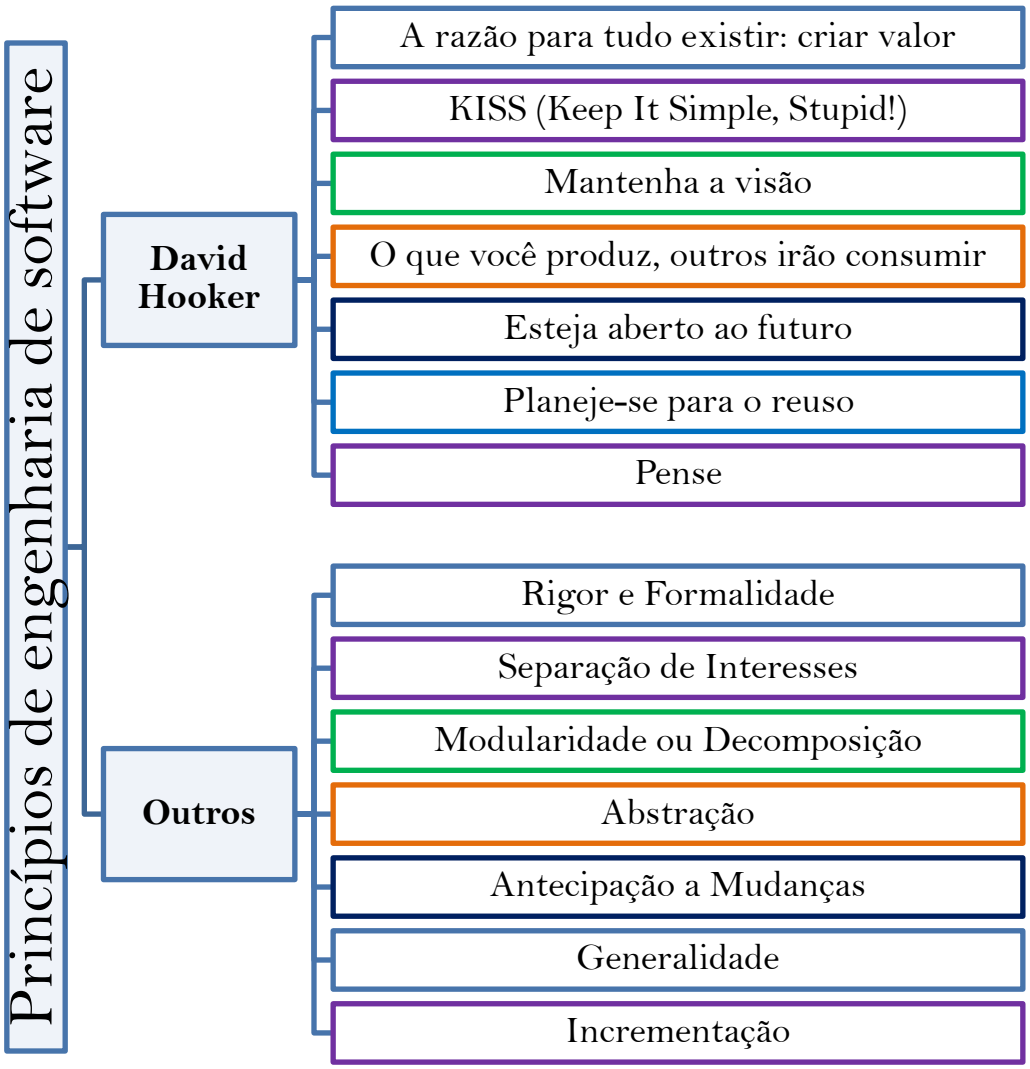
Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

- **Antecipação a Mudanças**: **suporte a evolução de um software considerando na arquitetura do software** aspectos relacionados ao processo de evolução e compatibilidade com mudanças futuras relacionadas ao domínio de aplicação do software.

- **Generalidade**: visa durante a resolução de um problema, **descobrir se ele é uma instância de um problema mais geral**, no qual a solução pode ser reutilizada em outros casos.

- **Incrementação**: relacionada a **evolução de um software através de incrementos estruturados**. Pode ser realizado através da entrega de subconjuntos de um sistema desde cedo, visando coletar o feedback dos usuários e adicionar funcionalidades de forma incremental. O processo incremental deve focar inicialmente na funcionalidade, para então, pensarmos na performance da solução, naturalmente o protótipo amadurecerá e se tornará um produto.

Em esquema tempos os seguintes princípios destacados:



Esquema 3 – Princípios da Engenharia de Software.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

3- (FCC - 2015 - TRE-RR - Analista Judiciário - Análise de Sistemas) Dentre os princípios da Engenharia de Software NÃO se encontra:

a) Um sistema de software existe por uma razão: para fornecer valor aos seus usuários. Todas as decisões devem ser tomadas com este princípio em mente.

b) Keep it Simple: todo projeto deve ser tão simples quanto possível, mas simples não significa rápido e mal feito. Simplificar precisa de muito raciocínio e trabalho em várias interações.

c) Sempre especifique, projete e implemente sabendo que mais alguém terá de entender o que se está fazendo. Mais alguém irá usar, manter, documentar ou precisará entender o sistema que uma pessoa desenvolve.

d) Reuso sempre poupa tempo e esforço, assim, conseguir um alto nível de reuso é a principal meta a ser alcançada no desenvolvimento. O reuso torna mais barato o custo do sistema, pois produz componentes reusáveis.

e) Raciocinar clara e completamente antes da ação quase sempre produz os melhores resultados. Quando o raciocínio límpido é aplicado ao sistema, é mais provável que funcione adequadamente.

Resolução:

Questão trata dos 7 princípios de David Hooker.

Vamos analisar cada uma das alternativas:

a) **Correto**: Segundo o princípio "**a razão para tudo existir"**, um sistema de software existe por uma razão: para fornecer valor aos seus usuários. Todas as decisões devem ser tomadas com este princípio em mente.

b) **Correto**: De acordo com o princípio **Keep it Simple**, todo projeto deve ser tão simples quanto possível, mas simples não significa rápido e mal feito. Simplificar precisa de muito raciocínio e trabalho em várias interações.

c) **Correto**: O princípio "**o que você produz, outros irão consumir"** solicita que você sempre especifique, projete e implemente sabendo que mais alguém terá de entender o que se está fazendo. Mais alguém irá usar, manter, documentar ou precisará entender o sistema que uma pessoa desenvolve.

d) **Incorreto**: **Reuso** ~~sempre~~ poupa tempo e esforço, assim, conseguir um alto nível de reuso é ~~a principal~~ **uma** meta a ser alcançada no desenvolvimento. O reuso torna mais barato o custo do sistema, pois produz componentes reusáveis. Não é razoável afirmar que sempre poupa tempo é nem que é a principal meta do desenvolvimento.

e) **Correto**: Conforme o princípio "**Pense",** raciocinar clara e completamente antes da ação quase sempre produz os melhores resultados. Quando o raciocínio límpido é aplicado ao sistema, é mais provável que funcione adequadamente.

Gabarito: Letra D.

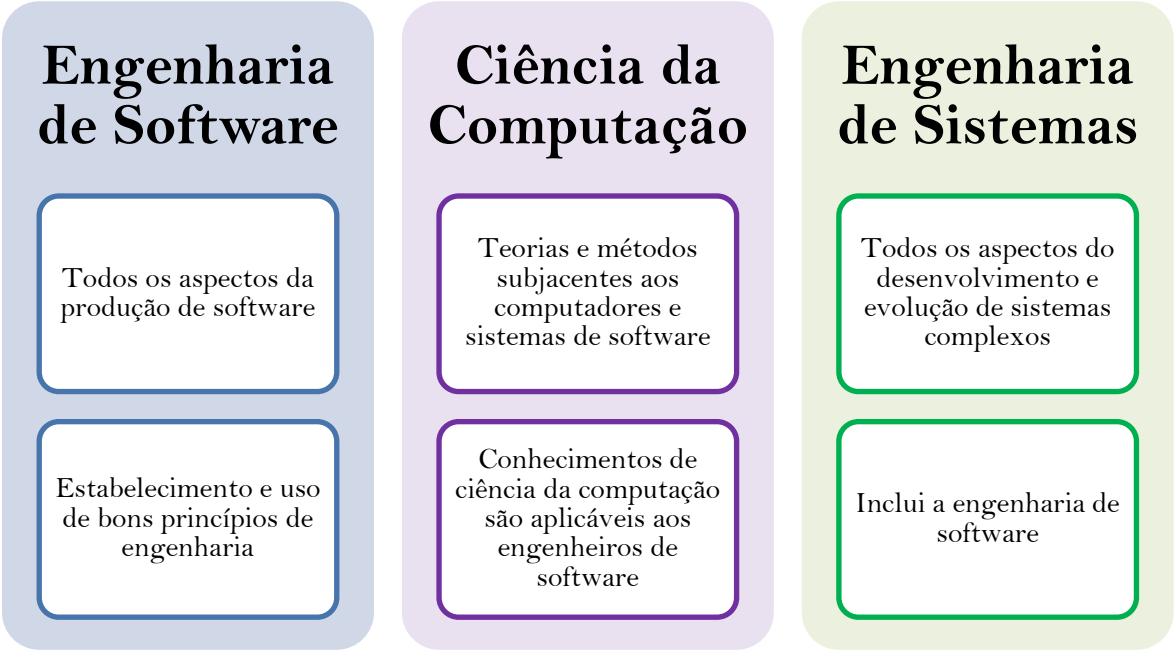
Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

1.3 Relação da engenharia de software com outras disciplinas

É importante destacar que a engenharia de software não se confunde com a engenharia de sistemas ou com a ciência da computação, mas se relaciona com elas.

A **ciência da computação** **se preocupa com as teorias e métodos subjacentes aos computadores e sistemas de software**, enquanto a **engenharia de software** **se preocupa com os problemas práticos da produção de software**. Algum conhecimento da **ciência da computação** é essencial para engenheiros de software da mesma forma que algum conhecimento de física é essencial para engenheiros elétricos. A teoria da **ciência da computação** é geralmente mais aplicável a programas relativamente pequenos. Teorias da ciência da computação nem sempre podem ser aplicadas a problemas grandes e complexos que exigem uma solução de software.

A **engenharia de sistemas** **se preocupa com todos os aspectos do desenvolvimento e evolução de sistemas complexos em que o software desempenha um papel importante**. A **engenharia de sistemas** está, portanto, preocupada com o desenvolvimento de hardware, projeto de políticas e processos e implantação de sistemas, bem como **engenharia de software**. Engenheiros de sistema estão envolvidos na especificação do sistema, definindo sua arquitetura geral e, então, integrando as diferentes partes para criar o sistema finalizado. Eles estão menos preocupados com a engenharia dos componentes do sistema (hardware, software, etc.).



Esquema 4 – Engenharia de Software e outras disciplinas.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

4- (FAURGS - 2018 - BANRISUL - Desenvolvimento de Sistemas) _____ se preocupa com todos os aspectos do desenvolvimento de sistemas computacionais, incluindo engenharia de hardware, software e processo; e _____ é uma disciplina da engenharia que se preocupa com todos aspectos da produção de software, desde os estágios iniciais da especificação do sistema até sua manutenção, quando o sistema já está sendo usado.

Assinale a alternativa que completa, correta e respectivamente, as lacunas do texto acima.

a) Engenharia de Domínio – Engenharia de Software

b) Engenharia Reversa – Engenharia de Sistemas

c) Engenharia de Sistemas – Engenharia de Domínio

d) Engenharia de Sistemas – Engenharia de Software

e) Engenharia Reversa – Engenharia de Domínio

Resolução:

A **engenharia de software** é uma **disciplina de engenharia que se preocupa com todos os aspectos da produção de software**, desde os estágios iniciais da especificação do sistema até a manutenção do sistema após sua utilização (SOMMERVILLE, 2011). A IEEE resume **engenharia de software** como a **aplicação de uma abordagem sistemática, disciplinada e quantificável para o desenvolvimento, operação e manutenção de software**.

Já a **engenharia de sistemas** **se preocupa com todos os aspectos do desenvolvimento e evolução de sistemas complexos em que o software desempenha um papel importante**. A **engenharia de sistemas** está, portanto, preocupada com o desenvolvimento de hardware, projeto de políticas e processos e implantação de sistemas, bem como **engenharia de software**. Engenheiros de sistema estão envolvidos na especificação do sistema, definindo sua arquitetura geral e, então, integrando as diferentes partes para criar o sistema finalizado. Eles estão menos preocupados com a engenharia dos componentes do sistema (hardware, software, etc.).

Assim, temos que:

Engenharia de Sistemas se preocupa com todos os aspectos do desenvolvimento de sistemas computacionais, incluindo engenharia de hardware, software e processo; e **Engenharia de Software** é uma disciplina da engenharia que se preocupa com todos aspectos da produção de software, desde os estágios iniciais da especificação do sistema até sua manutenção, quando o sistema já está sendo usado.

Gabarito: **Letra D**.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

5- (CESPE - 2015 - STJ - Técnico Judiciário - Tecnologia da Informação) A engenharia de software, cujo objetivo é o de apoiar o desenvolvimento de programas de computador, prioriza a programação individual e especializada, excluindo-se a documentação e as configurações do programa, que são tratados pela engenharia de sistemas e pela ciência da computação.

Resolução:

A **engenharia de software** é uma **disciplina de engenharia que se preocupa com todos os aspectos da produção de software**, desde os estágios iniciais da especificação do sistema até a manutenção do sistema após sua utilização. Logo, **não exclui a documentação e as configurações do programa**. Ademais **não prioriza a programação individual e especializada**, já que na grande maioria dos projetos, os softwares são desenvolvidos por equipes e não por indivíduos isolados.

Por sua vez, a **engenharia de sistemas** possui o escopo maior que o da engenharia de software, **se preocupando com todos os aspectos do desenvolvimento e evolução de sistemas complexos em que o software desempenha um papel importante**.

Já a **ciência da computação** **se preocupa com as teorias e métodos subjacentes aos computadores e sistemas de software**.

Gabarito: Errado.

6- (CESPE - 2013 - ANTT - Analista Administrativo - Desenvolvimento de Sistemas da Informação) A respeito de engenharia de software, julgue os itens de 91 a 100.

Engenheiros de softwares devem compreender a engenharia de sistemas, que aborda os aspectos de sistemas sociotécnicos.

Resolução:

A **engenharia de software** é uma **disciplina de engenharia que se preocupa com todos os aspectos da produção de software**, desde os estágios iniciais da especificação do sistema até a manutenção do sistema após sua utilização.

Por sua vez, a **engenharia de sistemas** possui o escopo maior que o da engenharia de software, **se preocupando com todos os aspectos do desenvolvimento e evolução de sistemas complexos em que o software desempenha um papel importante**.

É importante que um engenheiro de software tenha um entendimento do sistema no qual o software será inserido para que ele possa projetar o produto de software em conformidade com as requisitos e restrições adequados. Em resumo, é importante que o engenheiro de software tenha visão do todo (sistema) para projetar a parte (software).

Gabarito: Certo.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

2. CICLO DE VIDA E METODOLOGIAS DE SOFTWARE

2.1 Introdução ao ciclo de vida e metodologias

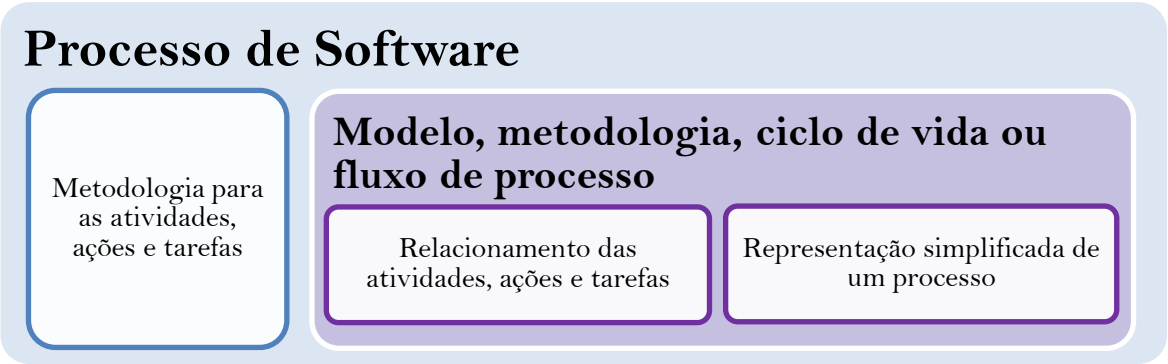
2.1.1 Conceitos iniciais

Quando se trabalha na elaboração de um produto ou sistema, é importante seguir uma série de passos previsíveis — um roteiro que ajude a criar um resultado de alta qualidade e dentro do prazo estabelecido. O roteiro é denominado **"processo de software"**.

Um **processo de software** é definido como uma **metodologia para as atividades, ações e tarefas necessárias para desenvolver um software de alta qualidade**.

Cada uma dessas atividades, ações e tarefas alocam-se dentro de uma **metodologia, modelo ou ciclo de vida** que **determina seu relacionamento com o processo e seu relacionamento umas com as outras**. Como um **modelo** descreve a **organização das atividades, tarefas e ações em relação à sequência e ao tempo**, podemos falar também em **fluxo de processo**.

Podemos dizer que um **modelo de processo de software** é **representação simplificada de um processo de software**. Os **modelos não são mutuamente exclusivos e são frequentemente usados em conjunto**, especialmente para o desenvolvimento de grandes sistemas.



Esquema 5 – Processo de Software.

DICA DO PROFESSOR!!!

Ressalto que, embora haja uma certa diferença entre os termos, as questões de concursos costumam trata-los como sinônimos e, portanto, não se apegue a distinção entre eles. Por exemplo, se a questão falar em modelo de processo cascata, ciclo de vida cascata, metodologia cascata, fluxo de processo cascata ou até mesmo processo cascata, estará se referindo a mesma coisa.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

2.1.2 Modelos de processos de negócios

Existe uma infinidade de modelos de processos, mas Pressman propõe uma estrutura genérica de cinco atividades a serem realizadas no desenvolvimento de software, são elas:

- **Comunicação**: antes de iniciar qualquer trabalho técnico, é de vital importância comunicar-se e colaborar com o cliente (e outros interessados). A intenção é **compreender os objetivos das partes interessadas e fazer o levantamento das necessidades** que ajudarão a definir as funções e características do software.

- **Planejamento**: qualquer jornada complicada pode ser simplificada caso exista um mapa. Um projeto de software é uma jornada complicada, e a atividade de planejamento cria um "mapa" que ajuda a guiar a equipe na sua jornada. O mapa — denominado plano de projeto de software — **define o trabalho de engenharia de software, descrevendo as tarefas técnicas a ser conduzidas, os riscos prováveis, os recursos que serão necessários, os produtos resultantes a ser produzidos e um cronograma de trabalho**.

- **Modelagem**: **cria-se um "esboço" de modo que se possa ter uma ideia do todo — qual será o seu aspecto em termos de arquitetura, como as partes constituintes se encaixarão e várias outras características**. Se necessário, refina-se o esboço com mais detalhes, numa tentativa de compreender melhor o problema e como resolvê-lo. Um engenheiro de software faz a mesma coisa criando modelos para melhor entender as necessidades do software e o projeto que irá atender a essas necessidades.

- **Construção**: combina **geração de código** (manual ou automatizada) **e testes** necessários para revelar erros na codificação.

- **Emprego ou Entrega**: o **software** (como uma entidade completa ou como um incremento parcialmente efetivado) **é entregue ao cliente, que avalia o produto entregue e fornece feedback**.

Esquema 6 – Modelo de Processo Genérico do Pressman.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Da mesma forma, Sommerville também propõe um modelo genérico destacando quatro atividades principais:

- **Especificação de software ou Engenharia de Requisitos**: processo de **compreensão e definição dos serviços requisitados do sistema e identificação de restrições** relativas à operação e ao desenvolvimento do sistema. A engenharia de requisitos é um estágio particularmente crítico do processo de software, pois erros nessa fase inevitavelmente geram problemas no projeto e na implementação do sistema.

- **Projeto e implementação**: envolve

  - **Projeto**: **descrição da estrutura** do software a ser implementado, dos modelos e estruturas de dados usados pelo sistema, das interfaces entre os componentes do sistema e, às vezes, dos algoritmos usados.

  - **Implementação**: processo de **conversão de uma especificação em um software executável**.

- **Validação (ou Verificação e Validação)**: tem a intenção de mostrar que o **sistema atende a sua especificação e satisfaz as necessidades do cliente**. Envolve testes, revisões e inspeções.

- **Evolução**: o **software é continuamente modificado, ao longo de seu ciclo de vida**, para responder a mudanças nos requisitos e necessidades do cliente. É visto como um processo contínuo ao desenvolvimento e não separado.

Esquema 7 – Modelo de Processo Genérico de Sommerville.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

7- (CESPE - 2019 - SLU-DF - Analista de Gestão de Resíduos Sólidos - Informática) Acerca de conceitos e disciplinas da engenharia de software, julgue o item que se segue.

O processo de desenvolvimento de software, independentemente do seu tamanho e da sua destinação, pode envolver atividades genéricas como comunicação, planejamento, modelagem, construção e uso.

Resolução:

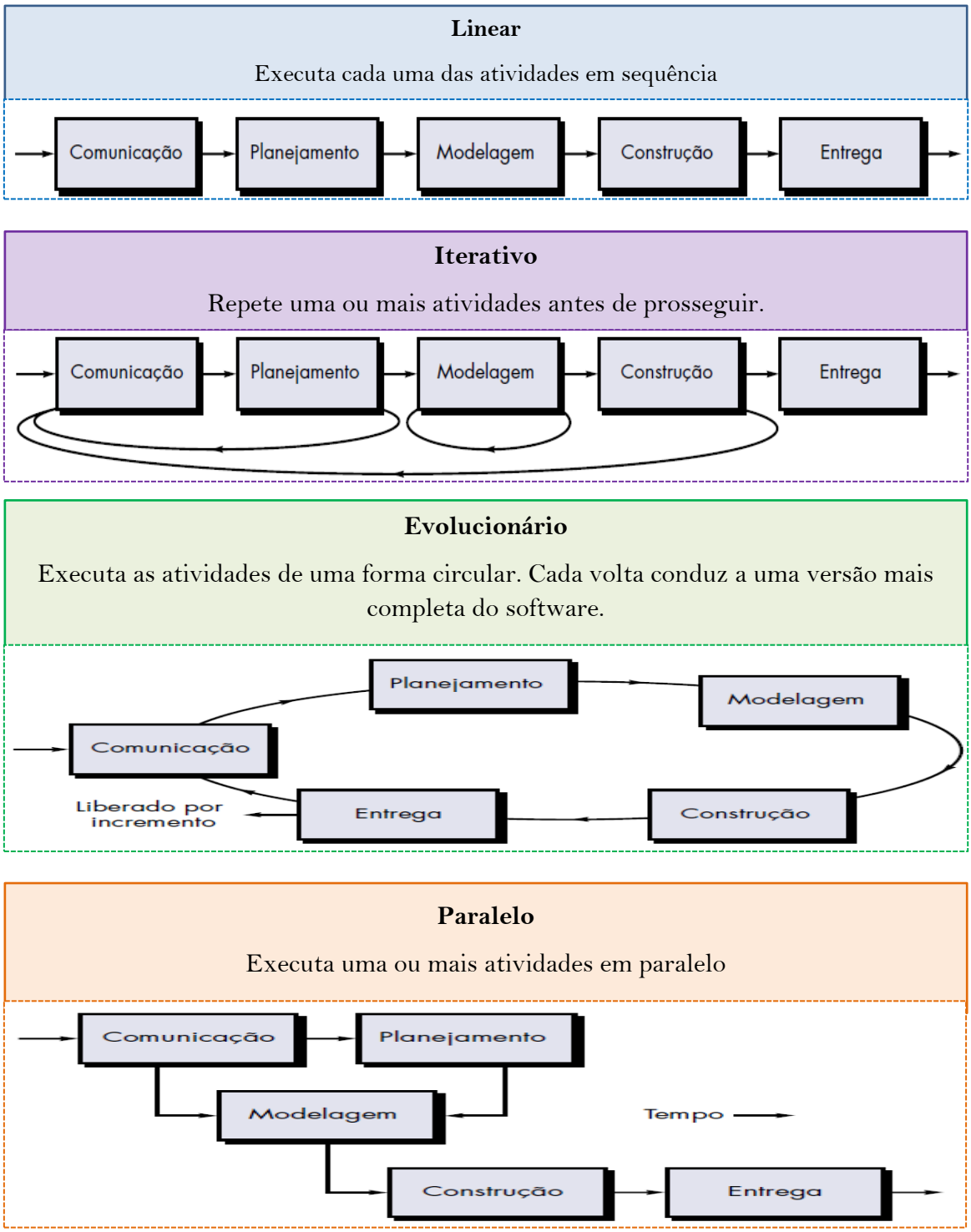
Existe uma infinidade de modelos de processos, mas Pressman propõe uma estrutura genérica de cinco atividades a serem realizadas no desenvolvimento de software, são elas:

Gabarito: Certo.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

2.1.3 Tipos de fluxos de processos

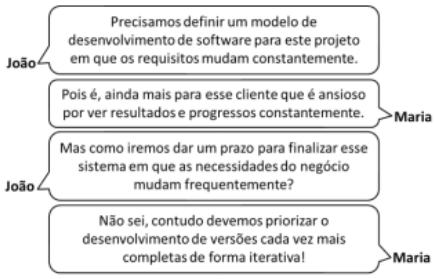
Dependendo da relação entre as atividades de um processo, podemos ter os seguintes tipos de fluxos de processos:



Esquema 8 – Tipos de Fluxos de Processo.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

8- (FGV - 2015 - PGE-RO - Analista da Procuradoria - Analista de Sistemas (Desenvolvimento)) Observe o seguinte diálogo entre dois Analistas de Sistemas:



O modelo de processo de software que atende às características do projeto que João e Maria irão desenvolver é:

a) Cascata;

b) Evolucionário;

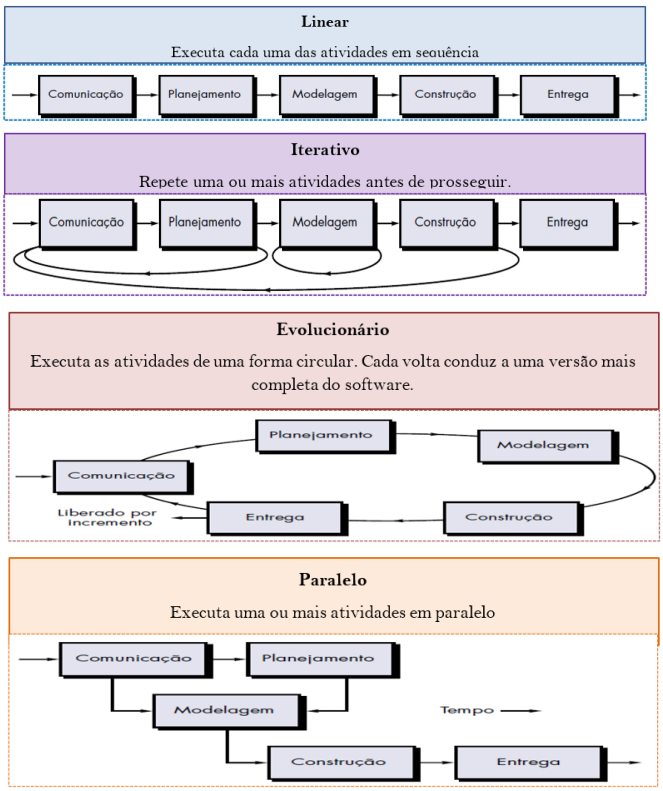
c) Codificação e correção;

d) Incremental;

e) Modelo em V.

Resolução:

Os tipos de fluxo podem ser:



Logo, o fluxo que envolve a construção de uma versão cada vez mais completa de um software e de forma iterativa é o evolucionário.

Gabarito: **Letra B.**

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

2.2 Modelos lineares ou sequenciais

2.2.1 Modelo cascata

O primeiro modelo do processo de desenvolvimento de software a ser publicado foi derivado de processos mais gerais da engenharia de sistemas. Por causa do **encadeamento entre uma fase e outra**, esse modelo é conhecido como **modelo em cascata, ciclo de vida clássico, ciclo de vida de software ou modelo "top-down"**. O **modelo em cascata** é um exemplo de um processo dirigido a planos — em princípio, **você deve planejar e programar todas as atividades do processo antes de começar a trabalhar nelas**.

O **modelo cascata** é um modelo **sequencial e sistemático**, em que, em princípio, o resultado de cada estágio é a aprovação de um ou mais documentos e o **estágio seguinte não deve ser iniciado até que a fase anterior seja concluída**.

O **modelo em cascata** é consistente com outros modelos de processos de engenharia, e a **documentação é produzida em cada fase do ciclo**. Dessa forma, o **processo torna-se visível**, e os gerentes podem monitorar o progresso de acordo com o plano de desenvolvimento. Seu maior problema é a **divisão inflexível do projeto em estágios distintos**. Os compromissos devem ser assumidos em um estágio inicial do processo, o que dificulta que atendam às mudanças de requisitos dos clientes.

Em princípio, o modelo em cascata deve ser **usado apenas quando os requisitos são bem compreendidos e pouco provavelmente venham a ser radicalmente alterados** durante o desenvolvimento do sistema. No entanto, o modelo em cascata reflete o tipo de processo usado em outros projetos de engenharia. Como é mais fácil usar um modelo de gerenciamento comum para todo o projeto, processos de software baseados no modelo em cascata ainda são comumente utilizados.

Embora seja o paradigma mais antigo da engenharia de software, tem se sujeitado a várias críticas, entre as quais.

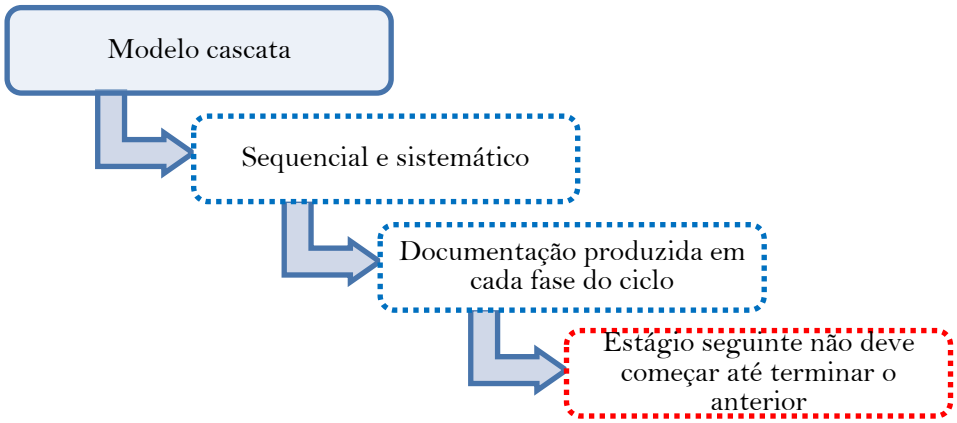
1. **Projetos reais raramente seguem o fluxo sequencial** que o modelo propõe. Embora o modelo linear possa conter iterações, ele o faz indiretamente. Como consequência, mudanças podem provocar confusão à medida que a equipe de projeto prossegue.

2. Frequentemente, é **difícil para o cliente estabelecer explicitamente todas as necessidades**. O modelo cascata requer isso e tem dificuldade para adequar a incerteza natural que existe no início de muitos projetos.

3. O **cliente deve ter paciência**. Uma versão operacional do(s) programa(s) não estará disponível antes de estarmos próximos do final do projeto. Um erro grave, se não detectado até o programa operacional ser revisto, pode ser desastroso.

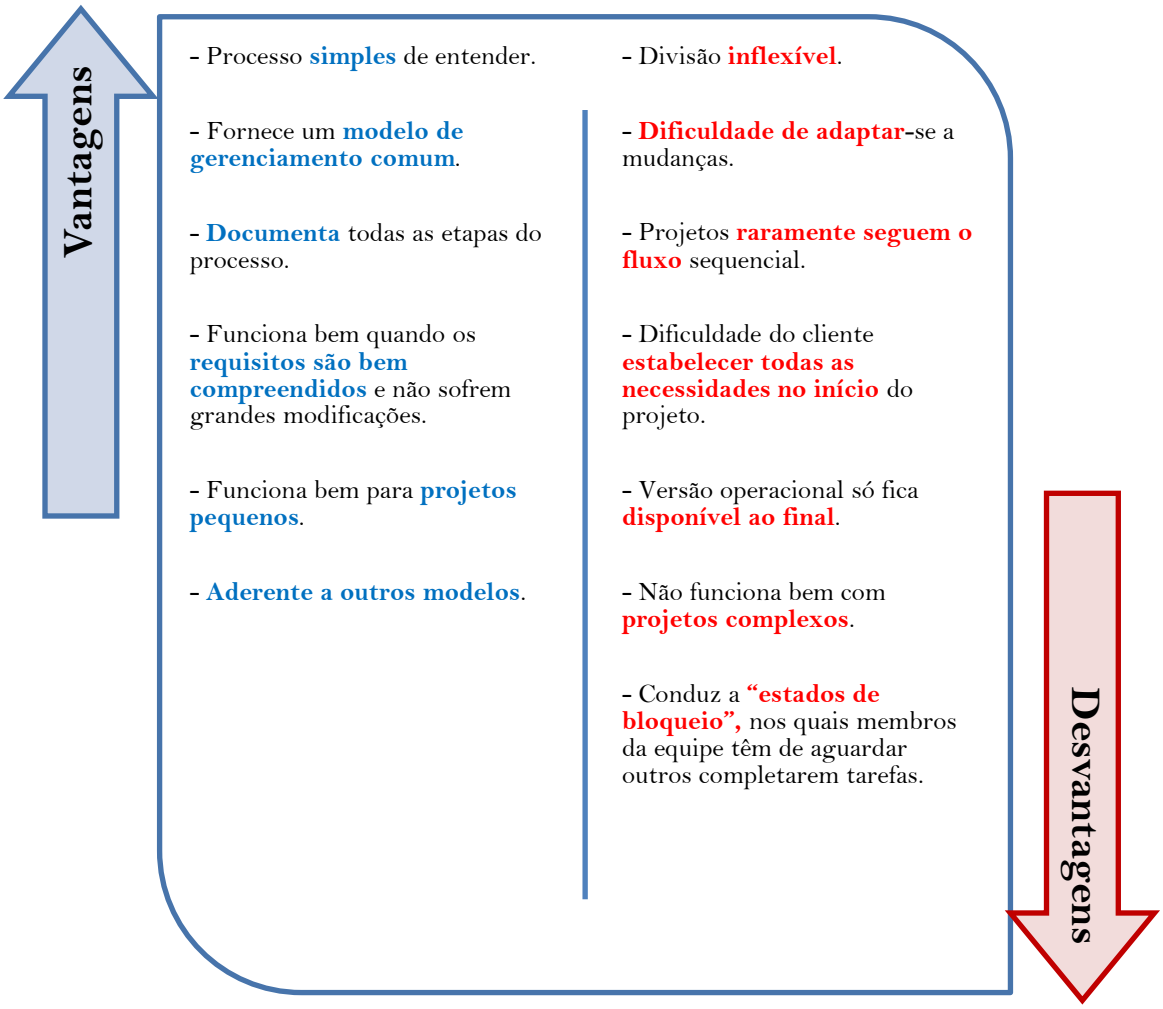
Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

De forma esquemática:



Esquema 9 – Modelo Cascata.

As vantagens e desvantagens podem ser ilustradas a seguir:

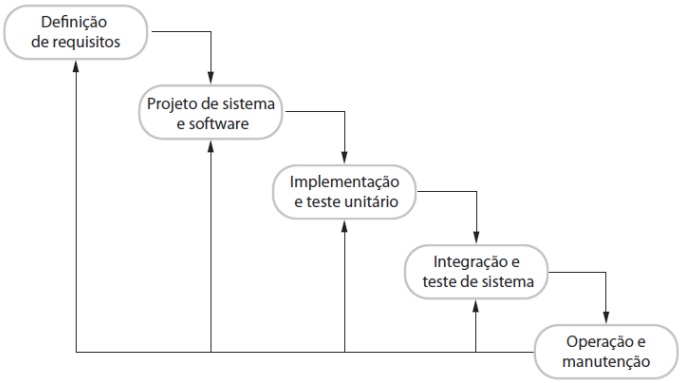


Esquema 10 – Vantagens e desvantagens do Modelo Cascata.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

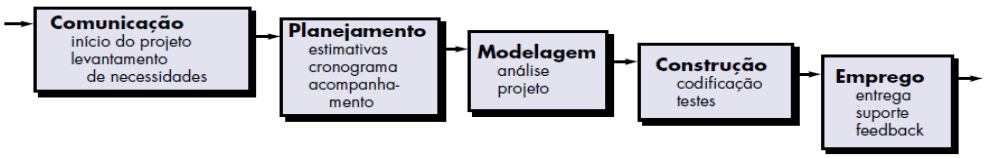
Segundo Sommerville, os principais estágios do modelo cascata são:

1. **Análise e definição de requisitos**: os serviços, restrições e metas do sistema são estabelecidos por meio de consulta aos usuários. Em seguida, são definidos em detalhes e funcionam como uma especificação do sistema.
2. **Projeto de sistema e software**: o processo de projeto de sistemas aloca os requisitos tanto para sistemas de hardware como para sistemas de software, por meio da definição de uma arquitetura geral do sistema. O projeto de software envolve identificação e descrição das abstrações fundamentais do sistema de software e seus relacionamentos.
3. **Implementação e teste unitário**: durante esse estágio, o projeto do software é desenvolvido como um conjunto de programas ou unidades de programa. O teste unitário envolve a verificação de que cada unidade atenda a sua especificação.
4. **Integração e teste de sistema**: as unidades individuais do programa ou programas são integradas e testadas como um sistema completo para assegurar que os requisitos do software tenham sido atendidos. Após o teste, o sistema de software é entregue ao cliente.
5. **Operação e manutenção**: normalmente (embora não necessariamente), essa é a fase mais longa do ciclo de vida. O sistema é instalado e colocado em uso. A manutenção envolve a correção de erros que não foram descobertos em estágios iniciais do ciclo de vida, com melhora da implementação das unidades do sistema e ampliação de seus serviços em resposta às descobertas de novos requisitos.



Esquema 11 – Estágios do modelo Cascata de Sommerville.

O modelo cascata proposto por Pressman segue as fases de seu ciclo de vida genérico dispostas de forma linear, conforme figura a seguir:



Esquema 12 – Fases do modelo Cascata de Pressman.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

9- (CESPE / CEBRASPE - 2021 - SERPRO - Analista - Especialização: Desenvolvimento de Sistemas) A respeito de processo orientado a reúso e ciclos de vida, julgue o item a seguir.

No modelo em cascata, dada a dificuldade natural para estabelecer todos os requisitos na fase inicial do projeto, os requisitos são definidos ao longo de todas as fases, acomodando-se gradualmente as incertezas e eventuais mudanças do projeto.

Resolução:

No modelo cascata, todos os requisitos devem ser estabelecidos de início. O **modelo cascata** é um modelo **sequencial e sistemático**, em que, em princípio, o resultado de cada estágio é a aprovação de um ou mais documentos e o **estágio seguinte não deve ser iniciado até que a fase anterior seja concluída**.

Gabarito: Errado.

10- (CESPE - 2019 - SLU-DF - Analista de Gestão de Resíduos Sólidos - Informática) Acerca de conceitos e disciplinas da engenharia de software, julgue o item que se segue.

No modelo de desenvolvimento de software em cascata, a abordagem é orientada ao risco e as tarefas são organizadas nos seguintes ciclos: determinar objetivos, identificar e resolver riscos, desenvolver e testar, e planejar a próxima iteração.

Resolução:

O **modelo cascata** **não é organizado em ciclos**, mas em fases sequenciais.

O **modelo cascata** é um modelo **sequencial e sistemático**, em que, em princípio, o resultado de cada estágio é a aprovação de um ou mais documentos e o **estágio seguinte não deve ser iniciado até que a fase anterior seja concluída**.

Gabarito: Errado.

11- (CESPE - 2018 - IPHAN - Analista I - Área 7) Com relação à engenharia de software, julgue o seguinte item.

No modelo em cascata, com exceção do sequenciamento dos estágios de requisitos e de análise, os demais são executados em paralelo, iniciando-se antes do término do estágio seguinte.

Resolução:

No **modelo em cascata** não temos a execução de estágios em paralelo. O **modelo cascata** é um modelo **sequencial e sistemático**, em que, em princípio, o resultado de cada estágio é a aprovação de um ou mais documentos e o **estágio seguinte não deve ser iniciado até que a fase anterior seja concluída**.

Gabarito: Errado

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

12- (INAZ do Pará - 2019 - CORE-SP - Analista de T.I) "O Modelo em Cascata (do inglês: Waterfall Model) é um modelo de desenvolvimento de software sequencial no qual o processo é visto como um fluir constante para frente (como uma cascata)"

Disponível em: https://pt.wikipedia.org/wiki/Modelo_em_cascata. Acesso em: 13.12.2018

No que tange ao processo de desenvolvimento de software em cascata, qual a afirmativa correta?

a) O modelo em cascata ou clássico também pode ser conhecido como "Bottom-UP".

b) Este modelo está defasado e não é mais utilizado, tendo sido descontinuado desde a década de 90.

c) As fases do modelo em cascata seguem a seguinte ordem: (1) Requerimento, (2) Verificação, (3) Projeto, (4) Implementação e (5) Manutenção.

d) As fases do modelo são como uma cascata, mantendo o fluxo do trabalho de cima para baixo, não podendo voltar às fases iniciais, somente pular etapas para frente.

e) A saída produzida em cada fase será utilizada como entrada da fase seguinte, tornando o modelo em cascata um modelo simples de entender e controlar.

Resolução:

Vamos analisar cada um dos itens:

a) **Incorreto**: O modelo em cascata ou clássico também pode ser conhecido como ~~"Bottom-UP"~~ **"Top-Down"**.

b) **Incorreto**: o modelo ainda pode ser utilizado. Como é mais fácil usar um modelo de gerenciamento comum para todo o projeto, processos de software baseados no modelo em cascata ainda são comumente utilizados. O cascata funciona bem em projetos pequenos e para projetos bem compreendidos.

c) **Incorreto**: As fases do modelo em cascata seguem a seguinte ordem: (1) Requerimento, ~~(2) Verificação, (3)~~ **(2)** Projeto, ~~(4)~~ **(3)** Implementação**,** **(4) Verificação** e (5) Manutenção.

d) **Incorreto**: a divisão do modelo cascata é inflexível e, portanto, não podemos voltar etapas, mas também **não podemos pular etapas**, devemos seguir rigidamente a sequencia de fases.

e) **Correto**: o **modelo cascata** é um modelo **sequencial e sistemático**, em que, em princípio, o resultado de cada estágio é a aprovação de um ou mais documentos e o **estágio seguinte não deve ser iniciado até que a fase anterior seja concluída**.

Gabarito: **Letra E**.

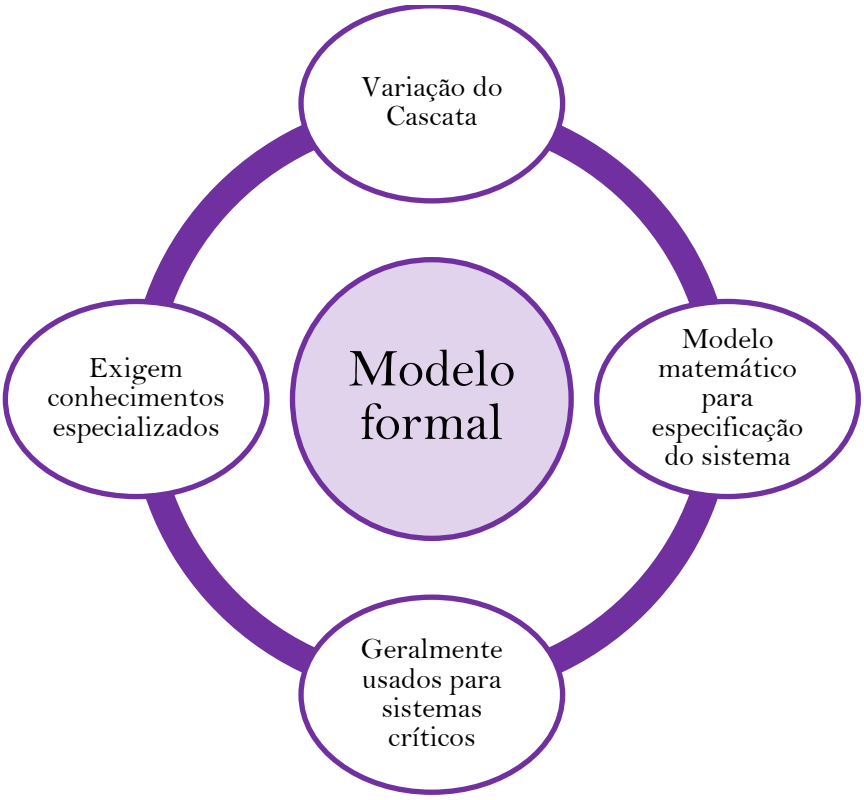
Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

2.2.2 Modelo formal

Uma variação importante do modelo em cascata é o **desenvolvimento formal** de um sistema, em que se **cria um modelo matemático de uma especificação do sistema**. Esse modelo é então refinado, usando transformações matemáticas que preservam sua consistência, em código executável. Partindo do pressuposto de que suas transformações matemáticas estão corretas, você pode, portanto, usar um forte argumento de que um programa gerado dessa forma é consistente com suas especificações.

Processos formais de desenvolvimento são particularmente adequados para o desenvolvimento de sistemas com requisitos rigorosos de segurança, confiabilidade e proteção. A abordagem formal simplifica a produção de casos de segurança ou proteção. Isso demonstra aos clientes ou reguladores que o sistema realmente cumpre com seus requisitos de proteção ou segurança.

Processos baseados em transformações formais são geralmente usados apenas no desenvolvimento de sistemas críticos de segurança ou de proteção. Eles **exigem conhecimentos especializados**. Para a maioria dos sistemas, esse processo não oferece custo-benefício significativo sobre outras abordagens para o desenvolvimento de sistemas.

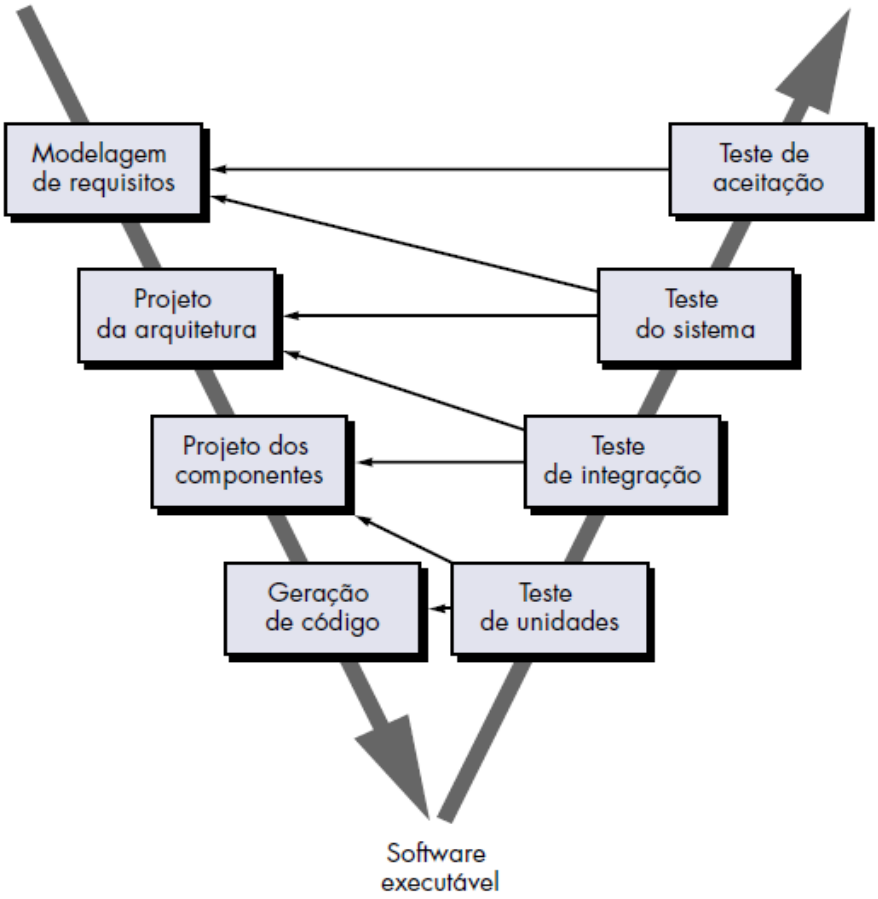


Esquema 13 – Modelos formais.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

2.2.3 Modelo V

Uma variação na representação do modelo cascata é denominada **modelo V**, que **descreve a relação entre ações de garantia da qualidade e as ações associadas à comunicação, modelagem e atividades de construção iniciais**.



Esquema 14 – Modelo V segundo Pressman.

À medida que a equipe de software desce em direção ao lado esquerdo do V, os requisitos básicos do problema são refinados em representações progressivamente cada vez mais detalhadas e técnicas do problema e de sua solução.

Uma vez que o código tenha sido gerado, a equipe se desloca para cima, no lado direito do V, realizando basicamente uma série de testes (ações de garantia da qualidade) que validem cada um dos modelos criados à medida que a equipe se desloca para baixo, no lado esquerdo do V.

Na realidade, não existe uma diferença fundamental entre o ciclo de vida clássico e o modelo V. O modelo V fornece uma forma para visualizar como a verificação e as ações de validação são aplicadas ao trabalho de engenharia anterior.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

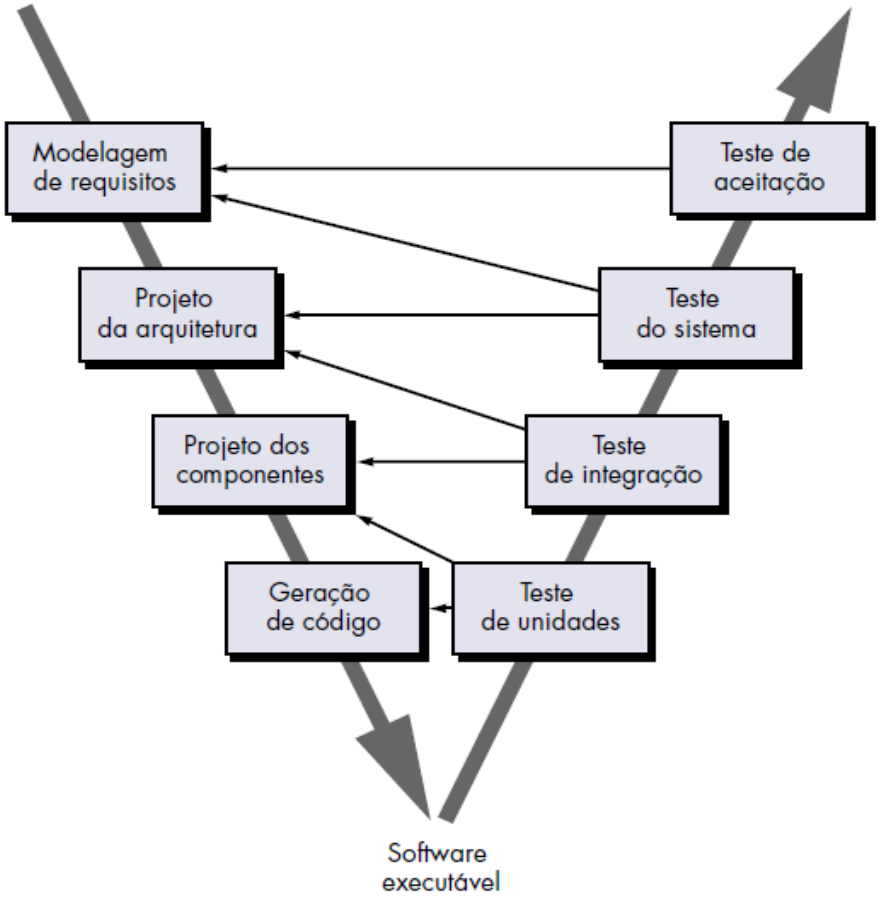
13- (INSTITUTO AOCP - 2021 - FUNPRESP-JUD - Analista de Tecnologia da Informação - Desenvolvimento de Sistemas) Em relação à gerenciamento do ciclo de vida do software, julgue o seguinte item.

O modelo de processo evolucionário é uma variação no modelo cascata. Esse modelo descreve a relação entre as ações de garantia da qualidade e as ações relacionadas à comunicação, à modelagem e às atividades de construção iniciais.

Resolução:

A assertiva se refere ao modelo v e não ao modelo evolucionário.

Uma variação na representação do modelo cascata é denominada **modelo V**, que **descreve a relação entre ações de garantia da qualidade e as ações associadas à comunicação, modelagem e atividades de construção iniciais**.



Gabarito: Errado.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

14- (FCC - 2017 - TRT - 11ª Região (AM e RR) - Analista Judiciário - Tecnologia da Informação) Na representação do modelo V, o objetivo principal do Teste de

a) Sistema é testar os componentes de acordo com as especificações e não de acordo com os requisitos.

b) Aceitação é verificar se o sistema está pronto para ser usado pelos usuários finais em ambiente de homologação.

c) Unidade é testar todos os componentes do sistema de forma integrada.

d) Integração é provar que o sistema não funciona.

e) Sistema é testar cada componente do sistema de forma isolada.

Resolução:

Vamos analisar cada um dos itens:

a) **Incorreto**: ~~Sistema~~ é testar os componentes de acordo com as especificações ~~e não~~ de acordo com os requisitos.

b) **Correto**: Aceitação é verificar se o sistema está pronto para ser usado pelos usuários finais em ambiente de homologação.

c) **Incorreto**: ~~Unidade~~ **Integração** é testar todos os componentes do sistema de forma integrada.

d) **Incorreto**: nenhum dos testes tem por objetivo provas que o sistema não funciona.

e) **Incorreto**: Sistema **De Unidades** é testar cada componente do sistema de forma isolada.

Gabarito: Letra B.

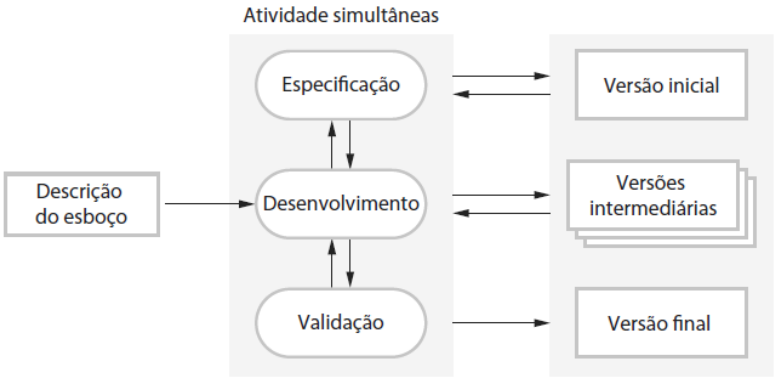
Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

2.3 Modelos incrementais

2.3.1 Desenvolvimento incremental

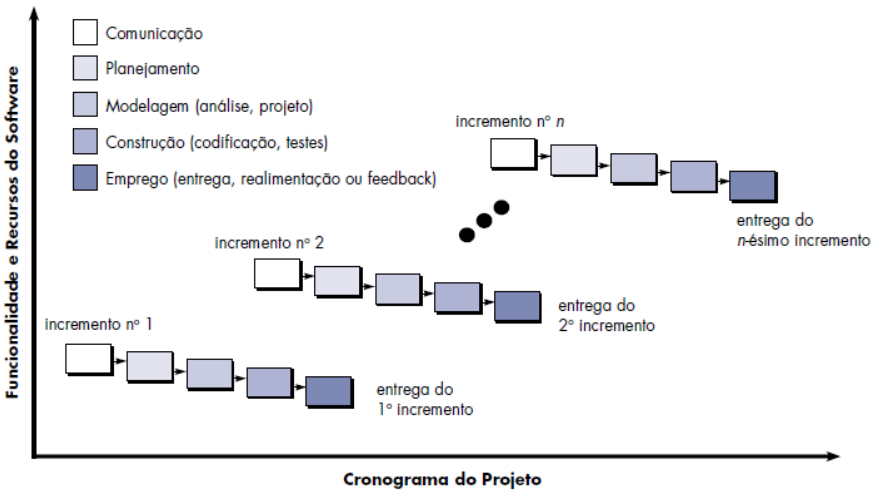
Em várias situações, os requisitos iniciais do software são razoavelmente bem definidos, entretanto, devido ao escopo geral do trabalho de desenvolvimento, o uso de um processo puramente linear não é utilizado. Pode ser necessário o rápido fornecimento de um determinado conjunto funcional aos usuários, para somente após esse fornecimento, refinar e expandir sua funcionalidade em versões de software posteriores. Em tais casos, pode-se optar por um modelo de processo projetado para desenvolver o software de forma incremental.

O **desenvolvimento incremental** é baseado na ideia de **desenvolver uma implementação inicial, expô-la aos comentários dos usuários e continuar por meio da criação de várias versões** até que um sistema adequado seja desenvolvido.



Esquema 15 – Desenvolvimento Incremental segundo Sommerville.

O **modelo incremental** aplica sequências lineares, de forma escalonada, à medida que o tempo vai avançando, em que **cada sequência linear gera "incrementais" (entregáveis/aprovados/liberados) do software**.



Esquema 16 – Modelo Incremental segundo Pressman.

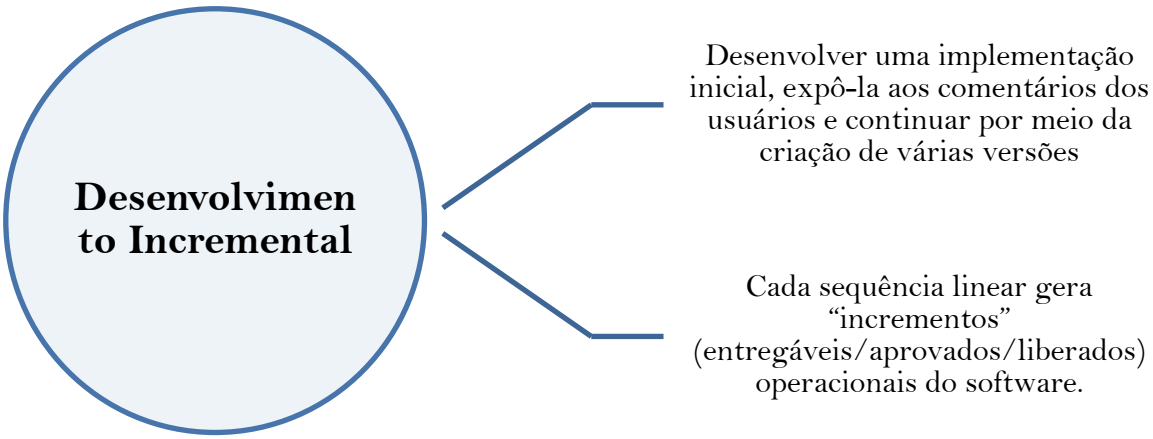
Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

O desenvolvimento incremental reflete a maneira como resolvemos os problemas. Raramente elaboramos uma completa solução do problema com antecedência; geralmente movemo-nos passo a passo em direção a uma solução, recuando quando percebemos que cometemos um erro. Ao desenvolver um software de **forma incremental**, é **mais barato e mais fácil fazer mudanças** no software durante seu desenvolvimento.

O **modelo de processo incremental** tem seu **foco voltado para a entrega de um produto operacional com cada incremento**. Os primeiros incrementos são versões seccionadas do produto final, mas eles realmente possuem capacidade para atender ao usuário e também oferecem uma plataforma para avaliação do usuário.

Quando se utiliza um **modelo incremental**, **frequentemente, o primeiro incremento é um produto essencial**. Isto é, as os requisitos básicos são atendidos, porém, muitos recursos complementares (alguns conhecidos, outros não) ainda não são entregues. Esse produto essencial é utilizado pelo cliente (ou passa por uma avaliação detalhada). Como resultado do uso e/ou avaliação, é desenvolvido um planejamento para o incremento seguinte. O planejamento já considera a modificação do produto essencial para melhor se adequar às necessidades do cliente e à entrega de recursos e funcionalidades adicionais. Esse processo é repetido após a liberação de cada incremento, até que seja produzido o produto completo.

O **desenvolvimento incremental** é **particularmente útil nos casos em que não há pessoal disponível para uma completa implementação** na época de vencimento do prazo para o projeto. Os primeiros incrementos podem ser implementados com número mais reduzido de pessoal. Se o produto essencial for bem acolhido, então um pessoal adicional (se necessário) poderá ser acrescentado para implementar o incremento seguinte. Além disso, os incrementos podem ser planejados para administrar riscos técnicos.



Esquema 17 – Características do Desenvolvimento Incremental.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

O desenvolvimento incremental tem **três vantagens** importantes quando comparado ao modelo em cascata:

1. O **custo de acomodar as mudanças nos requisitos do cliente é reduzido**. A quantidade de análise e documentação a ser refeita é muito menor do que o necessário no modelo em cascata.

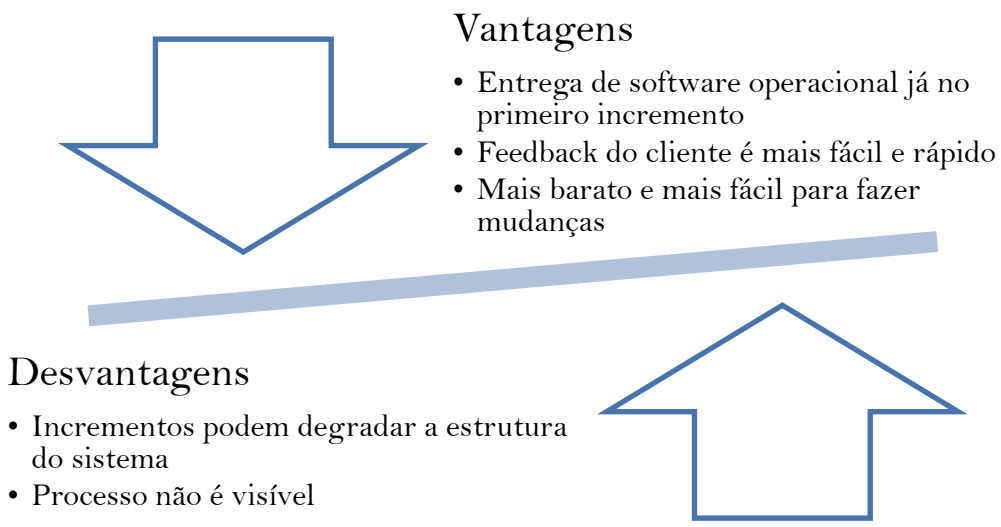
2. É **mais fácil obter feedback dos clientes sobre o desenvolvimento** que foi feito. Os clientes podem fazer comentários sobre as demonstrações do software e ver o quanto foi implementado. Os clientes têm dificuldade em avaliar a evolução por meio de documentos de projeto de software.

3. É possível **obter entrega e implementação rápida de um software útil ao cliente**, mesmo se toda a funcionalidade não for incluída. Os clientes podem usar e obter ganhos a partir do software inicial antes do que é possível com um processo em cascata.

Do ponto de vista do gerenciamento, a abordagem incremental tem **dois problemas**:

1. O **processo não é visível**. Os gerentes precisam de entregas regulares para mensurar o progresso. Se os sistemas são desenvolvidos com rapidez, não é economicamente viável produzir documentos que reflitam cada uma das versões do sistema.

2. A **estrutura do sistema tende a se degradar com a adição dos novos incrementos**. A menos que tempo e dinheiro sejam dispendidos em refatoração para melhoria do software, as constantes mudanças tendem a corromper sua estrutura. Incorporar futuras mudanças do software torna-se cada vez mais difícil e oneroso.



Esquema 18 – Vantagens e desvantagens do desenvolvimento incremental.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

É importante destacar que tecnicamente o modelo iterativo é diferente do modelo incremental, embora muitas vezes as bancas os tratam como um só modelo – o modelo iterativo e incremental. Vamos ver a diferença entre eles:

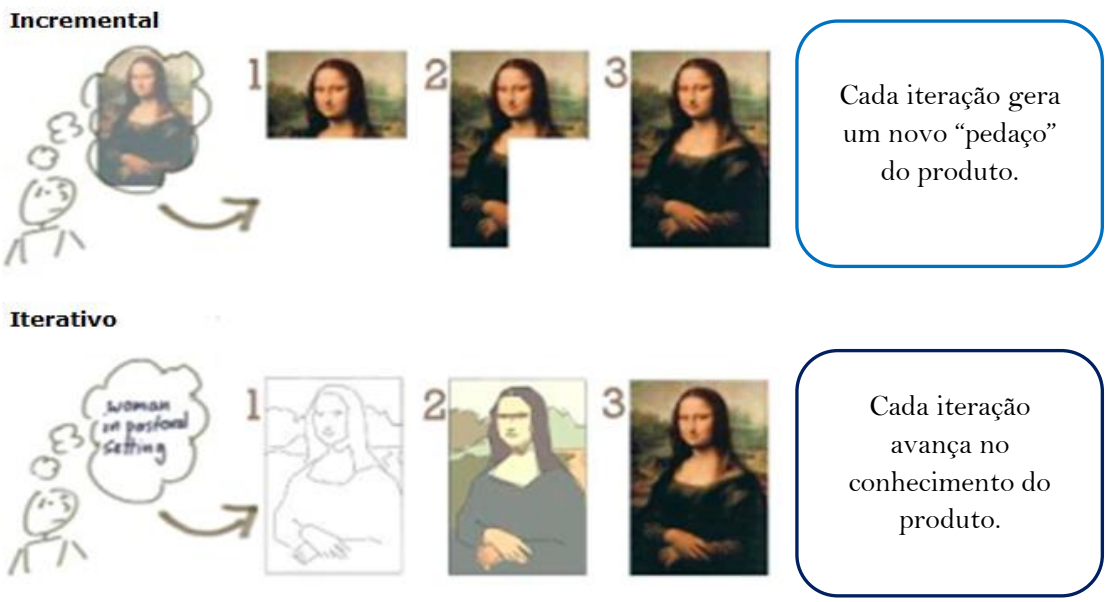
No **modelo de processo de software incremental** tem-se a ideia de se desenvolver uma implementação inicial, apresentá-la ao usuário e evoluir o software ao longo do tempo de acordo com o feedback do usuário. As fases do processo ocorrem em paralelo para cada um dos incrementos do software que estão em construção.

No modelo **iterativo**, a cada iteração uma ou mais atividades são executadas antes de se iniciar uma nova atividade, ou seja, o fluxo de execução do processo de desenvolvimento se repete.

O modelo iterativo muitas vezes é confundido com o modelo incremental. Um processo de desenvolvimento de software é **incremental** quando a **cada rodada é desenvolvido um "pedaço" inteiro do software**. Já no **iterativo**, **a cada iteração, se avança no conhecimento do projeto, novos requisitos são elicitados e a arquitetura do software é revisada**.

No modelo incremental, necessariamente teremos entrega de software ao término de uma rodada, o que não é verdade para o modelo iterativo.

Um exemplo bastante elucidativo utilizado para diferenciar esses modelos é a produção do quadro da Monalisa. No modelo incremental, há uma noção do todo, mas o quadro é desenhado uma parte (um incremento) de cada vez. No modelo iterativo, há apenas um esboço que vai sendo refinado ao longo das iterações.



Esquema 19 – Iterativo x Incremental.

Como já dissemos, geralmente os modelos costumam ser, ao mesmo tempo, iterativos (repetindo as fases do processo) e incrementais (gerando novas partes do produto).

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

15- (CESPE - 2018 - EBSERH - Analista de Tecnologia da Informação) Acerca de gerência de projetos, julgue o item a seguir.

No ciclo de vida incremental, o escopo do projeto é geralmente determinado no início, mas as estimativas de custo e de tempo são rotineiramente modificadas pela equipe à medida que o entendimento do produto final do projeto aumenta.

Resolução:

As características são do modelo iterativo e não do incremental.

Um processo de desenvolvimento de software é **incremental** quando a **cada rodada é desenvolvido um "pedaço" inteiro do software**. Já no **iterativo**, **a cada iteração, se avança no conhecimento do projeto, novos requisitos são elicitados e a arquitetura do software é revisada**.

Gabarito: Errado.

16- (INSTITUTO AOCP - 2020 - Prefeitura de Novo Hamburgo - RS - Analista de Desenvolvimento de Sistemas) Existem diversos modelos de desenvolvimento de software na literatura. Sabendo disso é correto afirmar que o modelo que se baseia na ideia de desenvolver uma versão inicial do produto, apresentá-la para os comentários dos clientes e continuar o desenvolvimento, por meio da criação de diversas versões, até que um produto final adequado seja alcançado, é o

a) modelo orientado a objetos.

b) modelo orientado ao reúso.

c) modelo incremental.

d) modelo cascata.

e) modelo híbrido.

Resolução:

O **desenvolvimento incremental** é baseado na ideia de **desenvolver uma implementação inicial, expô-la aos comentários dos usuários e continuar por meio da criação de várias versões** até que um sistema adequado seja desenvolvido. O **modelo incremental** aplica sequências lineares, de forma escalonada, à medida que o tempo vai avançando, em que **cada sequência linear gera "incrementais" do software**.

Gabarito: Letra C.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

2.3.2 Processo Unificado

O **Processo Unificado** é uma tentativa de aproveitar os melhores recursos e características dos modelos tradicionais de processo de software, mas caracterizando-os de modo a implementar muitos dos melhores princípios do desenvolvimento ágil de software.

O **Processo Unificado** reconhece a **importância da comunicação** com o cliente e de métodos racionalizados (sequencializados) para descrever a visão do cliente sobre um sistema (os **casos de uso**). Ele enfatiza o **importante papel da arquitetura de software** e "ajuda o arquiteto a manter o foco nas metas corretas, tais como compreensibilidade, confiança em mudanças futuras e reutilização". Ele sugere um fluxo de processo **iterativo e incremental**, proporcionando a sensação evolucionária que é essencial no desenvolvimento de software moderno.

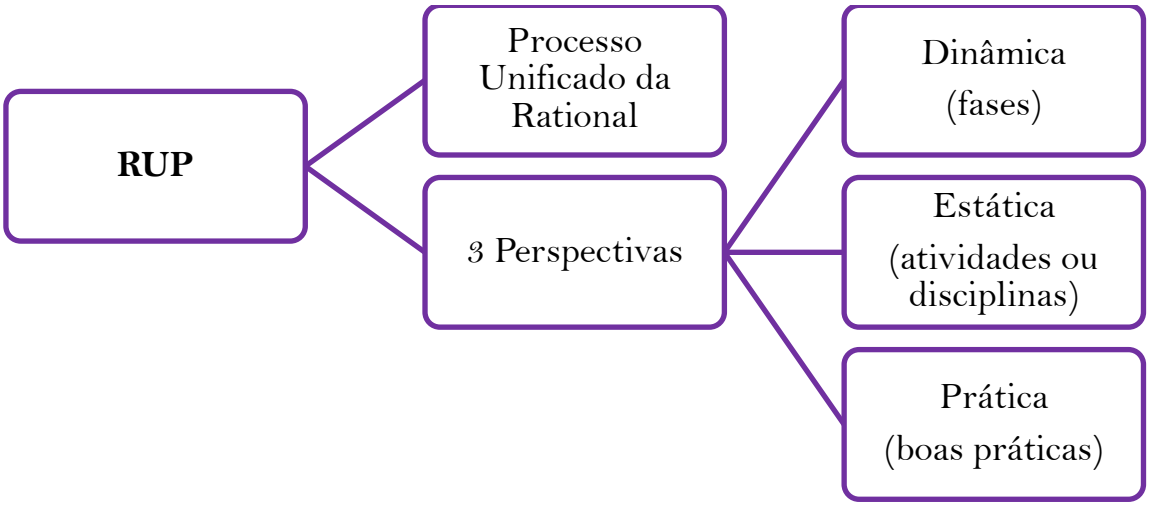
O **RUP, abreviação de Rational Unified Process (ou Processo Unificado da Rational)**, é um processo proprietário de Engenharia de software criado pela Rational Software Corporation, adquirida pela IBM, ganhando um novo nome IRUP que agora é uma abreviação de IBM Rational Unified Process.

O RUP reconhece que os modelos de processo convencionais apresentam uma visão única do processo. Em contrapartida, o **RUP** é normalmente descrito em **três perspectivas**:

1. Uma **perspectiva dinâmica**, que mostra as **fases do modelo** ao longo do tempo.

2. Uma **perspectiva estática**, que mostra as **atividades realizadas** no processo.

3. Uma **perspectiva prática**, que sugere **boas práticas** a serem usadas durante o processo.



Esquema 20 – Rational Unified Processo – RUP.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Perspectiva dinâmica ou fases do RUP

O **RUP** é um modelo que identifica **quatro fases distintas no processo de software**. No entanto, ao contrário do modelo em cascata, no qual as fases são equalizadas com as atividades do processo, **as fases do RUP** são **estreitamente relacionadas ao negócio**, **e não a assuntos técnicos**. As quatro fases do RUP são:

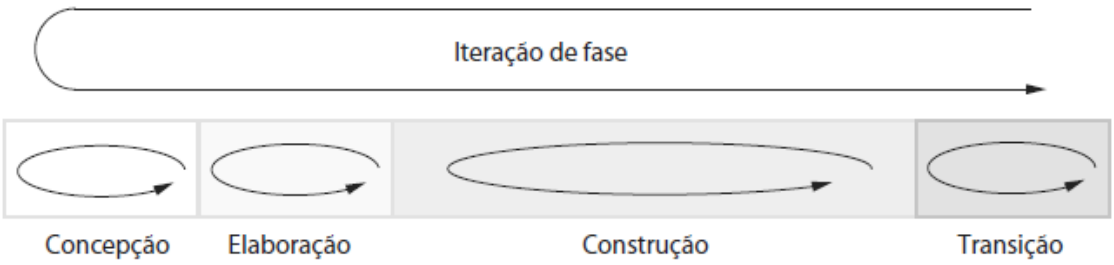
1. **Concepção ou Iniciação**: tem como objetivo **estabelecer um business case para o sistema**. Você deve identificar todas as entidades externas (pessoas e sistemas) que vão interagir com o sistema e definir as interações. Então, você deve usar essas informações para avaliar a contribuição do sistema para o negócio. Se essa contribuição for pequena, então o projeto poderá ser cancelado depois dessa fase.

2. **Elaboração**: tem como metas **desenvolver uma compreensão do problema dominante, estabelecer um framework da arquitetura para o sistema, desenvolver o plano do projeto e identificar os maiores riscos** do projeto. No fim dessa fase, você deve ter um modelo de requisitos para o sistema, que pode ser um conjunto de casos de uso da UML, uma descrição da arquitetura ou um plano de desenvolvimento do software.

3. **Construção**: **envolve projeto, programação e testes** do sistema. Durante essa fase, as partes do sistema são desenvolvidas em paralelo e integradas. Na conclusão dessa fase, você deve ter um sistema de software já funcionando, bem como a documentação associada pronta para ser entregue aos usuários.

4. **Transição**: implica **transferência do sistema da comunidade de desenvolvimento para a comunidade de usuários e em seu funcionamento em um ambiente real**. Isso é ignorado na maioria dos modelos de processo de software, mas é, de fato, uma atividade cara e, às vezes, problemática. Na conclusão dessa fase, você deve ter um sistema de software documentado e funcionando corretamente em seu ambiente operacional.

No RUP, a iteração é apoiada de duas maneiras. Cada fase pode ser executada de forma iterativa com os resultados desenvolvidos de forma incremental. Além disso, todo o conjunto de fases também pode ser executado de forma incremental.



Esquema 21 – Perspectiva Dinâmica ou fases do RUP.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Perspectiva estática ou disciplinas do RUP

A visão estática do RUP prioriza as **atividades que ocorrem durante o processo de desenvolvimento**. Na descrição do RUP, essas são chamadas **workflows ou disciplinas**. Existem **seis workflows centrais**, identificadas no processo, e **três workflows de apoio**. O RUP foi projetado em conjunto com a UML, assim, a descrição do workflow é orientada em torno de modelos associados à UML, como modelos de sequência, modelos de objetos etc.

Os workflows ou disciplinas do RUP são:

Workflow/Disciplina		Descrição
Disciplinas principais, centrais ou de engenharia de software	**Modelagem de negócios**	Os processos de negócio são modelados por meio de casos de uso de negócios.
	Requisitos	Atores que interagem com o sistema são identificados e casos de uso são desenvolvidos para modelar os requisitos do sistema.
	Análise e projeto	Um modelo de projeto é criado e documentado com modelos de arquitetura, modelos de componentes, modelos de objetos e modelos de sequência.
	Implementação	Os componentes do sistema são implementados e estruturados em subsistemas de implementação. A geração automática de código a partir de modelos de projeto ajuda a acelerar esse processo.
	Teste	O teste é um processo iterativo que é feito em conjunto com a implementação. O teste do sistema segue a conclusão da implementação.
	Implantação	Um release do produto é criado, distribuído aos usuários e instalado em seu local de trabalho.
Disciplinas de apoio/suporte	**Gerenciamento de configuração e mudanças**	Gerencia as mudanças do sistema
	Gerenciamento de projeto	Gerencia o desenvolvimento do sistema
	Meio ambiente	Relacionado com a disponibilização de ferramentas apropriadas para a equipe de desenvolvimento de software.

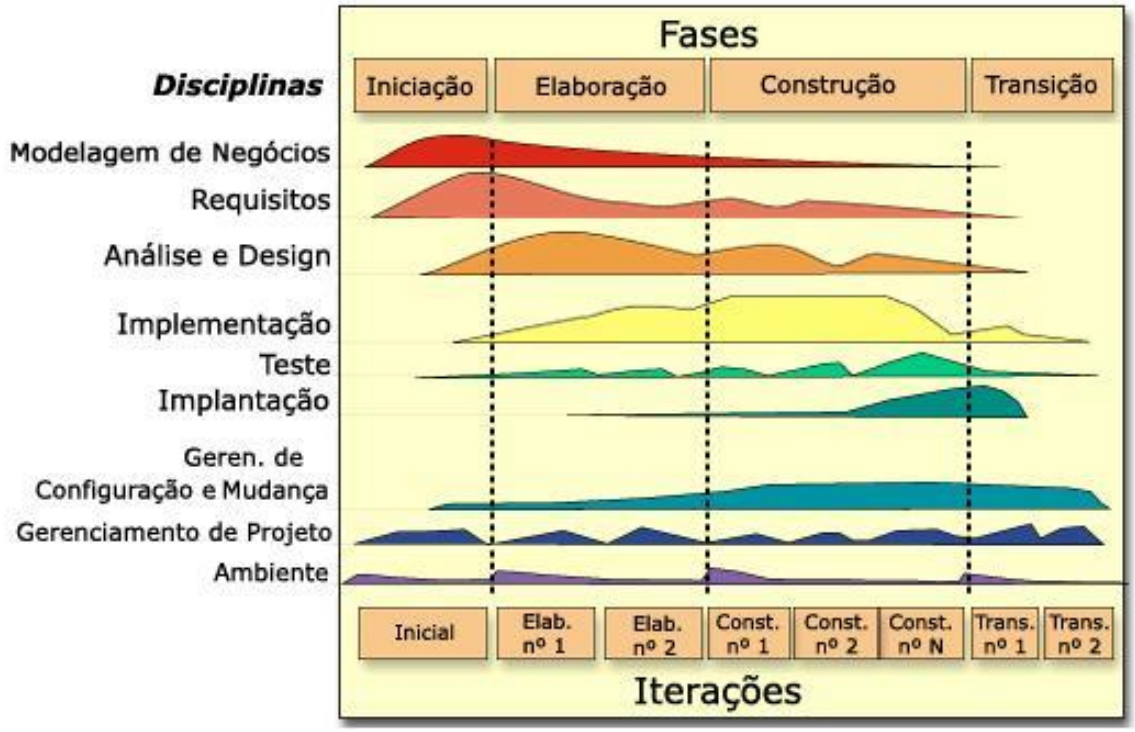
Esquema 22 – Perspectiva estática ou disciplinas do RUP.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Relação entre a perspectiva dinâmica e a prática

A vantagem de proporcionar visões estáticas e dinâmicas é que as fases do processo de desenvolvimento não estão associadas a workflows específicos. Ao menos em princípio, todos os workflows do RUP podem estar ativos em todas as fases do processo.

Uma figura muito utilizada para relacionar as fases com a disciplinas do RUP é o chamado gráfico de baleias. Neste gráfico, é possível avaliar em que grau as disciplinas são executadas nas fases:



Esquema 23 – Fases e Disciplinas do RUP.

Note, por exemplo, que nas fases iniciais, provavelmente, maiores esforços serão empenhados em workflows, como modelagem de negócios e requisitos, e, nas fases posteriores, no teste e na implantação.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Perspectiva prática ou boas práticas do RUP

A perspectiva prática sobre o RUP descreve as boas práticas da engenharia de software que são recomendadas para uso no desenvolvimento de sistemas. Seis boas práticas fundamentais são recomendadas:

1. **Desenvolver software iterativamente**: planejar os incrementos do sistema com base nas prioridades do cliente e desenvolver os recursos de alta prioridade no início do processo de desenvolvimento.

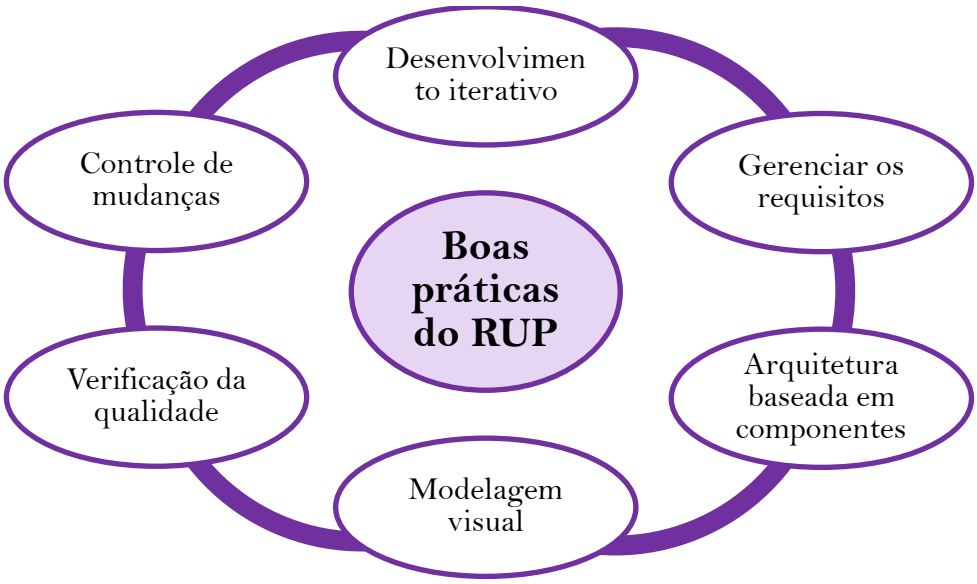
2. **Gerenciar os requisitos**: documentar explicitamente os requisitos do cliente e acompanhar suas mudanças. Analisar o impacto das mudanças no sistema antes de aceitá-las.

3. **Usar arquiteturas baseadas em componentes**: estruturar a arquitetura do sistema em componentes.

4. **Modelar o software visualmente**: usar modelos gráficos da UML para apresentar visões estáticas e dinâmicas do software.

5. **Verificar a qualidade do software**: assegurar que o software atenda aos padrões de qualidade organizacional.

6. **Controlar as mudanças do software**: gerenciar as mudanças do software, usando um sistema de gerenciamento de mudanças e procedimentos e ferramentas de gerenciamento de configuração.



Esquema 24 – Perspectiva prática ou boas práticas do RUP.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

17- (CESPE - 2019 - MPC-PA - Assistente Ministerial de Informática) O processo unificado (RUP) reúne boas práticas de especificação e de projeto de sistemas, sendo um modelo genérico de processo organizado em fases que podem gerar um conjunto de produtos de trabalho. Considerando isso, assinale a opção que identifica a fase do RUP na qual devem estar incluídos o refinamento e a expansão dos casos de uso preliminares, dos requisitos não funcionais e da descrição da arquitetura do software.

a) concepção

b) construção

c) elaboração

d) produção

e) transição

Resolução:

Vamos analisar cada um dos itens:

a) Incorreto: a concepção tem como objetivo **estabelecer um business case para o sistema.** Você deve identificar todas as entidades externas (pessoas e sistemas) que vão interagir com o sistema e definir as interações. Então, você deve usar essas informações para avaliar a contribuição do sistema para o negócio. Se essa contribuição for pequena, então o projeto poderá ser cancelado depois dessa fase.

b) Incorreto: a construção **envolve projeto, programação e testes** do sistema. Durante essa fase, as partes do sistema são desenvolvidas em paralelo e integradas. Na conclusão dessa fase, você deve ter um sistema de software já funcionando, bem como a documentação associada pronta para ser entregue aos usuários.

c) Correto: a elaboração tem como metas desenvolver uma compreensão do problema dominante, estabelecer um framework da arquitetura para o sistema, desenvolver o plano do projeto e identificar os maiores riscos do projeto. No fim dessa fase, você deve ter um modelo de requisitos para o sistema, que pode ser um conjunto de casos de uso da UML, uma descrição da arquitetura ou um plano de desenvolvimento do software.

d) Incorreto: produção não é fase do RUP.

e) Incorreto: a transição implica a transferência do sistema da comunidade de desenvolvimento para a comunidade de usuários e em seu funcionamento em um ambiente real.

Gabarito: Letra C.

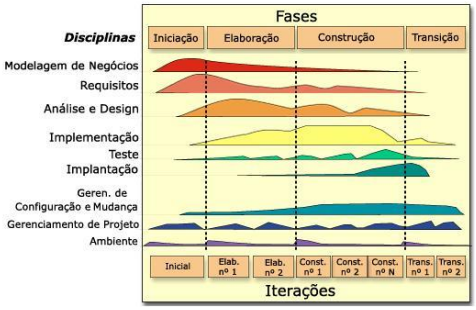
Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

18- (CESPE - 2019 - SLU-DF - Analista de Gestão de Resíduos Sólidos - Informática) Acerca de conceitos e disciplinas da engenharia de software, julgue o item que se segue.

No processo unificado, requisitos é a disciplina que demanda maior esforço nas fases de elaboração e construção de software.

Resolução:

No gráfico a seguir, é possível avaliar em que grau as disciplinas são executadas nas fases:



Logo, podemos perceber que a disciplina de Requisitos ocorre em maior grau nas fases de Iniciação e de Elaboração.

Gabarito: Errado.

19- (CESPE - 2018 - ABIN - Oficial Técnico de Inteligência - Área 9) Com relação à qualidade de software, julgue o item a seguir.

No processo unificado de desenvolvimento de software, a fase de construção está relacionada à entrada do sistema em funcionamento, o que requer entendimento do domínio do problema e um sistema de software documentado que funcione corretamente no ambiente operacional.

Resolução:

A fase de **transição** é que está relacionada à entrada do sistema em funcionamento. Vejamos quais são as quatro fases do RUP:

1. **Concepção ou Iniciação**: tem como objetivo **estabelecer um business case para o sistema**.
2. **Elaboração**: tem como metas **desenvolver uma compreensão do problema dominante, estabelecer um framework da arquitetura para o sistema, desenvolver o plano do projeto e identificar os maiores riscos** do projeto.
3. **Construção**: **envolve projeto, programação e testes** do sistema.
4. **Transição**: implica **transferência do sistema da comunidade de desenvolvimento para a comunidade de usuários e em seu funcionamento em um ambiente real**.

Gabarito: Errado.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

20- (FCC - 2019 - Prefeitura de Manaus - AM - Assistente Técnico de Tecnologia da Informação - Programador) Considerando o uso do RUP (Rational Unified Process), deve-se considerar que, segundo essa técnica,

a) a disciplina Modelagem de Negócio apresenta maior atividade na fase de Construção.

b) o número de iterações em cada uma de suas quatro fases é variável, conforme o projeto.

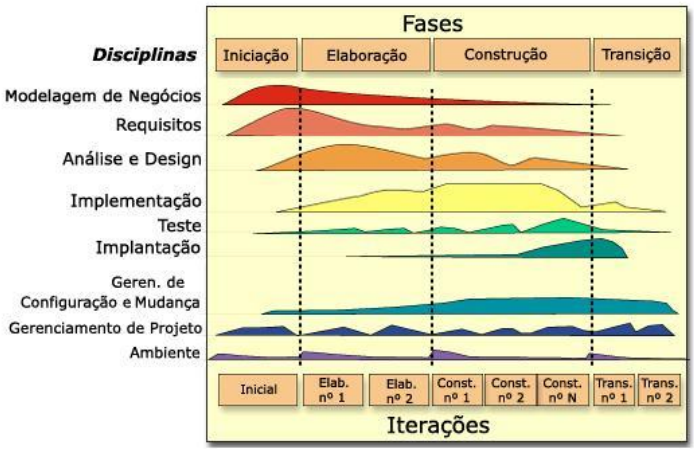
c) a disciplina Requisitos apresenta menor atividade na fase de Concepção (Inception).

d) a disciplina de Teste não é executada na fase de Elaboração.

e) a disciplina Configuração e Gerenciamento de Mudanças não é executada na fase de Elaboração.

Resolução:

Questão pode ser resolvida com o famoso diagrama das baleias:



Vamos analisar cada um dos itens:

a) **Incorreto**: a disciplina Modelagem de Negócio apresenta maior atividade na fase de ~~Construção~~ **Iniciação**.

b) **Correto**: o número de iterações em cada uma de suas quatro fases é variável, conforme o projeto. O gráfico apenas apresenta uma ideia de iterações, mas a quantidade dessas iterações não é fixa.

c) **Incorreto**: a disciplina Requisitos apresenta menor atividade na fase de ~~Concepção (Inception)~~ **Transição (Transition)**.

d) **Incorreto**: a disciplina de Teste ~~não~~ é executada na fase de Elaboração.

e) **Incorreto**: a disciplina Configuração e Gerenciamento de Mudanças ~~não~~ é executada na fase de Elaboração.

Gabarito: Letra B.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

2.3.3 Rapid Application Development - RAD

O **Rapid Application Development (RAD) ou Desenvolvimento Rápido de Aplicação**, é um modelo de processo **iterativo e incremental que enfatiza um ciclo de desenvolvimento extremamente curto (entre 60 e 90 dias)**. Comumente envolve o uso de ferramentas de programação de banco de dados e de apoio ao desenvolvimento, como geradores de telas e relatórios.

O RAD é composto por 5 fases:

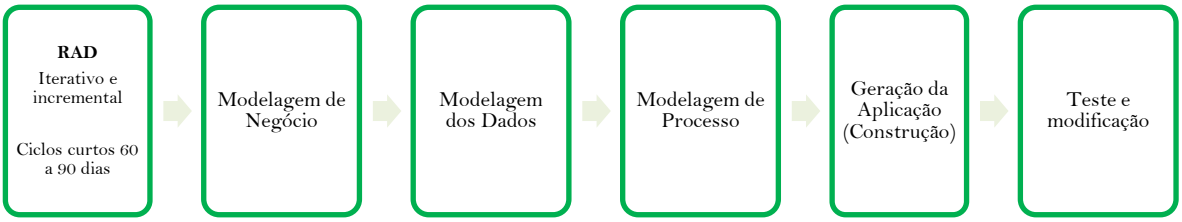
1. **Modelagem de Negócio**: o **fluxo de informações entre as funções de negócio é modelado** de modo a responder às seguintes questões: Que informação direciona o processo de negócio? - Que informação é gerada? Quem a gera? Para onde vai à informação? Quem a processa? Na modelagem de negócio são **levantados os processos suportados pelo sistema**.

2. **Modelagem dos Dados**: responde a um conjunto de questões específicas que são relevantes a qualquer aplicação. O fluxo de informação definido na fase de modelagem de negócio é refinado de forma a **extrair os principais objetos de dados a serem processados pelo sistema**, qual a composição de cada um dos objetos de dados, onde costumam ficar, qual a relação entre eles e quais as relações entre os objetos e os processos que os transformam.

3. **Modelagem do Processo**: os objetos de dados são transformados para **conseguir o fluxo necessário para implementar uma função do negócio**. Descrições do processamento são criadas para adicionar, modificar, descartar ou recuperar um objeto de dados.

4. **Geração da Aplicação ou Construção**: o RAD considera o **uso de técnicas de quarta geração** (especificação para máquina em linguagem próxima a natural), trabalha com a **reutilização de componentes** ou **cria componentes reusáveis**. São usadas ferramentas automatizadas para facilitar a construção do software.

5. **Teste e Modificação**: como o processo do RAD enfatiza o reuso, muitos componentes já estão testados, isso reduz o tempo total de teste. Todavia os **novos componentes devem ser testados e todas as interfaces devem ser exaustivamente exercitadas**.



Esquema 25 – Rapid Application Development (RAD).

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

2.4 Modelos evolucionários

Modelos evolucionários são iterativos. Apresentam características que possibilitam desenvolver versões cada vez mais completas do software.

Como um **modelo iterativo**, é embasado na ideia de **desenvolvimento de uma implementação inicial, expondo o resultado aos comentários do usuário e refinando esse resultado por meio de diversas versões**, até que seja desenvolvido um sistema adequado.

Os modelos evolucionários mais comuns são o **de prototipação e o espiral**. Além desses, destaca-se também o **modelo concorrente**.

2.4.1 Prototipação

Frequentemente, o cliente define uma série de objetivos gerais para o software, mas não identifica, detalhadamente, os requisitos para funções e recursos. Em outros casos, o desenvolvedor encontra-se inseguro quanto à eficiência de um algoritmo, quanto à adaptabilidade de um sistema operacional ou quanto à forma em que deva ocorrer a interação homem/máquina. Em situações como essas, e em muitas outras, o paradigma de **prototipação** pode ser a melhor escolha de abordagem.

Embora a **prototipação** possa ser **utilizada como um modelo de processo isolado** (stand-alone process) é mais comumente utilizada como **uma técnica passível de ser implementada no contexto de qualquer modelo de processo**. Independentemente da forma como é aplicado, quando os requisitos estão obscuros, o paradigma da **prototipação auxilia os interessados a compreender melhor o que está para ser construído**.

Um **protótipo** é uma **versão inicial de um sistema de software, usado para demonstrar conceitos, experimentar opções de projeto e descobrir mais sobre o problema e suas possíveis soluções**. O desenvolvimento rápido e iterativo do protótipo é essencial para que os custos sejam controlados e as partes interessadas do sistema possam experimentá-lo no início do processo de software.

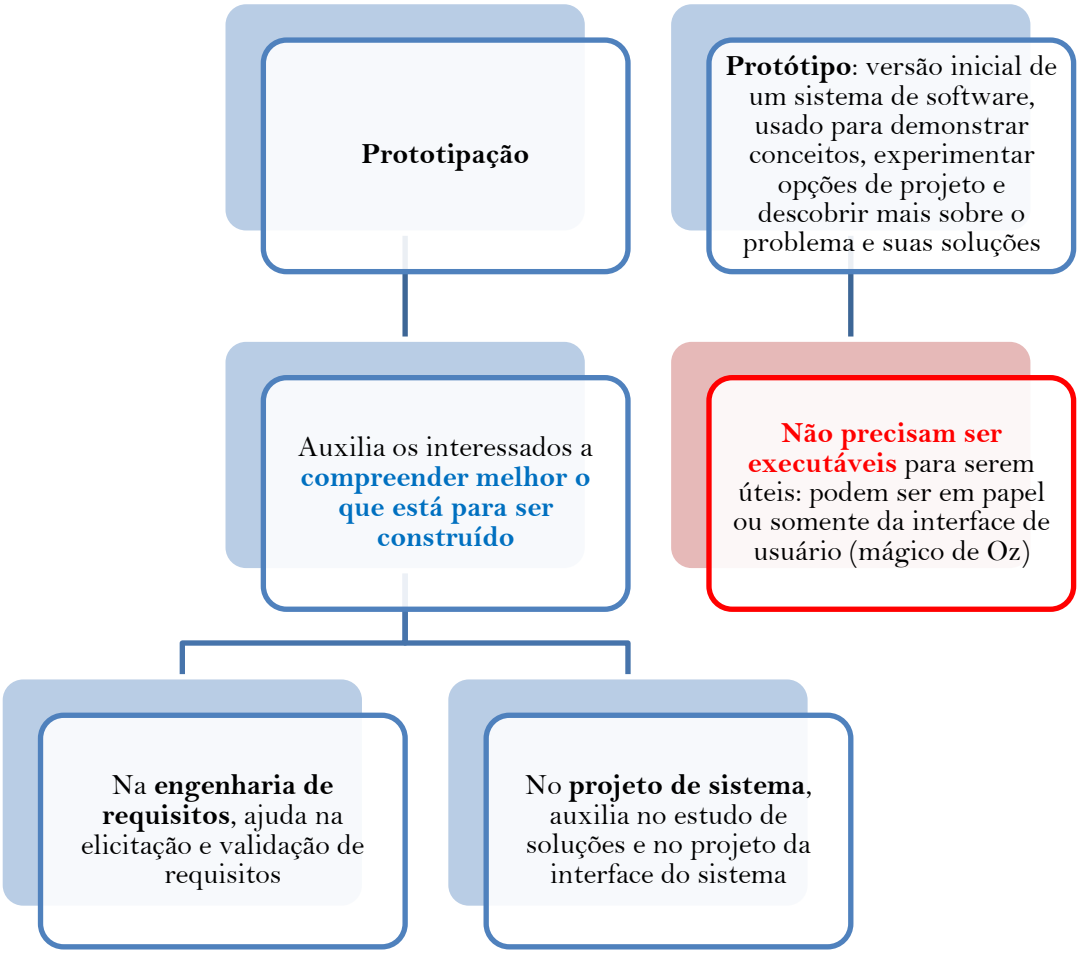
Um **protótipo de software** pode ser usado em um processo de desenvolvimento de software para ajudar a antecipar as mudanças que podem ser requisitadas:

- No processo de **engenharia de requisitos**, um protótipo pode **ajudar na elicitação e validação de requisitos de sistema**.

- No processo de **projeto de sistema**, um protótipo pode ser usado para **estudar soluções específicas do software e para apoiar o projeto de interface de usuário**.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Protótipos não precisam ser executáveis para serem úteis. Maquetes em papel da interface de usuário do sistema podem ser eficazes em ajudar os usuários a refinar o projeto de interface e trabalhar por meio de cenários de uso. Estes são muito baratos de se desenvolver e podem ser construídos em poucos dias. Uma extensão dessa técnica é o **protótipo Mágico de Oz**, no qual **apenas a interface de usuário é desenvolvida**. Os usuários interagem com essa interface, mas suas solicitações são passadas para uma pessoa que os interpreta e produz a resposta adequada.



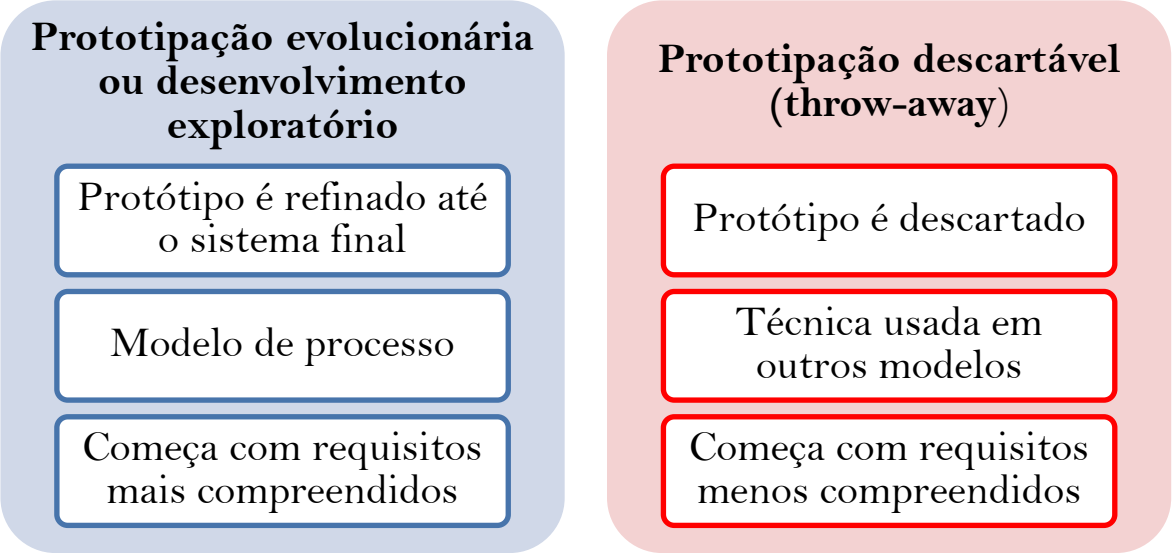
Esquema 26 – Prototipação.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

O protótipo pode servir como "o primeiro sistema", recomendando-se descartá-lo ou são evolucionários, no sentido de que evoluem lentamente até se transformar no sistema real. Assim, temos:

- **Prototipação evolucionária ou desenvolvimento exploratório**: abordagem para o desenvolvimento do sistema onde um **protótipo inicial é produzido e refinado através de vários estágios até atingir o sistema final**. É um modelo de processo que inicia com os requisitos mais bem compreendidos.

- **Prototipação descartável (throw-away)**: um **protótipo** o qual é usualmente uma implementação prática do sistema é **produzido para ajudar a levantar os problemas com os requisitos e depois descartado**. Funciona como uma técnica utilizada em outro modelo de processo que é aplicada geralmente aos requisitos menos compreendidos.



Esquema 27 – Tipos de prototipação.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Às vezes, os desenvolvedores são pressionados pelos gerentes para entregar protótipos descartáveis, especialmente quando há atrasos na entrega da versão final do software. No entanto, isso costuma ser desaconselhável:

- **Pode ser impossível ajustar o protótipo para atender aos requisitos não funcionais**, como requisitos de desempenho, proteção, robustez e confiabilidade, que foram ignorados durante o desenvolvimento do protótipo.

- Mudanças rápidas durante o desenvolvimento inevitavelmente significam que o **protótipo não está documentado**. A única especificação de projeto é o código do protótipo. Para a manutenção a longo prazo, isso não é bom o suficiente.

- As mudanças durante o desenvolvimento do protótipo provavelmente terão **degradado a estrutura do sistema**. O sistema será difícil e custoso de ser mantido.

- **Padrões de qualidade organizacional geralmente são relaxados** para o desenvolvimento do protótipo.

Além disso, a prototipação pode ser problemática pelas seguintes razões:

- Os interessados enxergam o que parece ser uma versão operacional do software, ignorando que o protótipo é mantido de forma não organizada e que, na pressa de fazer com que ele se torne operacional, não se considera a qualidade global do software, nem sua manutenção a longo prazo. Quando informados que o produto deve ser reconstruído para que altos níveis de qualidade possam ser mantidos, os **interessados protestam e solicitam que "umas poucas correções" sejam feitas para tornar o protótipo um produto operacional**. Frequentemente, a gerência do desenvolvimento de software aceita.

- O engenheiro de software, com frequência, assume compromissos de implementação para conseguir que o protótipo entre em operação rapidamente. Um sistema operacional ou linguagem de programação inapropriados podem ser utilizados simplesmente porque se encontram à disposição e são conhecidos; um algoritmo ineficiente pode ser implementado simplesmente para demonstrar capacidade. Após um tempo, pode-se **acomodar com tais escolhas e esquecer todas as razões pelas quais eram inapropriadas**. Uma escolha longe da ideal acaba se tornando parte integrante do sistema.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Vantagens	Desvantagens
•Auxilia na compreensão dos requisitos	•Dificuldade de implementar requisitos não funcionais
•Permite estudar soluções e interfaces de usuário	•Não documentado
•Não precisam ser executáveis	•Degradação da estrutura do sistema
•Pequeno investimento inicial para produzir uma primeira versão do software	•Padrões de qualidade relaxados
	•Pressão do cliente para entregar protótipo com poucas correções
	•Acomodação ou esquecimento de uso de tecnologias ineficientes

Esquema 28 – Vantagens e desvantagens da prototipação.

Sommerville define um processo de desenvolvimento do protótipo com base nas seguintes fases:

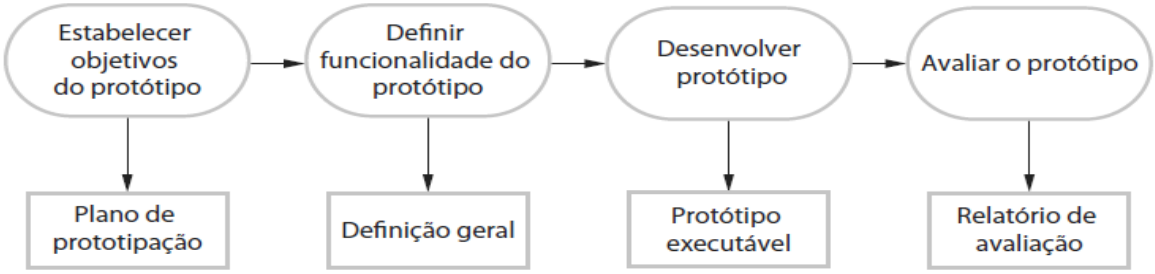
1. **Estabelecer objetivos do protótipo**: os objetivos da prototipação devem ser explicitados desde o início do processo. Estes podem ser o desenvolvimento de um sistema para prototipar a interface de usuário, o desenvolvimento de um sistema para validação dos requisitos funcionais do sistema ou o desenvolvimento de um sistema para demonstrar aos gerentes a viabilidade da aplicação. O mesmo protótipo não pode cumprir todos os objetivos. Se os objetivos não são declarados, a gerência ou os usuários finais podem não entender a função do protótipo. Consequentemente, eles podem não obter os benefícios que esperavam do desenvolvimento do protótipo.

2. **Definir a funcionalidade do protótipo**: o próximo estágio do processo é decidir o que colocar e, talvez mais importante ainda, o que deixar de fora do sistema de protótipo. Para reduzir os custos de prototipação e acelerar o cronograma de entrega, pode-se deixar alguma funcionalidade fora do protótipo. Você pode optar por relaxar os requisitos não funcionais, como tempo de resposta e utilização de memória. Gerenciamento e tratamento de erros podem ser ignorados, a menos que o objetivo do protótipo seja estabelecer uma interface de usuário. Padrões de confiabilidade e qualidade de programa podem ser reduzidos.

3. **Desenvolver protótipo**: criação do protótipo.

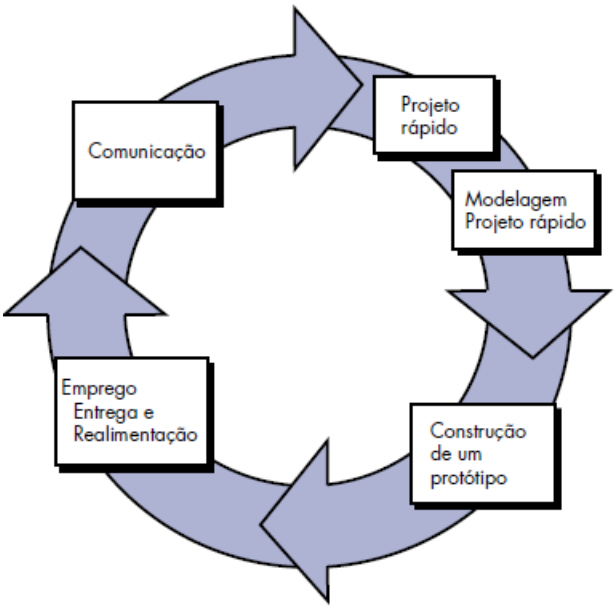
Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

4. **Avaliar protótipo**: provisões devem ser feitas para o treinamento do usuário, e os objetivos do protótipo devem ser usados para derivar um plano de avaliação. Os usuários necessitam de um tempo para se sentir confortáveis com um sistema novo e para se situarem em um padrão normal de uso. Uma vez que estejam usando o sistema normalmente, eles descobrem erros e omissões de requisitos.



Esquema 29 – Modelo de Processo para Desenvolvimento de Protótipos de Sommerville.

Pressman define um modelo que começa com a **comunicação**, com uma reunião com os envolvidos para definir os objetivos gerais do software, identificar quais requisitos já são conhecidos e esquematizar quais áreas necessitam, obrigatoriamente, de uma definição mais ampla. Uma iteração de prototipação é planejada rapidamente e ocorre a modelagem (na forma de um **"projeto rápido"**). Um projeto rápido se concentra em uma representação daqueles aspectos do software que serão visíveis aos usuários finais. O projeto rápido leva à **construção** de um protótipo, que é **empregado e avaliado** pelos envolvidos, que fornecerão um retorno (feedback), que servirá para aprimorar os requisitos.



Esquema 30 – Paradigma de prototipação segundo Pressman.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

21- (CESPE / CEBRASPE - 2021 - TCE-RJ - Analista de Controle Externo - Especialidade: Tecnologia da Informação) Julgue o item que se segue a respeito da prototipação relacionada ao desenvolvimento de software.

A prototipação pode ser considerada como uma atividade que reduz riscos de desenvolvimento relacionados aos requisitos de um software.

Resolução:

Perfeitamente. A prototipação pode ser usada para auxiliar na engenharia de requisitos.

Gabarito: Certo.

22- (CESPE / CEBRASPE - 2021 - TCE-RJ - Analista de Controle Externo - Especialidade: Tecnologia da Informação) Julgue o item que se segue a respeito da prototipação relacionada ao desenvolvimento de software.

Entre as atividades de prototipação de software, está o desenvolvimento rápido de software para validar requisitos.

Resolução:

Um **protótipo** é uma **versão inicial de um sistema de software, usado para demonstrar conceitos, experimentar opções de projeto e descobrir mais sobre o problema e suas possíveis soluções**. O desenvolvimento rápido e iterativo do protótipo é essencial para que os custos sejam controlados e as partes interessadas do sistema possam experimentá-lo no início do processo de software.

Gabarito: Certo.

23- (CESPE - 2018 - MPE-PI - Analista Ministerial - Tecnologia da Informação) Julgue o seguinte item, a respeito da identificação e delimitação de processos de negócios.

Prototipação é caracterizada como uma forma rápida e direta de identificar os requisitos com o gestor, mesmo que ele já saiba como será o resultado final do produto e conheça a plataforma de desenvolvimento desse produto.

Resolução:

Embora a **prototipação** possa ser **utilizada como um modelo de processo isolado** (stand-alone process) é mais comumente utilizada como **uma técnica passível de ser implementada no contexto de qualquer modelo de processo**. Independentemente da forma como é aplicado, quando os requisitos estão obscuros, o paradigma da **prototipação auxilia os interessados a compreender melhor o que está para ser construído**.

Gabarito: Certo.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

24- (FCC - 2015 - TCM-GO - Auditor de Controle Externo - Informática) Sobre a prototipação, considere:

I. Auxilia o engenheiro de software e o cliente a entenderem melhor o que deve ser construído quando os requisitos estão confusos.

II. Tem como vantagem a construção rápida de um protótipo que, não podendo ser descartado, passa a ser o núcleo do sistema que será construído.

III. Pode ser usada como um modelo de processo independente.

IV. Apresenta ao cliente uma versão executável do software, sempre utilizando os algoritmos mais eficientes para demonstrar a capacidade do sistema a ser criado.

V. É indicada para estudar as alternativas de interface do usuário e a viabilidade de atendimento dos requisitos de desempenho, dentre outras possibilidades.

Está correto o que consta APENAS em

a) II, III e IV.

b) I, III e V.

c) III, IV e V.

d) I e V.

e) I, II e IV.

Resolução:

Vamos analisar cada um dos itens:

I. **Correto**: o paradigma da **prototipação** **auxilia os interessados a compreender melhor o que está para ser construído**.

II. **Incorreto**: o protótipo pode ser descartável ou não, depende da abordadem.

III. **Correto**: A **prototipação** possa ser **utilizada como um modelo de processo isolado** (stand-alone process), mas é mais comumente utilizada como **uma técnica passível de ser implementada no contexto de qualquer modelo de processo**.

IV. **Incorreto**: o objetivo do protótipo é facilitar o entendimento, então um algoritmo ineficiente pode ser implementado simplesmente para demonstrar capacidade.

V. **Correto**: um **protótipo de software** pode ser usado em um processo de desenvolvimento de software para ajudar a antecipar as mudanças que podem ser requisitadas:

- No processo de **engenharia de requisitos**, um protótipo pode **ajudar na elicitação e validação de requisitos de sistema**.
- No processo de **projeto de sistema**, um protótipo pode ser usado para **estudar soluções específicas do software e para apoiar o projeto de interface de usuário**.

Gabarito: Letra B.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

2.4.2 Modelo espiral

O **modelo espiral** é um modelo de processo de software **evolucionário que acopla a natureza iterativa da prototipação com os aspectos sistemáticos e controlados do modelo cascata**. Fornece potencial para o rápido desenvolvimento de versões cada vez mais completas do software.

O **modelo espiral de desenvolvimento** é um gerador de **modelos de processos dirigidos a riscos e é utilizado para guiar a engenharia de sistemas intensivos de software, que ocorre de forma concorrente e tem múltiplos envolvidos**. Possui duas características que o distinguem:

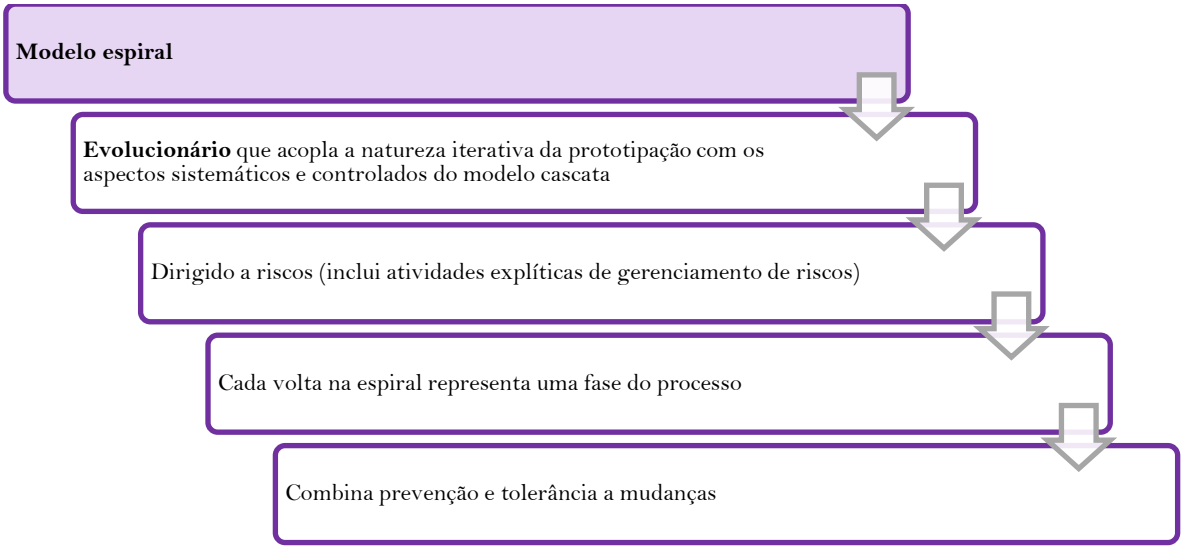
1. **Abordagem cíclica** voltada para ampliar, incrementalmente, o grau de definição e a implementação de um sistema, enquanto diminui o grau de risco do mesmo.

2. Uma **série de pontos âncora de controle** para assegurar o comprometimento de interessados quanto à busca de soluções de sistema que sejam mutuamente satisfatórias e praticáveis.

Usando-se o modelo espiral, o software será desenvolvido em uma série de versões evolucionárias. Nas primeiras iterações, a versão pode consistir em um modelo ou em um protótipo. Já nas iterações posteriores, são produzidas versões cada vez mais completas do sistema.

O processo de software é **representado como uma espiral**, e **não como uma sequência de atividades** com alguns retornos de uma para outra. **Cada volta na espiral representa uma fase do processo de software**. Dessa forma, a volta mais interna pode preocupar-se com a viabilidade do sistema; o ciclo seguinte, com definição de requisitos; o seguinte, com o projeto do sistema, e assim por diante.

O **modelo em espiral** **combina prevenção e tolerância a mudanças**, assume que mudanças são um resultado de riscos de projeto e **inclui atividades explícitas de gerenciamento de riscos** para sua redução.



Esquema 31 – Modelo Espiral.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

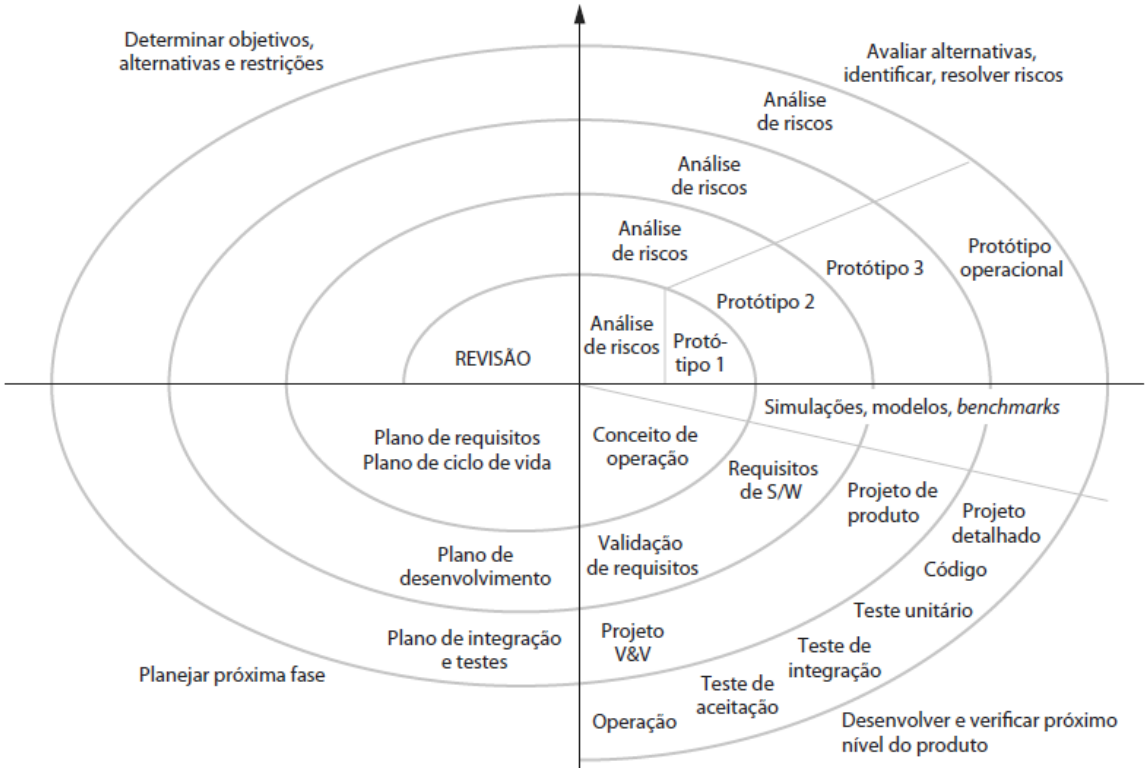
O modelo espiral divide-se em quatro setores:

1. **Definição de objetivos**: objetivos específicos para essa fase do projeto são definidos; restrições ao processo e ao produto são identificadas, e um plano de gerenciamento detalhado é elaborado; os riscos do projeto são identificados. Podem ser planejadas estratégias alternativas em função desses riscos.

2. **Avaliação e redução de riscos**: para cada risco identificado do projeto, é feita uma análise detalhada. Medidas para redução do risco são tomadas. Por exemplo, se houver risco de os requisitos serem inadequados, um protótipo de sistema pode ser desenvolvido.

3. **Desenvolvimento e validação**: é selecionado um modelo de desenvolvimento para o sistema. Por exemplo, a prototipação descartável pode ser a melhor abordagem de desenvolvimento de interface de usuário se os riscos forem dominantes. Se os riscos de segurança forem a principal consideração, o desenvolvimento baseado em transformações formais pode ser o processo mais adequado, e assim por diante.

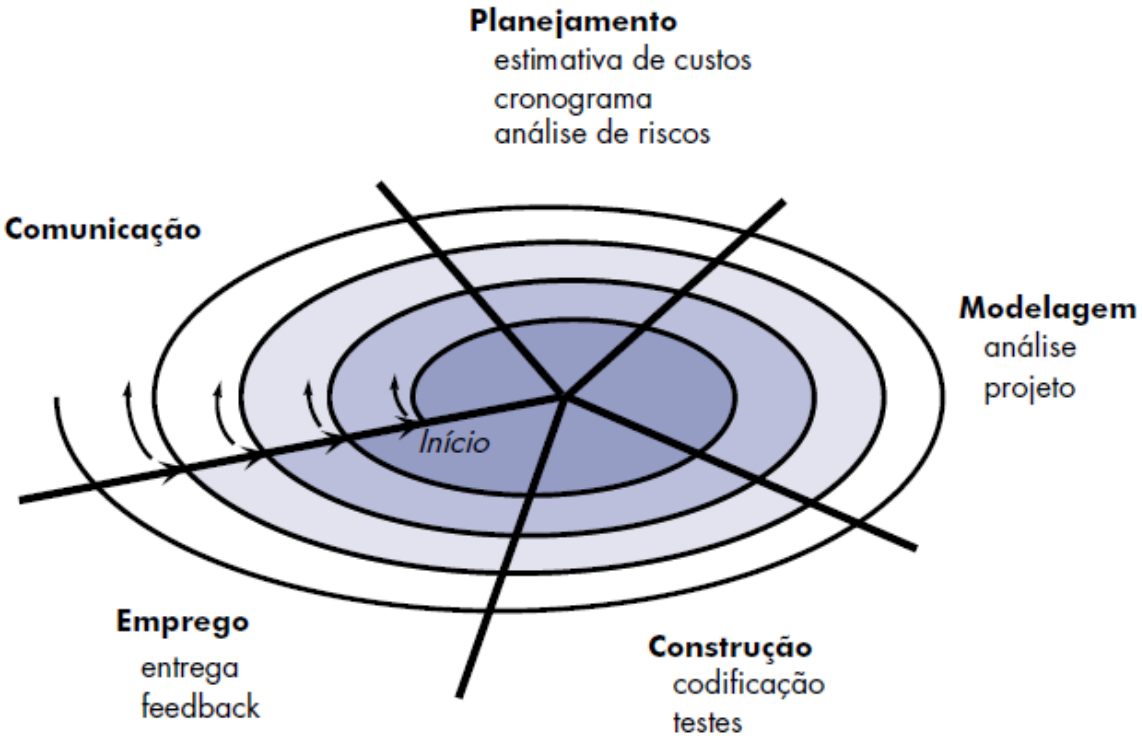
4. **Planejamento**: o projeto é revisado, e uma decisão é tomada a respeito da continuidade do modelo com mais uma volta da espiral. Caso se decida pela continuidade, planos são elaborados para a próxima fase do projeto.



Esquema 32 – Modelo espiral de Boehm segundo Sommerville.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Pressman propõe uma variação para o modelo espiral formada pelas fases de seu ciclo de vida genérico:



Esquema 33 – Modelo espiral proposto por Pressman.

Vejamos as vantagens e desvantagens do modelo espiral:

Vantagens	Desvantagens
Evolucionário, combinando a iteratividade da prototipação com o controle do cascata	Exige especialização em avaliação de riscos
Define uma série de pontos âncora de controle	Um risco não descoberto pode causar grandes impactos
Combina prevenção e tolerância a mudanças	Gerenciamento de processo pode ser mais complexo
Inclui explicitamente atividades de gestão de riscos	

Esquema 34 – Vantagens e desvantagens do modelo espiral.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

25- (CESPE / CEBRASPE - 2021 - Polícia Federal - Escrivão de Polícia Federal) Considerando a teoria geral de sistemas e sistemas de informação, julgue o item a seguir.

Embora não seja dirigido a riscos, o modelo de desenvolvimento de sistemas espiral de Boehm inclui, em seu framework, a etapa de análise e validação dos requisitos.

Resolução:

O **modelo espiral trata explicitamente dos riscos**. O **modelo espiral de desenvolvimento** é um gerador de **modelos de processos dirigidos a riscos e é utilizado para guiar a engenharia de sistemas intensivos de software, que ocorre de forma concorrente e tem múltiplos envolvidos**. Possui duas características que o distinguem:

3. **Abordagem cíclica** voltada para ampliar, incrementalmente, o grau de definição e a implementação de um sistema, enquanto diminui o grau de risco do mesmo.
4. Uma **série de pontos âncora de controle** para assegurar o comprometimento de interessados quanto à busca de soluções de sistema que sejam mutuamente satisfatórias e praticáveis.

Gabarito: Errado.

26- (CESPE - 2018 - FUB - Técnico de Tecnologia da Informação) Acerca dos processos de desenvolvimento de software, julgue o item a seguir.

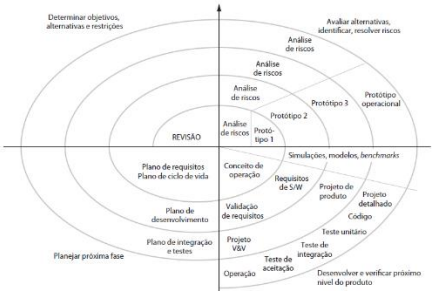
O modelo espiral é um tipo de modelo iterativo de desenvolvimento em que cada ciclo se encerra com a entrega de uma versão final do software.

Resolução:

No **modelo espiral**, cada ciclo representa uma fase do processo e não a entrega de uma versão final do software.

O **modelo espiral** é um modelo de processo de software **evolucionário que acopla a natureza iterativa da prototipação com os aspectos sistemáticos e controlados do modelo cascata**. Fornece potencial para o rápido desenvolvimento de versões cada vez mais completas do software.

O processo de software é **representado como uma espiral**, e **não como uma sequência de atividades** com alguns retornos de uma para outra. **Cada volta na espiral representa uma fase do processo de software**.



Gabarito: Errado.

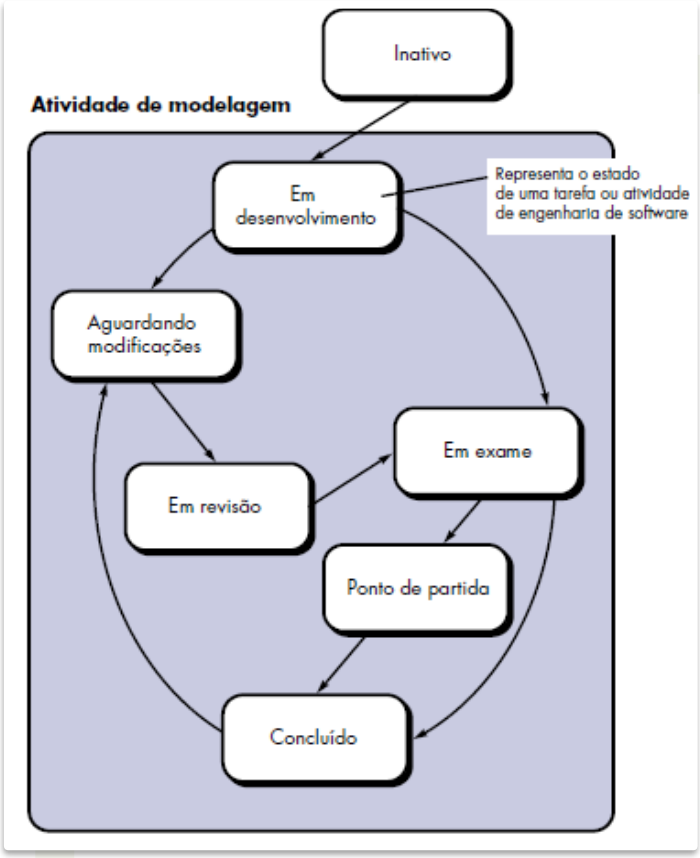
Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

2.4.3 Modelo concorrente

O **modelo de desenvolvimento concorrente ou engenharia concorrente** possibilita à equipe de software **representar elementos concorrentes e iterativos de qualquer um dos modelos de processos**. Por exemplo, a atividade de modelagem definida para o modelo espiral é realizada invocando uma ou mais das seguintes ações de engenharia de software: prototipagem, análise e projeto.

A **modelagem concorrente** **define uma série de eventos que irão disparar transições de estado para estado para cada uma das atividades, ações ou tarefas** da engenharia de software.

A **modelagem concorrente** se aplica a todos os tipos de desenvolvimento de software e **fornece uma imagem precisa do estado atual de um projeto**. Em vez de limitar as atividades, ações e tarefas da engenharia de software a uma sequência de eventos, ela **define uma rede de processos**. Cada atividade, ação ou tarefa na rede existe simultaneamente com outras atividades, ações ou tarefas. Eventos gerados em um ponto da rede de processos disparam transições entre os estados.



Modelo de Desenvolvimento Concorrente

- Representa elementos concorrentes e iterativos de qualquer modelo de processo.

- Fornece uma imagem precisa do estado do projeto.

- Define uma rede de processos.

Esquema 35 – Modelo de Desenvolvimento Concorrente ou Engenharia Concorrente.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

2.5 Modelos especializados

Os modelos de processo especializado levam em conta muitas das características de um ou mais dos modelos tradicionais. Tais modelos tendem a ser aplicados quando se opta por uma abordagem de engenharia de software especializada ou definida de forma restrita.

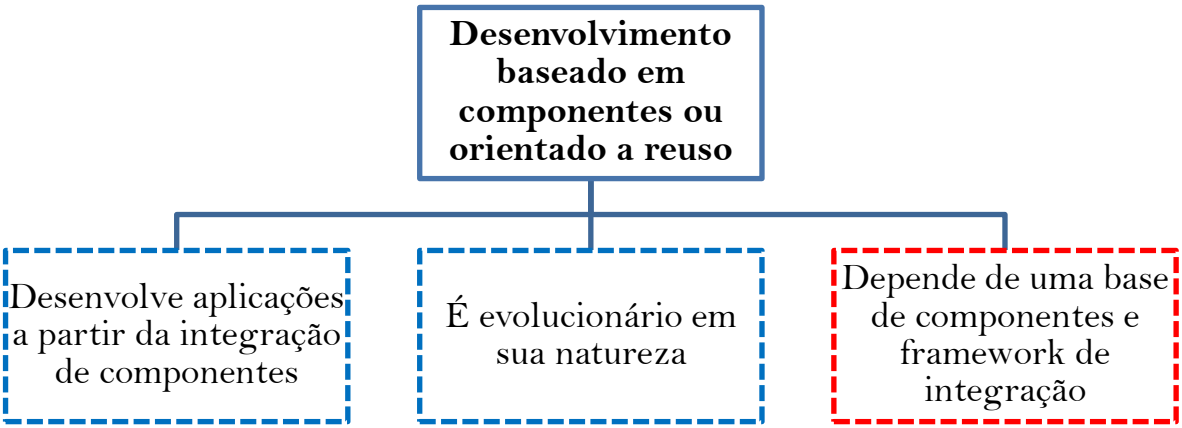
2.5.1 Desenvolvimento baseado em componentes

O **desenvolvimento baseado em componentes ou engenharia de software orientada a reuso** conduz ao **reuso do software** e a reusabilidade proporciona uma série de benefícios mensuráveis aos engenheiros de software como uma **redução no tempo do ciclo de desenvolvimento**, bem como uma **redução no custo** do projeto, caso a reutilização de componentes se torne parte de sua cultura.

As **abordagens orientadas a reuso dependem de uma ampla base de componentes reusáveis de software e de um framework de integração** para a composição desses componentes. Em alguns casos, esses componentes são sistemas completos (COTS ou de prateleira), capazes de fornecer uma funcionalidade específica.

Os **componentes de software comercial de prateleira ou COTS** (sigla para Commercial Off-The-Shelf) são desenvolvidos por vendedores que os oferecem como produtos e **disponibilizam a funcionalidade almejada juntamente com as bem definidas interfaces**, sendo que essas interfaces permitem que o componente seja integrado ao software a ser desenvolvido.

O **modelo de desenvolvimento baseado em componentes desenvolve aplicações a partir de componentes de software pré-empacotados**. O **modelo de desenvolvimento baseado em componentes** incorpora muitas das características do modelo espiral. **É evolucionário em sua natureza**, demandando uma abordagem iterativa para a criação de software.



Esquema 36 – Desenvolvimento baseado em componentes ou orientado a reuso.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Sommerville propõe as um modelo para o desenvolvimento orientado a reuso. Embora o estágio de especificação de requisitos iniciais e o estágio de validação sejam comparáveis a outros processos de software, os estágios intermediários em um processo orientado a reuso são diferentes, sendo:

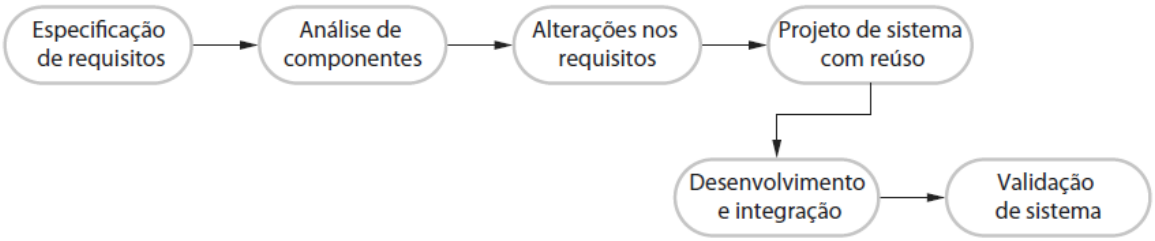
1. **Análise de componentes**: dada a especificação de requisitos, é feita uma **busca por componentes para implementar essa especificação**. Em geral, não há correspondência exata, e os componentes que podem ser usados apenas fornecem alguma funcionalidade necessária.

2. **Modificação de requisitos**: os **requisitos são analisados usando-se informações sobre os componentes que foram descobertos**. Em seguida, estes serão modificados para refletir os componentes disponíveis. No caso de modificações impossíveis, a atividade de análise dos componentes pode ser reinserida na busca por soluções alternativas.

3. **Projeto do sistema com reuso**: o **framework do sistema é projetado ou algo existente é reusado**. Os projetistas têm em mente os componentes que serão reusados e organizam o framework para reuso. Alguns softwares novos podem ser necessários, se componentes reusáveis não estiverem disponíveis.

4. **Desenvolvimento e integração**: **softwares que não podem ser adquiridos externamente são desenvolvidos, e os componentes e sistemas COTS são integrados** para criar o novo sistema. A integração de sistemas, nesse modelo, pode ser parte do processo de desenvolvimento, em vez de uma atividade separada.

Logo, o desenvolvimento orientado a reuso é composto das seguintes fases:



Esquema 37 – Desenvolvimento orientado a reuso de Sommerville.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Pressman considera que o independentemente da tecnologia usada para criar os componentes, o modelo de desenvolvimento baseado em componentes incorpora as seguintes etapas:

Esquema 38 – Desenvolvimento baseado em componentes de Pressman.

27- (FCC - 2014 - TCE-RS - Auditor Público Externo - Técnico em Processamento de Dados - Conhecimentos Específicos) A arquitetura baseada em componentes se constitui em um paradigma de importância crescente na engenharia de software. Nesse tipo de arquitetura,

a) detalhes da implementação de cada componente são abertos, ou seja, conhecidos por todos os demais componentes do sistema.

b) componentes são independentes, no sentido de que não há interferência entre eles.

c) a substituição de um componente sempre obriga a realização de alterações de porte no sistema afetado.

d) na substituição de um componente por outro, sua interface sempre requer alterações.

e) há um número máximo de componentes em cada sistema.

Resolução:

Vamos analisar cada um dos itens:

a) **Incorreto**: apenas as interfaces são abertas, não os detalhes da implementação dos componentes.

b) **Correto:** exatamente, os componentes não devem ter ou sofrer interferência de outros.

c) **Incorreto**: a substituição de componentes nem sempre obriga alterações de grande porte, podem ser alterações pequenas ou mesmo sem ter alterações.

d) **Incorreto**: na substituição de um componente por outro, sua interface ~~sempre~~ **pode** requer**er** alterações.

e) **Incorreto**: **não** há um número máximo de componentes em cada sistema.

Gabarito: Letra B.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

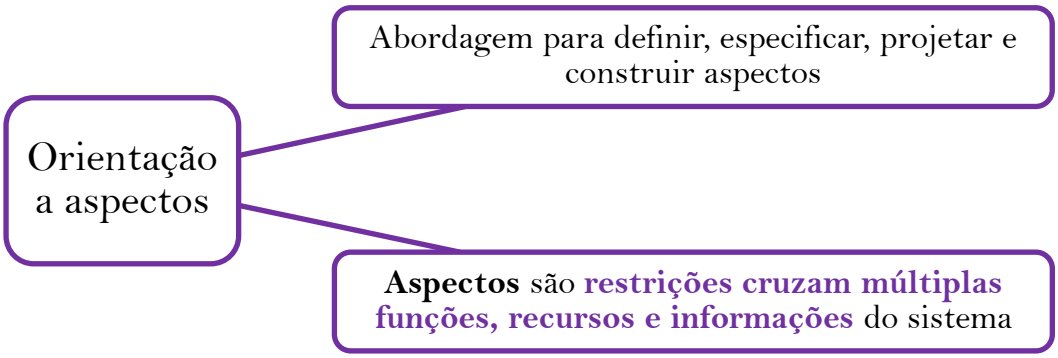
2.5.2 Desenvolvimento orientado a aspectos

Os desenvolvedores de software complexos, invariavelmente, implementam um conjunto de recursos, funções e conteúdo localizados. Essas características de software localizadas são modeladas como componentes (por exemplo, classes orientadas a objetos) e, em seguida, construídas dentro do contexto da arquitetura do sistema. À medida que os modernos sistemas baseados em computadores se tornam mais sofisticados (e complexos), **certas restrições** — propriedades exigidas pelo cliente ou áreas de interesse técnico — **se estendem por toda a arquitetura**. Algumas restrições são propriedades de alto nível de um sistema (por exemplo, segurança, tolerância a falhas). Outras afetam funções (por exemplo, a aplicação de regras de negócio), sendo que outras são sistêmicas (por exemplo, sincronização de tarefas ou gerenciamento de memória).

Quando **restrições cruzam múltiplas funções, recursos e informações do sistema**, elas são, frequentemente, denominadas **restrições cruzadas**. Os requisitos de aspectos definem as restrições cruzadas que têm um impacto por toda a arquitetura de software.

O **desenvolvimento de software orientado a aspectos AOSD, Aspect-Oriented Software Development)**, com frequência conhecido como **programação orientada a aspectos (AOP, Aspect-Oriented Programming)**, é um paradigma de engenharia de software relativamente novo que oferece uma **abordagem metodológica e de processos para definir, especificar, projetar e construir aspectos** — "mecanismos além das sub-rotinas e herança para localizar a expressão de uma restrição cruzada".

O **desenvolvimento orientado a aspectos** usa um conceito de fatias horizontais através de componentes de software decompostos verticalmente, chamados **"aspectos"**, para **caracterizar propriedades funcionais ou não funcionais cruzadas dos componentes**. Aspectos sistêmicos comuns incluem interfaces com o usuário, trabalho colaborativo, distribuição, persistência, gerenciamento de memória, processamento de transações, segurança, integridade e assim por diante.



Esquema 39 – Desenvolvimento orientado a aspectos.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

3. METODOLOGIAS ÁGEIS

3.1 Modelos Ágeis

Em 2001, Kent Beck e outros dezesseis renomados desenvolvedores, autores e consultores da área de software (batizados de "Agile Alliance"- "Aliança dos Ágeis") assinaram o "Manifesto para o Desenvolvimento Ágil de Software" ("Manifesto for Agile Software Development"), que se inicia da seguinte maneira:

Desenvolvendo e ajudando outros a desenvolver software, estamos desvendando formas melhores de desenvolvimento. Por meio deste trabalho passamos a valorizar:

Indivíduos e interações acima de **processos e ferramentas**

Software operacional acima de **documentação completa**

Colaboração dos clientes acima de **negociação contratual**

Respostas a mudanças acima de **seguir um plano**

Ou seja, embora haja valor nos itens à direita, valorizaremos os da esquerda mais ainda.

Em essência, **métodos ágeis** se desenvolveram em um esforço para sanar fraquezas reais e perceptíveis da engenharia de software convencional. O **desenvolvimento ágil** **oferece benefícios importantes**, no entanto, **não é indicado para todos os projetos, produtos, pessoas e situações**. Também não é a antítese da prática de engenharia de software consistente e pode ser aplicado como uma filosofia geral para todos os trabalhos de software.

Em resumo:

Metódos ágeis	**Indivíduos e interações** acima de **processos e ferramentas**
	Software operacional acima de **documentação completa**
	Colaboração dos clientes acima de **negociação contratual**
	Respostas a mudanças acima de **seguir um plano**

Esquema 40 – Métodos ágeis.

Os **princípios por trás do manifesto ágil** são:

- Nossa maior prioridade é satisfazer o cliente através da entrega contínua e adiantada de software com valor agregado.
- Mudanças nos requisitos são bem-vindas, mesmo tardiamente no desenvolvimento.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

- Processos ágeis tiram vantagem das mudanças visando vantagem competitiva para o cliente.
- Entregar frequentemente software funcionando, de poucas semanas a poucos meses, com preferência à menor escala de tempo.
- Pessoas de negócio e desenvolvedores devem trabalhar diariamente em conjunto por todo o projeto.
- Construa projetos em torno de indivíduos motivados. Dê a eles o ambiente e o suporte necessário e confie neles para fazer o trabalho.
- O método mais eficiente e eficaz de transmitir informações para e entre uma equipe de desenvolvimento é através de conversa face a face.
- Software funcionando é a medida primária de progresso.
- Os processos ágeis promovem desenvolvimento sustentável. Os patrocinadores, desenvolvedores e usuários devem ser capazes de manter um ritmo constante indefinidamente.
- Contínua atenção à excelência técnica e bom design aumenta a agilidade.
- Simplicidade - a arte de maximizar a quantidade de trabalho não realizado - é essencial.
- As melhores arquiteturas, requisitos e designs emergem de equipes auto-organizáveis.
- Em intervalos regulares, a equipe reflete sobre como se tornar mais eficaz e então refina e ajusta seu comportamento de acordo.

28- (CESPE - 2018 - STJ - Técnico Judiciário - Desenvolvimento de Sistemas) No que se refere a padrões de projeto e a processos e práticas ágeis de desenvolvimento, julgue o próximo item.

Metodologias ágeis aplicam técnicas de desenvolvimento iterativo e evolutivo de tempo limitado, por isso excluem qualquer tipo de documentação, apoiando-se apenas em conversas informais.

Resolução:

Um dos princípios do manifesto ágil é **Software operacional** acima de **documentação completa.**

Porém, não significa que seja excluída toda a documentação do processo de software. O que ocorre é a simplificação da documentação e o maior foco na produção de software operacional. Contudo, ainda se produzem documentos.

Gabarito: Errado.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

29- (CESPE / CEBRASPE - 2021 - SERPRO - Analista - Especialização: Desenvolvimento de Sistemas) A respeito de processo orientado a reúso e ciclos de vida, julgue o item a seguir.

Um dos princípios da modelagem ágil é a abstração das ferramentas que serão utilizadas para criar os modelos e suas notações particulares.

Resolução:

A abstração de ferramentas não é um princípio ágil.

Os **princípios por trás do manifesto ágil** são:

- Nossa maior prioridade é satisfazer o cliente através da entrega contínua e adiantada de software com valor agregado.
- Mudanças nos requisitos são bem-vindas, mesmo tardiamente no desenvolvimento.
- Processos ágeis tiram vantagem das mudanças visando vantagem competitiva para o cliente.
- Entregar frequentemente software funcionando, de poucas semanas a poucos meses, com preferência à menor escala de tempo.
- Pessoas de negócio e desenvolvedores devem trabalhar diariamente em conjunto por todo o projeto.
- Construa projetos em torno de indivíduos motivados. Dê a eles o ambiente e o suporte necessário e confie neles para fazer o trabalho.
- O método mais eficiente e eficaz de transmitir informações para e entre uma equipe de desenvolvimento é através de conversa face a face.
- Software funcionando é a medida primária de progresso.
- Os processos ágeis promovem desenvolvimento sustentável. Os patrocinadores, desenvolvedores e usuários devem ser capazes de manter um ritmo constante indefinidamente.
- Contínua atenção à excelência técnica e bom design aumenta a agilidade.
- Simplicidade - a arte de maximizar a quantidade de trabalho não realizado - é essencial.
- As melhores arquiteturas, requisitos e designs emergem de equipes auto-organizáveis.
- Em intervalos regulares, a equipe reflete sobre como se tornar mais eficaz e então refina e ajusta seu comportamento de acordo.

Gabarito: Errado.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

3.2 Extreme Programming - XP

Extreme Programming (XP) é talvez o mais conhecido e mais utilizado dos métodos ágeis. O nome foi cunhado por Beck (2000), pois a **abordagem foi desenvolvida para impulsionar práticas reconhecidamente boas, como o desenvolvimento iterativo, a níveis 'extremos'**. Por exemplo, em XP, várias novas versões de um sistema podem ser desenvolvidas, integradas e testadas em um único dia por programadores diferentes.

Em **Extreme Programming**, os **requisitos são expressos como cenários** (chamados de estórias do usuário), que são implementados diretamente como uma série de tarefas. Os **programadores trabalham em pares** e **desenvolvem testes para cada tarefa antes de escreverem o código**. Quando o novo código é integrado ao sistema, todos os testes devem ser executados com sucesso. Há um **curto intervalo entre os releases** do sistema.

3.2.1 Valores do XP

Beck define um conjunto de **cinco valores** que estabelecem as bases para todo trabalho realizado como parte da XP — comunicação, simplicidade, feedback (realimentação ou retorno), coragem e respeito. Cada um desses valores é usado como um direcionador das atividades, ações e tarefas específicas da XP.

- **Comunicação**: a XP enfatiza a **colaboração estreita, embora informal (verbal)**, entre clientes e desenvolvedores, o estabelecimento de metáforas eficazes para comunicar conceitos importantes, feedback (realimentação) contínuo e evitar documentação volumosa como meio de comunicação.

- **Simplicidade**: a XP restringe os desenvolvedores a **projetar apenas para as necessidades imediatas**, em vez de considerarem as necessidades futuras. O intuito é criar um projeto simples que possa ser facilmente implementado em código. Se o projeto tiver que ser melhorado, ele poderá ser refatorado mais tarde.

- **Feedback**: o feedback provém de três fontes: do **próprio software implementado, do cliente e de outros membros** da equipe de software.

- **Coragem**: uma equipe XP ágil deve ter **disciplina (coragem) para projetar para hoje, reconhecendo que as necessidades futuras podem mudar dramaticamente** exigindo, consequentemente, substancial retrabalho em relação ao projeto e ao código implementado.

- **Respeito**: a equipe ágil busca **respeito entre seus membros, entre outros envolvidos** e os membros da equipe, e, indiretamente, para o próprio software.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Esquema 41 – Valores do XP.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

3.2.2 Atividades do XP

Segundo Pressman, a **Extreme Programming** (programação extrema) emprega uma **abordagem orientada a objetos** como seu paradigma de desenvolvimento preferido e envolve um conjunto de regras e práticas constantes no contexto de quatro atividades metodológicas: planejamento, projeto, codificação e testes.

- **Planejamento**: a atividade de planejamento (também denominada o jogo do planejamento) se inicia com a atividade de ouvir — uma atividade de **levantamento de requisitos** que capacita os membros técnicos da equipe XP a entender o ambiente de negócios do software e possibilita que se consiga ter uma percepção ampla sobre os resultados solicitados, fatores principais e funcionalidade. A atividade de "Ouvir" conduz à **criação de um conjunto de "histórias"** (também denominado histórias de usuários) que descreve o resultado, as características e a funcionalidade requisitados para o software a ser construído. O **cliente atribui um valor (uma prioridade) à história** baseando-se no valor de negócio global do recurso ou função. Os **membros da equipe XP avaliam então cada história e atribuem um custo** — medido em semanas de desenvolvimento — a ela.

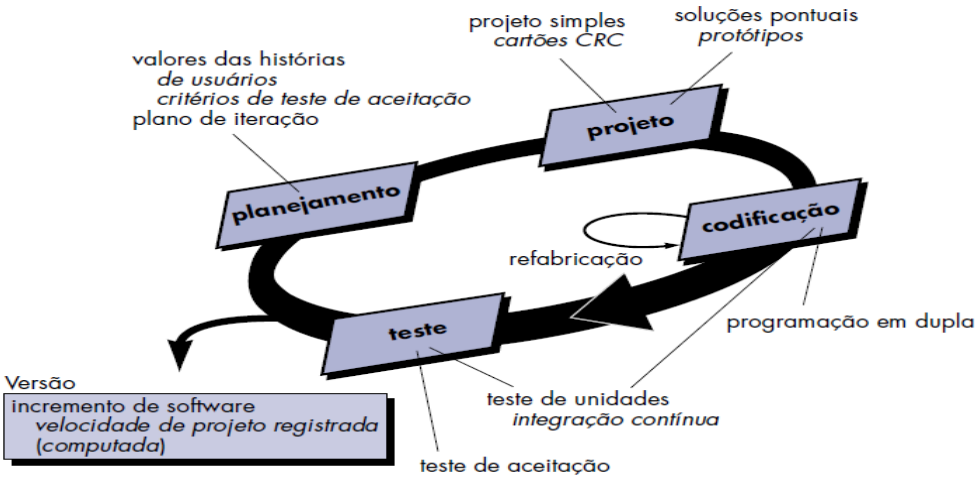
- **Projeto**: o projeto XP **segue rigorosamente o princípio KIS (keep it simple, ou seja, preserve a simplicidade)**. É preferível sempre um projeto simples do que uma representação mais complexa. Como acréscimo, o projeto oferece um guia de implementação para uma história à medida que é escrita — nada mais, nada menos. O projeto de funcionalidade extra (pelo fato de o desenvolvedor supor que ela será necessária no futuro) é desencorajado. O XP **encoraja o uso de cartões CRC** como um mecanismo eficaz para pensar sobre o software em um contexto orientado a objetos. Os cartões CRC (classe-responsabilidade-colaborador) identificam e organizam as classes orientadas a objetos relevantes para o incremento de software corrente. Se um difícil problema de projeto for encontrado como parte do projeto de uma história, a XP recomenda a **criação imediata de um protótipo operacional dessa parte do projeto**. Denominada solução pontual, o protótipo do projeto é implementado e avaliado.

- **Codificação**: depois de desenvolvidas as histórias e o trabalho preliminar de elaboração do projeto ter sido feito, a equipe não passa para a codificação, mas sim, desenvolve uma série de testes de unidades que exercitarão cada uma das histórias a ser inclusas na versão corrente (incremento de software). **Uma vez criado o teste de unidades, o desenvolvedor poderá melhor focar-se no que deve ser implementado para ser aprovado no teste**. Nada estranho é adicionado (KIS). Estando o código completo, este pode ser testado em unidade imediatamente, e, dessa forma, prover, instantaneamente, feedback para os desenvolvedores. Um conceito-chave na atividade de codificação (e um dos mais discutidos aspectos da XP) é a **programação em dupla**. O XP recomenda que duas pessoas trabalhem juntas em uma mesma estação de trabalho para criar

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

código para uma história. Isso fornece um mecanismo para resolução de problemas em tempo real (duas cabeças normalmente funcionam melhor do que uma) e garantia da qualidade em tempo real (o código é revisto à medida que é criado).

- **Testes**: como os testes de unidades individuais são organizados em um "conjunto de testes universal", os **testes de integração e validação do sistema podem ocorrer diariamente**. Isso dá à equipe XP uma indicação contínua do progresso e permite lançar alertas logo no início, caso as coisas não andem bem. Os **testes de aceitação da XP, também denominados testes de cliente, são especificados pelo cliente** e mantêm o foco nas características e na funcionalidade do sistema total que são visíveis e que podem ser revistas pelo cliente.



Esquema 42 – Processo de Extremming Programming segundo Pressman.

30- (CESPE - 2019 - TJ-AM - Analista Judiciário - Analista de Sistemas) Julgue o item seguinte, a respeito das metodologias de desenvolvimento de software.
No XP (Extreme Programming), o valor de uma história de usuário é atribuído pelos membros da equipe e é medido em termos de semanas estimadas para o desenvolvimento.

Resolução:

Quem atribui valor a uma história de usuário é o cliente, pois é ele que define as prioridades. Já a estimativa de tempo de desenvolvimento é realizada pelos desenvolvedores. Isso ocorre no chamado Jogo de Planejamento.

O **cliente atribui um valor (uma prioridade) à história** baseando-se no valor de negócio global do recurso ou função. Os **membros da equipe XP avaliam então cada história e atribuem um custo** — medido em semanas de desenvolvimento — a ela.

Em resumo, o **cliente** identifica prioridades e os **desenvolvedores** as estimam. O cliente é essencial neste processo e assim ele fica sabendo o que está acontecendo e o que vai acontecer no projeto.

Gabarito: Errado.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

3.2.3 Práticas do XP

Para aplicar os valores e princípios durante o desenvolvimento de software, XP propõe uma série de práticas. Há uma confiança muito grande na sinergia entre elas, os pontos fracos de cada uma são superados pelos pontos fortes de outras. Vejamos as práticas do XP:

- **Jogo de Planejamento (Planning Game)**: o **desenvolvimento é feito em interações semanais**. No início da semana, desenvolvedores e cliente reúnem-se para priorizar as funcionalidades. Essa reunião recebe o nome de Jogo do Planejamento e nelas já devem estar criadas antecipadamente pelos usuários as User Stories (história dos usuários). Nessa reunião, o cliente identifica prioridades e os desenvolvedores as estimam. O cliente é essencial neste processo e assim ele fica sabendo o que está acontecendo e o que vai acontecer no projeto. Como o escopo é reavaliado semanalmente, o projeto é regido por um contrato de escopo negociável, que difere significativamente das formas tradicionais de contratação de projetos de software. Ao final de cada semana, o cliente recebe novas funcionalidades, completamente testadas e prontas para serem postas em produção.

- **Fases pequenas (Small Releases)**: a **liberação de pequenas versões funcionais do projeto auxilia muito no processo de aceitação** por parte do cliente, que já pode testar uma parte do sistema que está comprando. As versões chegam a ser ainda menores que as produzidas por outras metodologias incrementais, como o RUP.

- **Metáfora (Metaphor)**: procura facilitar a comunicação com o cliente, entendendo a realidade dele. O conceito de rápido para um cliente de um sistema jurídico é diferente para um programador experiente em controlar comunicação em sistemas em tempo real, como controle de tráfego aéreo. É preciso **traduzir as palavras do cliente para o significado que ele espera dentro do projeto**.

- **Design Simples (Simple Design)**: **simplicidade é um princípio da XP**. Projeto simples significa dizer que caso o cliente tenha pedido que na primeira versão apenas o usuário "teste" possa entrar no sistema com a senha "123" e assim ter acesso a todo o sistema, você vai fazer o código exato para que esta funcionalidade seja implementada, sem se preocupar com sistemas de autenticação e restrições de acesso. Um erro comum ao adotar essa prática é a confusão por parte dos programadores de código simples e código fácil. Nem sempre o código mais fácil de ser desenvolvido levará a solução mais simples por parte de projeto. Esse entendimento é fundamental para o bom andamento do XP. Código fácil deve ser identificado e substituído por código simples.

- **Testes de Aceitação (Customer Tests)**: são **testes construídos pelo cliente** em conjunto de analistas e testadores, para aceitar um determinado requisito do sistema.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

- **Semana de 40 horas (Sustainable Pace)**: **trabalhar com qualidade, buscando ter ritmo de trabalho saudável (40 horas/semana, 8 horas/dia)**, **sem horas extras**. Horas extras são permitidas quando trouxerem produtividade para a execução do projeto. Outra prática que se verifica neste processo é a prática de trabalho energizado, onde se busca trabalho motivado sempre. Para isto o ambiente de trabalho e a motivação da equipe devem estar sempre em harmonia.

- **Propriedade Coletiva (Collective Ownership)**: o **código fonte não tem dono** e ninguém precisa solicitar permissão para poder modificá-lo. O objetivo com isto é fazer a equipe conhecer todas as partes do sistema.

- **Programação em Pares (Pair Programming)**: é a **programação em par/dupla num único computador**. Geralmente a dupla é formada por um iniciante na linguagem e outra pessoa funcionando como um instrutor. Como é apenas um computador, o novato é que fica à frente fazendo a codificação, e o instrutor acompanha ajudando a desenvolver suas habilidades. Desta forma o programa sempre é revisto por duas pessoas, evitando e diminuindo assim a possibilidade de defeitos. Com isto busca-se sempre a evolução da equipe, melhorando a qualidade do código fonte gerado.

- **Padronização do Código (Coding Standards)**: a equipe de desenvolvimento precisa estabelecer **regras para programar e todos devem seguir estas regras**. Desta forma parecerá que todo o código fonte foi editado pela mesma pessoa, mesmo quando a equipe possui 10 ou 100 membros.

- **Desenvolvimento Orientado a Testes (Test Driven Development)**: **Primeiro crie os testes unitários (unit tests) e depois crie o código para que os testes funcionem**. Esta abordagem é complexa no início, pois vai contra o processo de desenvolvimento de muitos anos, mas os testes unitários são essenciais para manter a qualidade do projeto.

- **Refatoração (Refactoring)**: é um processo que permite a **melhoria contínua da programação, com o mínimo de introdução de erros e mantendo a compatibilidade com o código já existente**. Refatorar melhora a clareza (leitura) do código, divide-o em módulos mais coesos e de maior reaproveitamento, evitando a duplicação de código-fonte. A refatoração melhora o código, mas **sem modificar a funcionalidade**.

  É interessante **refatorar** o código para melhorar a sua conformidade com padrões e a legibilidade, o que facilita o seu entendimento sem alterar o seu comportamento externo. Isto é, quando um código refatorado é executado, ele continua tendo o mesmo resultado final.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

As vantagens de refatorar são:

- **Melhora o projeto**: sem refatoração a estrutura interna de um software geralmente se degrada com as mudanças que ocorrem no decorrer do tempo. Programadores que nunca viram o código antes são encarregados de adicionar uma nova funcionalidade e provavelmente vão fugir um pouco do padrão que foi utilizado quando o software foi projetado. Aplicar refatoração após a adição dessa nova funcionalidade poderá deixá-la em conformidade com os padrões que foram utilizados nas outras funcionalidades que o software já possui.

- **Torna o software fácil de entender**: desenvolvedores geralmente estão sempre com pouco tempo para terminar suas tarefas e entregar a funcionalidade pronta. Para conseguir entregar dentro do prazo, alguns acabam se apressando na hora de codificar, o que pode prejudicar a legibilidade (clareza do código) e alguns bugs podem passar despercebidos. Erros comuns são: não nomear as variáveis corretamente, não comentar, duplicar código, colocar métodos em classes inadequadas, entre outros. Ao empregar a refatoração, muito provavelmente, tais "sujeiras" serão removidas do código, o que o tornará muito mais fácil de compreender e manter.

- **Ajuda a achar bugs**: quando um desenvolvedor vai refatorar certo código, ele deve entender exatamente o que aquele código está fazendo. Entendendo muito bem o seu funcionamento, os possíveis bugs existentes naquele código acabam se tornando evidentes. Como o desenvolvedor já está alterando o código, recomenda-se também eliminar o bug.

- **Ajuda a programar mais rápido**: com todas as vantagens citadas até agora, pode-se notar que quando a refatoração é aplicada por uma pessoa que tem domínio sobre a técnica, a qualidade do software aumenta e o código fica mais fácil de entender. Assim, quando um desenvolvedor pega o código para adicionar uma nova funcionalidade e consegue compreendê-lo mais rapidamente, o processo de desenvolvimento também ocorre de maneira mais rápida e tranquila.

- **Integração Contínua (Continuous Integration)**: sempre que produzir uma nova funcionalidade, nunca esperar uma semana para integrar à versão atual do sistema. Isto só aumenta a possibilidade de conflitos e a possibilidade de erros no código fonte. **Integrar de forma contínua permite saber o status real da programação**.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Cuidado para não confundir a integração contínua com a entrega contínua.

- Na **integração contínua**, os desenvolvedores integram as suas mudanças ao branch principal sempre que possível. Nessa situação, as mudanças são validadas com builds de testes que rodam de forma automatizada, impedindo problemas na hora de lançar mudanças nas integrações.
- Já a **entrega contínua** é uma extensão, que não só traz um fluxo de automatização contínua para os testes da integração, como também para os deploys e processos de lançamento, permitindo que você atualize sua aplicação com apenas um clique de botão. Desta forma, é possível fazer muito mais mudanças em ciclos menores, que trazem uma série de benefícios para seu software.

A **entrega contínua** é uma abordagem na qual os **times de desenvolvimento lançam produtos de qualidade de forma frequente, previsível e automatizada**. Em vez de fazer grandes entregas de uma vez, fazem várias pequenas e rápidas — reduzindo as chances de erros e conquistando maior controle de qualidade.

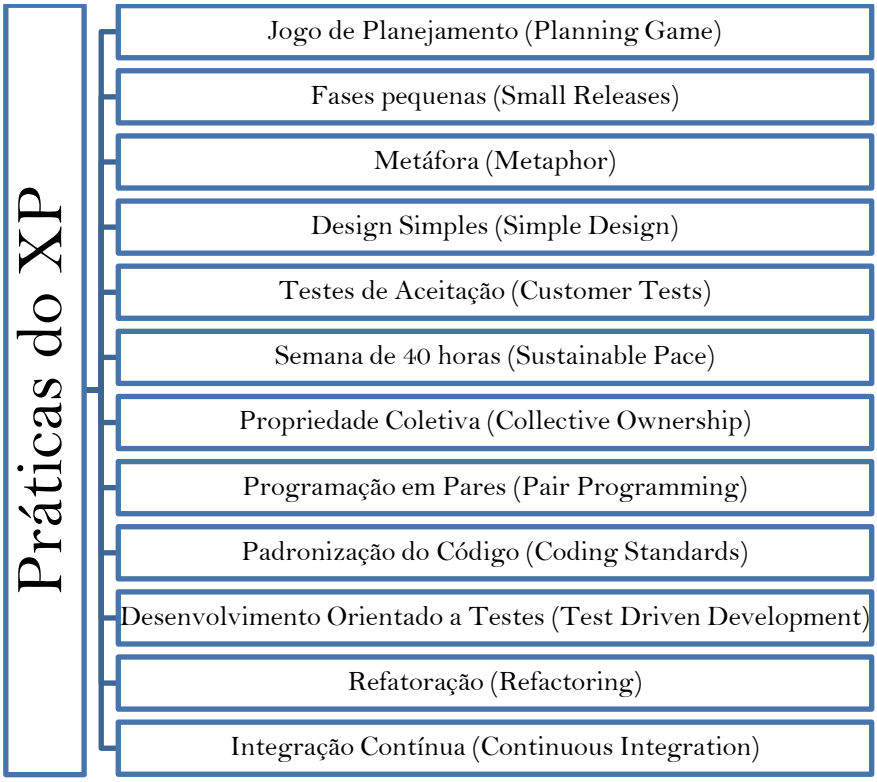
O uso de **automação nos testes, infraestrutura e deploy** é uma das principais características da entrega contínua, o que viabiliza que os versionamentos sejam mais velozes.

Entre as vantagens de usar entrega contínua, podemos citar a otimização do trabalho dos desenvolvedores, maior agilidade no lançamento de produtos (time to market) e redução de custos.

No entanto para que a entrega contínua funcione, é importante que sua **equipe tenha maturidade suficiente para lidar com o acúmulo de responsabilidades** que essa abordagem exige. Além de um bom planejamento inicial, é fundamental que haja participação constante dos responsáveis pelo projeto no processo de validação. Caso contrário, mesmo com as ferramentas necessárias, a equipe não conseguirá lançar novas versões com frequência.

Para garantir a entrega contínua, é necessário fazer com que o código esteja sempre pronto para o deploy, mesmo em times com centenas de desenvolvedores fazendo alterações diárias. Para isso, é necessário eliminar as fases de integração e testes que normalmente pedem "code freeze" e interrompem a produção.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07



Esquema 43 – Práticas do XP.

31- (CESPE - 2019 - SLU-DF - Analista de Gestão de Resíduos Sólidos - Informática) Com relação a desenvolvimento de software, julgue o item a seguir.

Refactoring (refatoração) é o processo utilizado para reescrever aplicações desatualizadas, com a finalidade de incrementar e melhorar suas funcionalidades; o uso dessa técnica normalmente aprimora aplicações para disponibilizá-las na Internet.

Resolução:

Não mesmo. Refatoração não incrementa nenhuma funcionalidade.

Refactoring (refatoração) é um processo que permite a **melhoria contínua da programação, com o mínimo de introdução de erros e mantendo a compatibilidade com o código já existente**. **Refatorar** melhora a clareza (leitura) do código, divide-o em módulos mais coesos e de maior reaproveitamento, evitando a duplicação de código-fonte. A refatoração melhora o código, mas **sem modificar a funcionalidade**.

Gabarito: Errado.

32- (CESPE - 2018 - STJ - Técnico Judiciário - Desenvolvimento de Sistemas) Julgue o seguinte item, relativo à gestão ágil de projetos com XP (extreme programming).

A integração contínua descrita na XP pode fazer parte do processo de desenvolvimento de software que utiliza o Scrum, pois, à medida que as entregas sejam realizadas na Scrum, pode-se validá-las por meio de testes automatizados.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Resolução:

A **Integração Contínua (Continuous Integration)** significa que sempre que produzir uma nova funcionalidade, nunca esperar uma semana para integrar à versão atual do sistema. Isto só aumenta a possibilidade de conflitos e a possibilidade de erros no código fonte. **Integrar de forma contínua permite saber o status real da programação**. Embora seja uma prática do XP, é possível que seja adotada em um ambiente gerenciado pelo Scrum. A revisão da Sprint pode ser apoiada por ferramentas para testes automatizados para inspecionar o incremento.

Gabarito: Certo.

33- (FCC - 2017 - TST - Técnico Judiciário – Programação) Uma dupla de programadores, utilizando o modelo Extreme Programming – XP, realiza, na fase de

a) desenvolvimento, a implementação das user stories que fazem parte da iteração corrente.

b) desenvolvimento, a entrega das user stories totais do sistema.

c) validação do sistema, a análise dos requisitos técnicos entregáveis.

d) validação do sistema, a integração total dos incrementos das user stories.

e) projeto da arquitetura do sistema, a implementação das user stories totais do sistema.

Resolução:

Um conceito-chave na atividade de codificação (e um dos mais discutidos aspectos da XP) é a **programação em dupla**. O XP recomenda que duas pessoas trabalhem juntas em uma mesma estação de trabalho para criar código para uma história. Isso fornece um mecanismo para resolução de problemas em tempo real (duas cabeças normalmente funcionam melhor do que uma) e garantia da qualidade em tempo real (o código é revisto à medida que é criado).

Gabarito: Letra A.

34- (INSTITUTO AOCP - 2021 - ITEP - RN - Perito Criminal - Computação) O XP (Extreme Programming) é um método ágil para o desenvolvimento de sistemas. Assinale a alternativa correta em relação a esse método.

a) É recomendado para qualquer tipo de projeto, produto ou situação.

b) Busca agilidade na entrega do software e, portanto, não exige uma etapa de planejamento para não perder tempo.

c) A entrega do software é feita de maneira incremental, ou seja, através de "releases" menores.

d) Recomenda o desenvolvimento de forma individual, ou seja, cada desenvolvedor cuida sozinho do desenvolvimento de código para uma história.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

e) Tenta prever, de modo antecipado, tudo o que o cliente pode querer solicitar futuramente, adiantando o desenvolvimento desses módulos previstos e facilitando a entrega final.

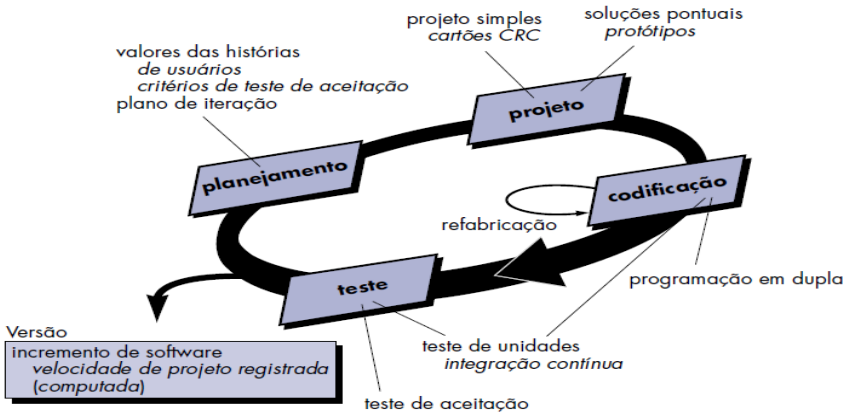
Resolução:

Vamos analisar cada um dos itens:

a) **Incorreto**: nenhuma metodologia é aplicável ou pelo menos sugerida para todos os tipos de projeto. Cada metodologia é melhor aplicável em um determinado tipo de projeto. O XP é uma metodologia ágil de desenvolvimento de software para equipes pequenas, coesas e multidisciplinares cujos projetos possuem requisitos vagos e em constante mudança

b) **Incorreto**: Busca agilidade na entrega do software ~~e, portanto, não exige uma etapa de planejamento para não perder tempo~~.

O XP possui etapa de planejamento.



c) **Correto**: A **liberação de pequenas versões funcionais do projeto auxilia muito no processo de aceitação** por parte do cliente, que já pode testar uma parte do sistema que está comprando. As versões chegam a ser ainda menores que as produzidas por outras metodologias incrementais, como o RUP.

d) **Incorreto**: Recomenda o desenvolvimento ~~de forma individual~~ **em pares**, ou seja, ~~cada desenvolvedor cuida sozinho~~ **uma dupla de desenvolvedores cuidam** do desenvolvimento de código para uma história.

e) **Incorreto**: muito pelo contrário. O foco é justamente no simples. Projeto simples significa dizer que caso o cliente tenha pedido que na primeira versão apenas o usuário "teste" possa entrar no sistema com a senha "123" e assim ter acesso a todo o sistema, você vai fazer o código exato para que esta funcionalidade seja implementada, sem se preocupar com sistemas de autenticação e restrições de acesso.

Gabarito: Letra C.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

3.3 Scrum

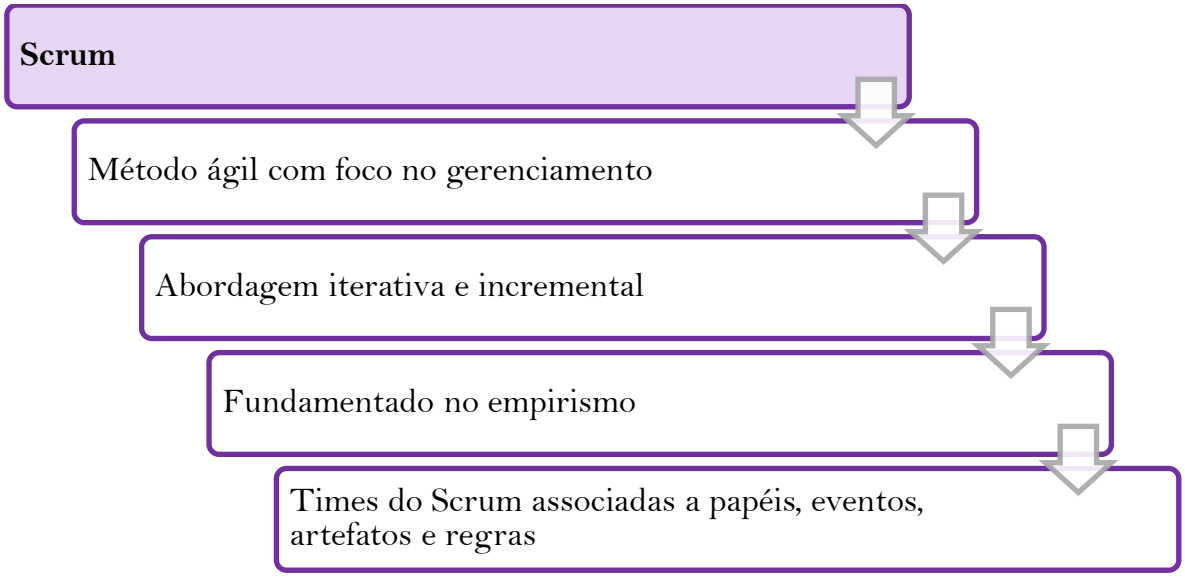
Como todos os outros processos profissionais de desenvolvimento de software, o desenvolvimento ágil tem de ser gerenciado de modo que se faça o melhor uso com o tempo e os recursos disponíveis para a equipe. Isso requer do gerenciamento de projeto uma abordagem diferente, adaptada para o desenvolvimento incremental e para os pontos fortes dos métodos ágeis.

A abordagem **Scrum** é um **método ágil geral**, mas seu **foco está no gerenciamento do desenvolvimento iterativo**, ao invés das abordagens técnicas específicas da engenharia de software ágil.

O **Scrum** é um framework dentro do qual pessoas podem **tratar e resolver problemas complexos e adaptativos**, enquanto **produtiva e criativamente entregam produtos com o mais alto valor** possível.

O **framework Scrum** consiste nos **times do Scrum associadas a papéis, eventos, artefatos e regras**. Cada componente dentro do framework serve a um propósito específico e é essencial para o uso e sucesso do Scrum.

O **Scrum** é **fundamentado nas teorias empíricas de controle de processo, ou empirismo**. O empirismo afirma que o conhecimento vem da experiência e de tomada de decisões baseadas no que é conhecido. O Scrum emprega uma **abordagem iterativa e incremental para aperfeiçoar a previsibilidade e o controle de riscos**.



Esquema 44 – Scrum.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

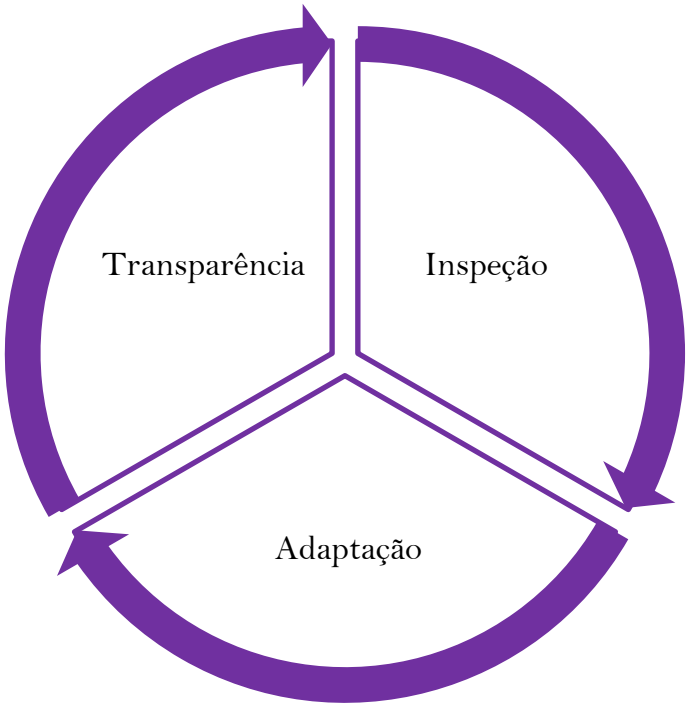
3.3.1 Pilares do Scrum

Três pilares apoiam a implementação de controle de processo empírico:

- **Transparência**: o **processo emergente e o trabalho devem ser visíveis tanto para quem executa o trabalho quanto para quem recebe** o trabalho. Com o Scrum, decisões importantes são baseadas no estado percebido de seus três artefatos formais. Artefatos com baixa transparência podem levar a decisões que diminuem o valor e aumentam o risco. A transparência permite a inspeção. A inspeção sem transparência é enganosa e gera desperdício.

- **Inspeção**: os **artefatos do Scrum e o progresso em direção às metas acordadas devem ser inspecionados com frequência e diligência** para detectar variações ou problemas potencialmente indesejáveis.

- **Adaptação**: se algum aspecto de um processo se desviar fora dos limites aceitáveis ou se o produto resultante for inaceitável, o processo que está sendo aplicado ou os materiais que estão sendo produzidos devem ser ajustados. O **ajuste deve ser feito o mais rápido possível para minimizar novos desvios**.



Esquema 45 – Pilares do Scrum.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

35- (CESPE - 2019 - SLU-DF - Analista de Gestão de Resíduos Sólidos - Informática) Acerca da gestão ágil de projetos com Scrum, julgue o item a seguir.

Entre os processos da gestão de projetos com Scrum, as inspeções constituem os processos mais complexos e formais e, por isso, ocorrem somente ao fim de um ciclo de várias sprints, após a liberação de uma funcionalidade plena e o seu reconhecimento pelo demandante.

Resolução:

As inspeções devem ser realizadas de forma constante e não apenas no final de uma liberação.

Segundo o princípio da **Inspeção**, os **usuários Scrum devem, frequentemente, inspecionar os artefatos Scrum e o progresso em direção a detectar variações**. Esta inspeção não deve, no entanto, ser tão frequente que atrapalhe a própria execução das tarefas. As inspeções são mais benéficas quando realizadas de forma diligente por inspetores especializados no trabalho a se verificar.

Gabarito: Errado.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

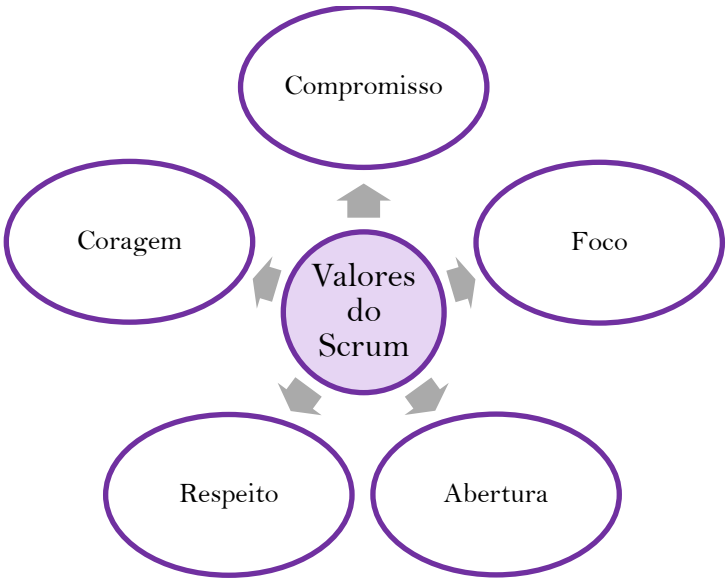
3.3.2 Valores do Scrum

O sucesso do uso do Scrum depende das pessoas se tornarem mais proficientes em viver **cinco valores**:

1. Compromisso;
2. Foco;
3. Abertura;
4. Respeito;
5. Coragem.

O Scrum Team se **compromete** a atingir seus objetivos e suportar uns aos outros. Seu **foco** principal é o trabalho da Sprint para fazer o melhor progresso possível em direção a essas metas. O Scrum Team e seus stakeholders são **abertos** quanto ao trabalho e os desafios. Os membros do Scrum Team se **respeitam** quanto a serem pessoas capazes e independentes, e são respeitados como tal pelas pessoas com quem trabalham. Os membros do Scrum Team têm a **coragem** de fazer a coisa certa e trabalhar em problemas difíceis.

As decisões que são tomadas, os passos dados e a forma como o Scrum é usado devem reforçar esses valores, não diminuí-los ou miná-los. Os membros do Scrum Team aprendem e exploram os valores à medida que trabalham com os eventos e artefatos do Scrum. Quando esses valores são incorporados pelo Scrum Team e pelas pessoas com quem trabalham, os pilares empíricos do Scrum de transparência, inspeção e adaptação ganham vida, construindo confiança.



Esquema 46 – Valores do Scrum.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

3.3.3 Time Scrum

O **Time Scrum** é composto pelo Product Owner, os Desenvolvedores e o Scrum Master. **Times Scrum** são **autogerenciáveis e multifuncionais**. Times autogerenciáveis decidem internamente quem faz o quê, quando e como. Times multifuncionais possuem todas as habilidades necessárias para criar valor a cada Sprint. Dentro de um Time, **não há sub-times ou hierarquias**.

Vejamos as responsabilidades específicas dentro do Scrum:

- **Product Owner (dono do produto)**: **responsável por maximizar o valor do produto e do trabalho do Time de Desenvolvimento**. Como isso é feito pode variar amplamente através das organizações, Times Scrum e indivíduos. É a **única pessoa responsável por gerenciar o Backlog** do Produto. O Product Owner pode **pode delegar** a responsabilidade a outros, mas, independentemente disso, **ainda é o responsável**.

- **Desenvolvedores**: pessoas do Scrum Team que estão comprometidas em **criar qualquer aspecto de um Incremento utilizável a cada Sprint**. Desenvolvedores são sempre responsáveis por:
  - Criar um plano para a Sprint, o Spring Backlog;
  - Introduzir gradualmente qualidade aderindo a uma definição de Pronto;
  - Adaptar seu plano a cada dia em direção à meta da Sprint; e,
  - Responsabilizar-se mutuamente como profissionais.

- **Scrum Master**: responsável por **garantir que o Scrum seja entendido e aplicado**. O Scrum Master faz isso para garantir que o Time Scrum adere à teoria, práticas e regras do Scrum. O Scrum Master é um servo-líder para o Time Scrum. O Scrum Master ajuda aqueles que estão fora do Time Scrum a entender quais as suas interações com o Time Scrum são úteis e quais não são. O Scrum Master ajuda todos a mudarem estas interações para maximizar o valor criado pelo Time Scrum.

Esquema 47 – Time Scrum.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

36- (FCC - 2019 - SANASA Campinas - Analista de Tecnologia da Informação - Análise e Desenvolvimento) Um Analista de TI tem como tarefas ordenar os itens do Backlog do Produto visando o alcance das metas e missões do projeto, buscando garantir que o Backlog do Produto esteja claro de forma a mostrar no que o Time Scrum vai trabalhar a seguir e ainda visando garantir que o Time de Desenvolvimento entenda os itens do Backlog do Produto no nível necessário. Considerando que o projeto é baseado no Scrum, o Analista está no papel de

a) Scrum Master.

b) Gerente do Produto.

c) Sprint Manager.

d) Product Owner.

e) Development Team Leader.

Resolução:

Quem é responsável por priorizar os itens do backlog é o **Product Owner (Dono do Produto)**.

Vamos analisar cada um dos itens:

a) **Incorreto**: o **Scrum Master** é o responsável por **garantir que o Scrum seja entendido e aplicado**. O Scrum Master faz isso para garantir que o Time Scrum adere à teoria, práticas e regras do Scrum. O Scrum Master é um servo-líder para o Time Scrum. O Scrum Master ajuda aqueles que estão fora do Time Scrum a entender quais as suas interações com o Time Scrum são úteis e quais não são.

b) **Incorreto**: **Gerente do Produto** não é papel em equipe Scrum.

c) **Incorreto**: **Sprint Manager** não é papel em equipe Scrum.

d) **Correto**: **Product Owner** é o **responsável por maximizar o valor do produto e do trabalho do Time de Desenvolvimento**. Como isso é feito pode variar amplamente através das organizações, Times Scrum e indivíduos. É a **única pessoa responsável por gerenciar o Backlog** do Produto.

e) **Incorreto**: **Development Team Leader** não é papel em equipe Scrum.

Gabarito: Letra D.

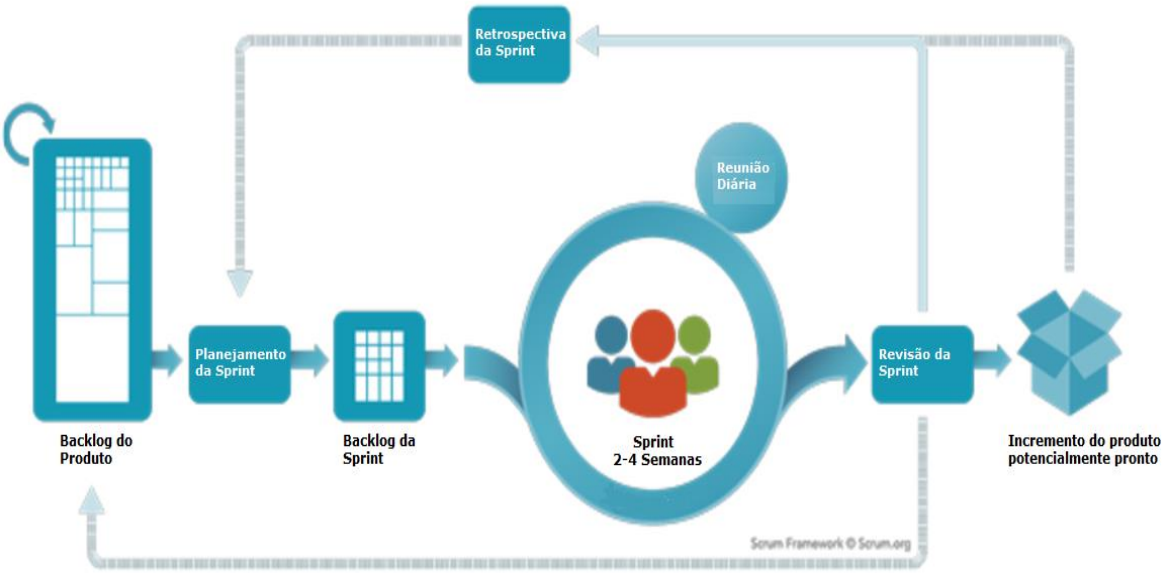
Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

3.3.4 Estrutura do Scrum

O **Scrum** está estruturado em um conjunto de **práticas conduzidas por equipes em papéis específicos, organizadas em um fluxo de atividades/eventos de duração fixa totalmente controlado**, com artefatos e regras bem definidos, que visa a obtenção de produtos utilizáveis em intervalos curtos de tempo.

As **práticas do SCRUM** estão baseadas em iterações sucessivas de atividades de desenvolvimento, as chamadas **sprints**, (cada uma delas gerando um incremento de produto), inspecionadas e ajustadas diariamente por membros da própria equipe de trabalho, e orientadas por uma lista de requisitos inicial (**backlog** do produto).

O processo preconizado pelo Scrum é demonstrado na figura a seguir:



Esquema 48 – Processo do Scrum.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

3.3.5 Eventos do Scrum

Os **eventos** estabelecidos no Scrum são **utilizados para criar regularidade e para minimizar a necessidade de reuniões não definidas no Scrum**. Todos os eventos têm janela temporal pré-definida, sendo que todos eles têm uma duração máxima estipulada. Assim que o Sprint começa, a sua duração é fixa e não pode ser reduzida ou aumentada. Os eventos restantes podem terminar sempre que a finalidade do evento é atingida, assegurando que é utilizada um tempo sem permitir desperdícios no processo.

O **Sprint** é um elemento vital no Scrum, com **duração limitada de um mês ou menos, durante o qual se cria um incremento potencialmente comercializável**. Os Sprints terão de ter durações consistentes durante todo o esforço de desenvolvimento. Um novo Sprint começa imediatamente após a conclusão do Sprint anterior.

Durante o Sprint:

- Nenhuma mudança é feita que coloque em risco a meta da Sprint;
- A qualidade não diminui;
- O Product Backlog é refinado conforme necessário; e,
- O escopo pode ser esclarecido e renegociado com o Product Owner conforme mais é aprendido.

Os Sprints contêm e consistem na reunião de Planejamento de Sprint, nas reuniões diárias ("Daily Scrums"), no trabalho de desenvolvimento em si, na Revisão de Sprint e na Retrospectiva de Sprint.

- No **planejamento do Sprint**, **o trabalho a executar no Sprint é planejado** na reunião de Planejamento do Sprint. Este plano é criado recorrendo ao trabalho colaborativo de toda a Equipe Scrum. O Planejamento do Sprint tem uma duração máxima limitada de oito horas para um Sprint de um mês. Para Sprints menores este evento será naturalmente menor. Um Planejamento do Sprint responde às seguintes questões:
  - Por que esta Sprint é valiosa?
  - O que pode ser feito nesta Sprint?
  - Como o trabalho escolhido será realizado?

- A **Reunião diária (Daily Scrum ou Stand-up Meetings)** é uma **reunião de duração limitada a 15 minutos para os desenvolvedores sincronizarem atividades e criar um plano para as próximas 24 horas**. Isto é feito inspecionando o trabalho desde a última Reunião Diária e prevendo o trabalho que poderá ser realizado até à próxima. A Reunião de Sincronização Diária acontece, como o próprio nome indica, diariamente, sempre à mesma hora e no mesmo local para reduzir complexidade. Durante a reunião os membros da equipe de desenvolvimento explicam:
  - O que é que eu fiz ontem que ajudou a Equipe de Desenvolvimento a atingir o Objetivo do Sprint?
  - O que é que vou fazer hoje para ajudar a Equipe de Desenvolvimento a atingir o Objetivo do Sprint?

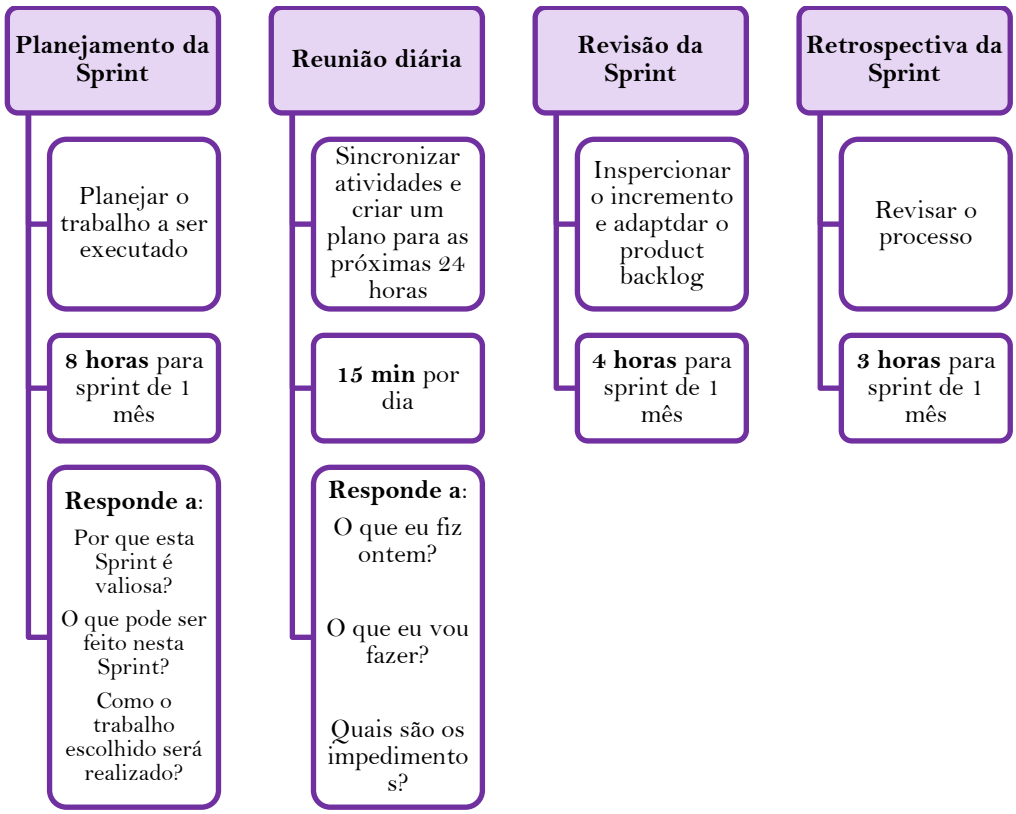
Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

- Prevejo algum impedimento que faça com que eu ou a Equipe de Desenvolvimento não atinja o Objetivo do Sprint?

- No final do Sprint ocorre uma reunião denominada **Revisão do Sprint** que serve para **inspecionar o incremento e adaptar o Product Backlog se necessário**. Durante a revisão do Sprint a Equipe Scrum e os demais Stakeholders discutem sobre o que foi feito durante o Sprint. Os participantes discutem as próximas atividades que podem ser executadas para otimizar valor. Esta reunião é delimitada a quatro horas para Sprints de um mês. Para Sprints menores o evento é normalmente menor. O resultado da revisão do Sprint é um Product Backlog revisto que define os itens do Product Backlog prováveis para o próximo Sprint. O Product Backlog também pode ser ajustado no geral para ir ao encontro de novas oportunidades.

- A reunião de **Retrospectiva do Sprint** é uma oportunidade para a Equipe Scrum **revisar o seu processo de desenvolvimento à luz das práticas e do modelo de processo do Scrum**, visando torná-lo mais eficaz e gratificante para a próxima Sprint. Esta reunião é delimitada a três horas para Sprints de um mês. Para Sprints menores tomará naturalmente menos tempo.

Os eventos do Scrum são resumidos a seguir:



Esquema 49 – Eventos Scrum.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

37- (IF-PA - 2019 - IF-PA - Analista de Tecnologia da Informação – Desenvolvimento de Sistemas) A gestão de projetos é um dos grandes desafios no desenvolvimento de produtos de software, pois uma gestão padronizada, aliada às boas práticas de desenvolvimento minimizam os fracassos nos projetos de softwares. O Scrum é um dos frameworks mais utilizados na gestão de projetos de software e sobre ele é correto afirmar.

a) Por definição, o seu ciclo é composto pela seguinte sequência: "Sprint", "Sprint View" e "Daily Scrum".

b) Por definição, "Sprint Review" é uma reunião informal que ocorre ao final de cada "Sprint" para avaliar o que foi feito, e se necessário, adaptar o "Backlog do Produto".

c) O "Sprint Retrospective" é um plano feito pelo "Product Owner", que demonstra como se espera que o produto evolua ao longo do tempo.

d) O "Daily Review" é um evento de curta duração realizado todos os dias durante um "Sprint", neste evento, a equipe de desenvolvimento planeja o trabalho das próximas 24 horas.

e) É de responsabilidade exclusiva do "Scrum Master" a gerência dos itens do "Product Backlog", bem como as suas prioridades e objetivos do projeto.

Resolução:

Quem é responsável por priorizar os itens do backlog é o **Product Owner (Dono do Produto)**.

É papel do Scrum Master garantir que a equipe esteja aderindo aos valores do Scrum.

Vamos analisar cada um dos itens:

a) **Incorreto**: Por definição, o seu ciclo é composto pela seguinte sequência: ~~"Sprint", "Sprint View" e "Daily Scrum"~~ **"Sprint Planning", "Daily Scrum" (uma para cada dia), "Sprint Review e Sprint Retrospective"**.

b) **Correto**: a "Sprint Review" é a reunião que serve para **inspecionar o incremento e adaptar o Product Backlog se necessário**.

c) **Incorreto**: O "Sprint Retrospective" **é uma reunião para a Equipe Scrum revisar o seu processo de desenvolvimento à luz das práticas e do modelo de processo do Scrum, visando torná-lo mais eficaz e gratificante para a próxima Sprint**.

d) **Incorreto**: O ~~"Daily Review"~~ **"Daily Scrum"** é um evento de curta duração realizado todos os dias durante um "Sprint", neste evento, a equipe de desenvolvimento planeja o trabalho das próximas 24 horas.

e) **Incorreto**: É de responsabilidade exclusiva do ~~"Scrum Master"~~ **"Product Owner"** a gerência dos itens do "Product Backlog", bem como as suas prioridades e objetivos do projeto.

Gabarito: Letra B.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

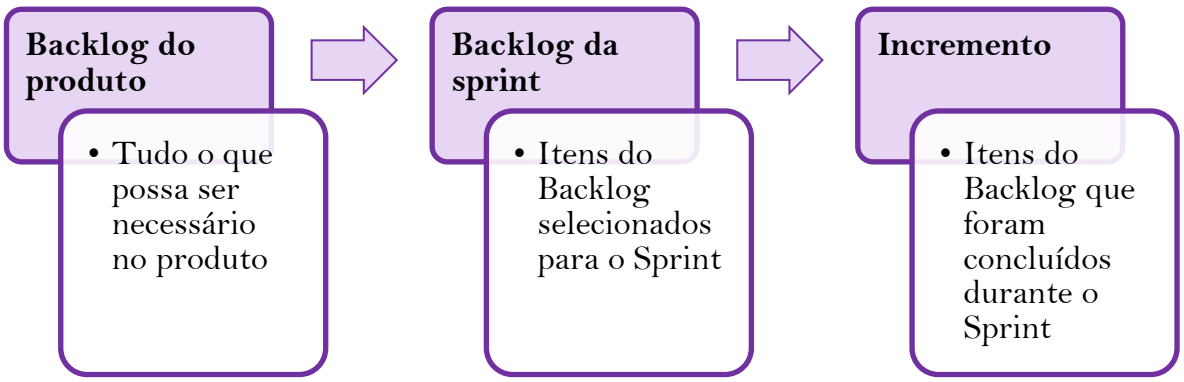
3.3.6 Artefatos do Scrum

Os **artefatos** do Scrum **representam trabalho ou valor que fornecem transparência e oportunidades para inspecionar e adaptar**. Artefatos definidos pelo Scrum são especificamente desenhados para maximizar a transparência de informação chave para que todas as pessoas tenham o mesmo entendimento sobre o artefato.

- **Backlog do Produto**: é uma **lista ordenada de tudo o que possa ser necessário no produto e é a única fonte de requisitos para quaisquer alterações efetuadas a esse produto**. O Product Owner é responsável pelo Product Backlog incluindo o seu conteúdo, disponibilidade e ordenação. Um Product Backlog nunca está completo. O seu desenvolvimento inicial apenas estabelece o que se conhece inicialmente e os requisitos mais bem entendidos. O Product Backlog evolui à medida que o próprio produto e o ambiente em que ele é utilizado evoluem. O Product Backlog é dinâmico; muda constantemente para identificar o que o produto necessita para ser apropriado, competitivo e útil. Enquanto um produto existir, o seu Product Backlog também existe.

- **Backlog do Sprint**: é o **conjunto de itens do Product Backlog selecionados para o Sprint, em conjunto com um plano para os entregar** e assim concretizar o Objetivo do Sprint. O Sprint Backlog é uma previsão dada pela Equipe de Desenvolvimento sobre quais as funcionalidades que vão fazer parte do próximo incremento assim como o trabalho necessário para entregar essa funcionalidade num estado concluído.

- **Incremento**: é a **soma de todos os itens pertencentes ao Product Backlog que foram concluídos durante o Sprint, juntamente com o valor de todos os incrementos concluídos** em Sprints anteriores. No final de um Sprint, o novo incremento tem de estar no estado "Pronto", o que significa que tem de estar num estado utilizável e respeitar a definição de "Pronto" da equipe, caso contrário não é considerado concluído.



Esquema 50 – Artefatos do Scrum.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

38- (CESPE - 2019 - MPC-PA - Analista Ministerial – Tecnologia da Informação)
A gestão ágil é uma das tendências nos projetos de desenvolvimento de software. O backlog é um dos artefatos que auxiliam na organização do projeto, em especial na definição das características tanto do produto (Product Backlog) quanto das Sprints (Sprint Backlog). Com relação a esses conceitos, assinale a opção correta.

a) O backlog da Sprint não pode ser alterado após sua elaboração.

b) O backlog do produto deve ser atualizado diariamente, para refletir novos requisitos a serem incorporados ao produto.

c) O backlog do produto é um dos produtos do backlog da Sprint.

d) O backlog da Sprint não deve ser embasado em Sprints anteriores, pois o tempo estimado para cada Sprint depende do Product Backlog.

e) O backlog da Sprint deve prever a duração de, no máximo, um mês para cada Sprint.

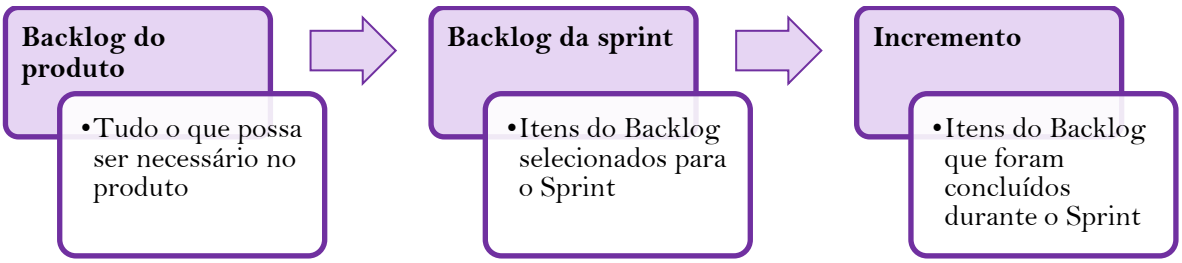
Resolução:

Vamos analisar cada um dos itens:

a) **Incorreto**: O backlog da Sprint ~~não~~ pode ser alterado após sua elaboração. Durante uma Sprint, não são feitas alterações que coloquem em perigo o objetivo do Sprint, mas isso não exclui alterações menores ou que não prejudiquem o seu objetivo.

b) **Incorreto**: O backlog do produto deve ser atualizado ~~diariamente~~ **quando necessário**, para refletir novos requisitos a serem incorporados ao produto.

c) **Incorreto**: O backlog ~~do produto~~ **da Sprint** é um dos produtos do backlog ~~da Sprint~~ **do produto**.



d) **Incorreto**: as lições aprendidas em sprints anteriores devem ser levadas para a próxima sprint. Para isso, temos inclusive a **reunião de retrospectiva** que busca **revisar o seu processo de desenvolvimento à luz das práticas e do modelo de processo do Scrum**, visando torná-lo mais eficaz e gratificante para a próxima Sprint.

e) **Correto**: O **Sprint** é um elemento vital no Scrum, com **duração limitada de um mês ou menos, durante o qual se cria um incremento potencialmente comercializável**.

Gabarito: Letra E.

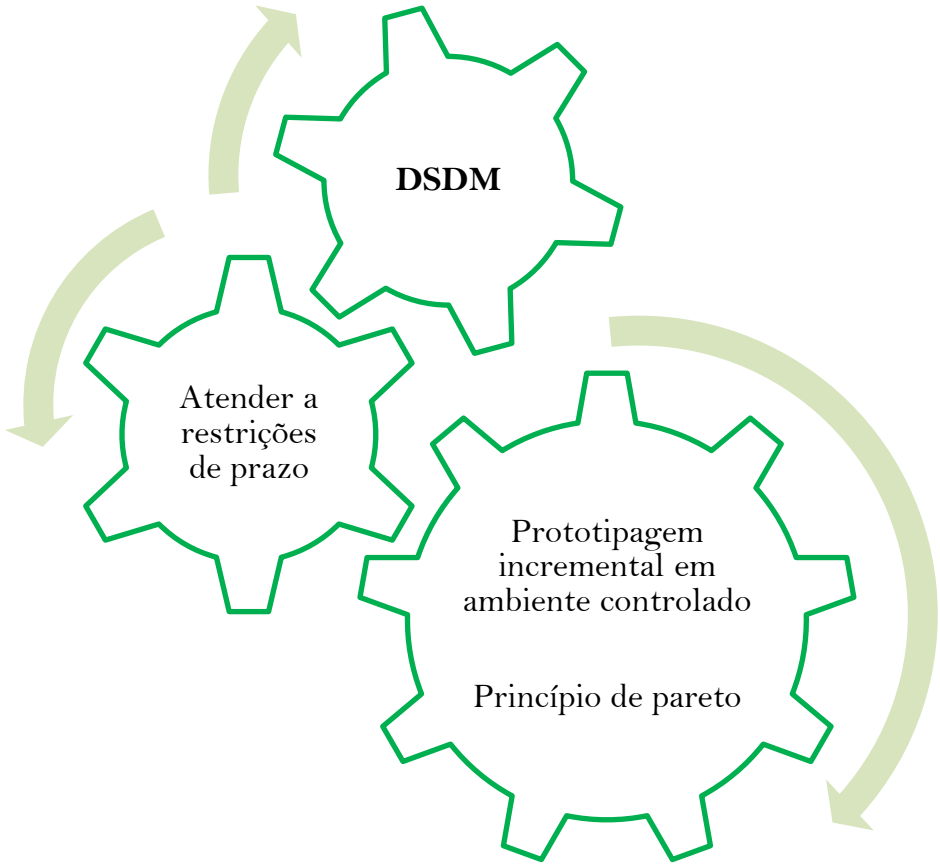
Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

3.4 DSDM

O **método de desenvolvimento de sistemas dinâmicos (Dynamic Systems Development Method - DSDM)** é uma abordagem de desenvolvimento de software ágil que oferece uma **metodologia para construir e manter sistemas que atendem restrições de prazo apertado através do uso da prototipagem incremental em um ambiente de projeto controlado**.

A filosofia **DSDM** baseia-se em uma **versão modificada do princípio de Pareto** — 80% de uma aplicação pode ser entregue em 20% do tempo que levaria para entregar a aplicação completa (100%).

O **DSDM** é um **processo de software iterativo em que cada iteração segue a regra dos 80%,** ou seja, somente o trabalho suficiente é requisitado para cada incremento, para facilitar o movimento para o próximo incremento. Os detalhes remanescentes podem ser completados depois, quando outros requisitos de negócio forem conhecidos ou alterações tiverem sido solicitadas e acomodadas.



Esquema 51 – Método de Desenvolvimento de Sistemas Dinâmicos (DSDM).

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

O DSDM Consortium (www.dsdm.org) é um grupo mundial que assume o papel de "mantenedor" do método. Esse consórcio definiu um modelo de processos ágeis, chamado **ciclo de vida DSDM** que define três ciclos iterativos diferentes, precedidos por duas atividades de ciclo de vida adicionais:

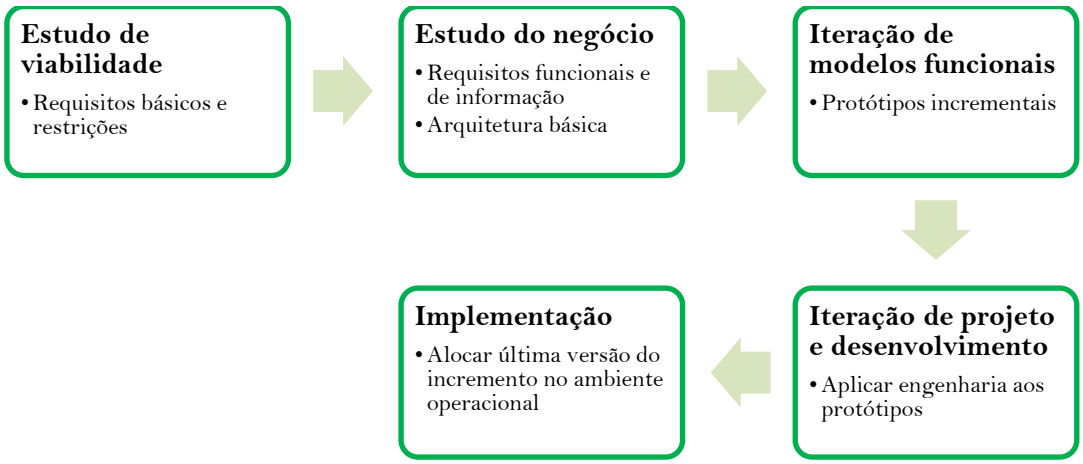
- **Estudo da viabilidade**: estabelece os **requisitos básicos de negócio e restrições associados** à aplicação a ser construída e depois avalia se a aplicação é ou não um candidato viável para o processo DSDM.

- **Estudo do negócio**: estabelece os **requisitos funcionais e de informação que permitirão à aplicação agregar valor de negócio; define também a arquitetura básica** da aplicação e identifica os requisitos de facilidade de manutenção para a aplicação.

- **Iteração de modelos funcionais**: produz um **conjunto de protótipos incrementais que demonstram funcionalidade para o cliente**. (Nota: Todos os protótipos DSDM são feitos com a intenção de que evoluam para a aplicação final entregue ao cliente.) Durante esse ciclo iterativo, o objetivo é juntar requisitos adicionais ao se obter feedback dos usuários, conforme testam o protótipo.

- **Iteração de projeto e desenvolvimento**: **revisita protótipos desenvolvidos durante a iteração de modelos funcionais para assegurar-se de que cada um tenha passado por um processo de engenharia** para capacitá-los a oferecer, aos usuários finais, valor de negócio em termos operacionais. Em alguns casos, ocorre ao mesmo tempo da iteração de modelos funcionais.

- **Implementação**: **aloca a última versão do incremento de software (um protótipo "operacionalizado") no ambiente operacional**. Deve-se notar que: (1) o incremento pode não estar 100% completo ou (2) alterações podem vir a ser solicitadas conforme o incremento seja alocado. Em qualquer dos casos, o trabalho de desenvolvimento do DSDM continua, retornando-se à atividade de iteração do modelo funcional.



Esquema 52 – Ciclo de vida DSDM.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

39- (FUNCAB - 2016 - CREA-AC - Analista de Sistema) No método de desenvolvimento de sistemas ágil denominado Método de Desenvolvimento de Sistemas Dinâmicos, ou DSDM, o ciclo interativo de Iteração de Modelos Funcionais é aquele no qual se:

a) alteram as linhas de comunicação cliente-equipe, com a confecção de uma lista dos aspectos que não estejam bem definidos.

b) constrói uma lista de funcionalidades principais e em cada uma faz-se um projeto independente, que será juntado ao final.

c) passa por um processo de engenharia para capacitar os módulos desenvolvidos, a fim de obter-se um valor operacional.

d) realizam diversas alterações incrementais nos módulos para produção da última versão do sistema.

e) produzem protótipos incrementais que servem para demonstrar as funcionalidades do sistema.

Resolução:

O **método de desenvolvimento de sistemas dinâmicos (Dynamic Systems Development Method - DSDM)** é uma abordagem de desenvolvimento de software ágil que oferece uma **metodologia para construir e manter sistemas que atendem restrições de prazo apertado através do uso da prototipagem incremental em um ambiente de projeto controlado**.

Gabarito: Letra E.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

3.5 FDD

O **desenvolvimento dirigido a funcionalidades (Feature Driven Development - FDD)** foi concebido como um **modelo de processos prático para a engenharia de software orientada a objetos**.

Como outras abordagens ágeis, o **FDD** adota uma filosofia que

1. Enfatiza a **colaboração entre pessoas da equipe** FDD;

2. Gerencia problemas e complexidade de projetos utilizando a **decomposição baseada em funcionalidades**, seguida pela integração dos incrementos de software, e

3. **Comunicação de detalhes técnicos usando meios verbais, gráficos e de texto**. O FDD enfatiza as atividades de garantia da qualidade de software por meio do encorajamento de uma estratégia de desenvolvimento incremental, o uso inspeções do código e do projeto, a aplicação de auditorias para garantia da qualidade de software, a coleta de métricas e o uso de padrões (para análise, projeto e construção).

No contexto do FDD, **funcionalidade** "é uma função valorizada pelo cliente passível de ser implementada em duas semanas ou menos".

A FDD é, classicamente, descrita por cinco processos separados na fase de concepção e planejamento que é executada uma vez para todo o projeto e na fase de construção é realizada para cada funcionalidade.

1. **Desenvolver um Modelo Abrangente**: pode envolver desenvolvimento de requisitos, análise orientada por objetos, modelagem lógica de dados e outras técnicas para **entendimento do domínio de negócio em questão**. O resultado é um modelo de objetos (e/ou de dados) de alto nível, que guiará a equipe durante os ciclos de construção.

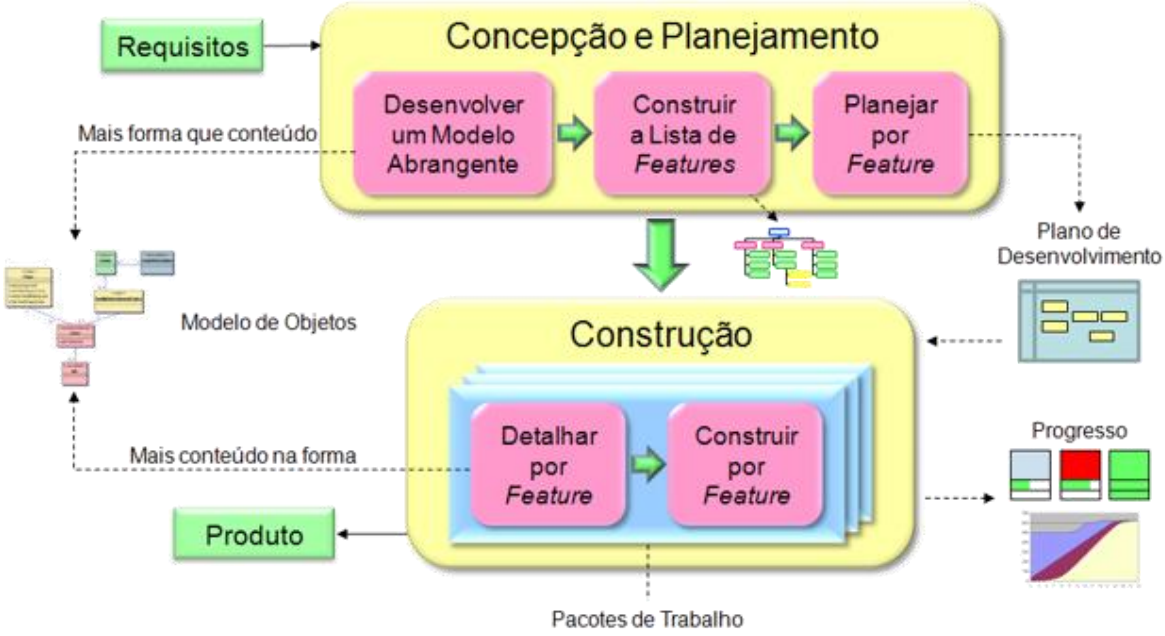
2. **Construir uma Lista de Funcionalidades**: decomposição funcional do modelo do domínio, em três camadas típicas: áreas de negócio, atividades de negócio e passos automatizados da atividade (funcionalidades). O resultado é uma **hierarquia de funcionalidades que representa o produto a ser construído** (também chamado de product backlog, ou lista de espera do produto).

3. **Planejar por Funcionalidade**: abrange a estimativa de complexidade e dependência das funcionalidades, também levando em consideração a prioridade e valor para o negócio/cliente. O resultado é um **plano de desenvolvimento, com os pacotes de trabalho na sequência apropriada para a construção**.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

4. **Detalhar por Funcionalidade**: já dentro de uma iteração de construção, a equipe detalha os requisitos e outros artefatos para a codificação de cada funcionalidade, incluindo os testes. O projeto para as funcionalidades é inspecionado. O resultado é o **modelo de domínio mais detalhado e os esqueletos de código prontos para serem preenchidos**.

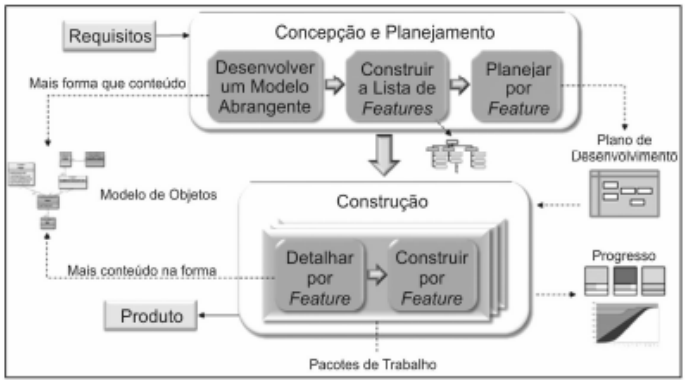
5. **Construir por Funcionalidade**: cada esqueleto de código é preenchido, testado e inspecionado. O resultado é um **incremento do produto integrado ao repositório principal de código**, com qualidade e potencial para ser usado pelo cliente/usuário.



Esquema 53 – Feature Driven Development – FDD.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

40- **(FCC - 2016 - PGE-MT - Analista - Analista de Sistemas)** Considere a figura abaixo.



Pelas características apresentadas na figura, este processo de software

a) é uma metodologia ágil para grandes equipes que desenvolvem software baseado em requisitos precisos, que pouco se modificam, e na qual o cliente não atua diretamente.

b) é adequado para projetos nos quais as versões devem ser entregues a cada mês, ou no máximo a cada seis meses, aumentando a possibilidade de feedback rápido do cliente.

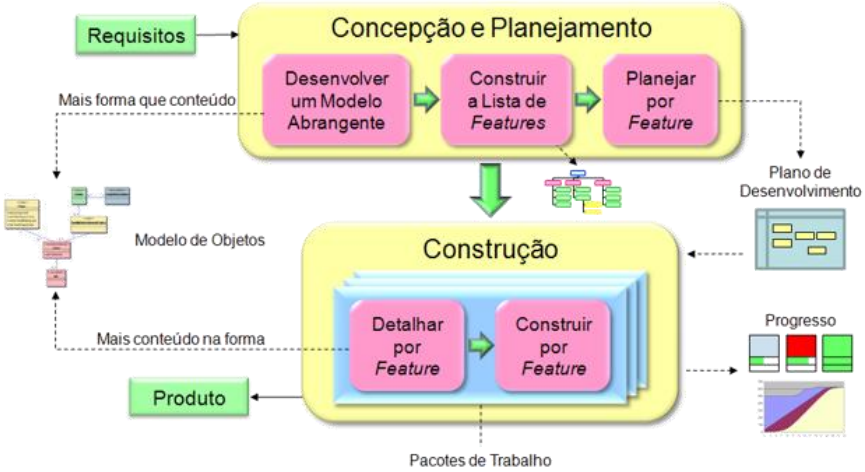
c) mantém as funcionalidades a serem implementadas em uma lista denominada Product Backlog ou Pacote de Trabalho. Na reunião de planejamento o Product Owner prioriza os itens dos Pacotes de Trabalho.

d) possui 2 fases: a fase de concepção e planejamento é executada uma vez para todo o projeto e a fase de construção é realizada para cada feature.

e) é conhecido como TDD – Test Driven Development e sua principal característica é que o código é comunitário, não pertencendo a um único desenvolvedor.

Resolução:

A FDD é, classicamente, descrita por cinco processos separados na fase de concepção e planejamento que é executada uma vez para todo o projeto e na fase de construção é realizada para cada funcionalidade.



Gabarito: **Letra D**.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

3.6 TDD

O **TDD (Test Driven Development) ou Desenvolvimento Orientado a Testes** é uma técnica ou filosofia de programação que incorpora o teste ao processo de produção de código da seguinte forma:

- Primeiramente o desenvolvedor, que recebe a especificação de uma nova funcionalidade a ser implementada, deve **programar um conjunto de testes** automatizados para testar o código que ainda não existe.
- Esse conjunto de testes deve ser **executado e falhar**. Isso é feito para mostrar que os testes efetivamente têm algum poder de teste e não terão sucesso a não ser que o código específico seja desenvolvido. Se o código passar em algum desses testes, é possível que ou o teste não seja efetivo, ou que a característica a ser implementada já exista no sistema.
- Em seguida, o **código deve ser desenvolvido** da forma mais minimalista possível, isto é, apenas para passar nos testes.
- Depois que o código **passar em todos os testes**, deve **ser refatorado** para atender a padrões de qualidade interna e testado novamente até que passe em todos os testes novamente.

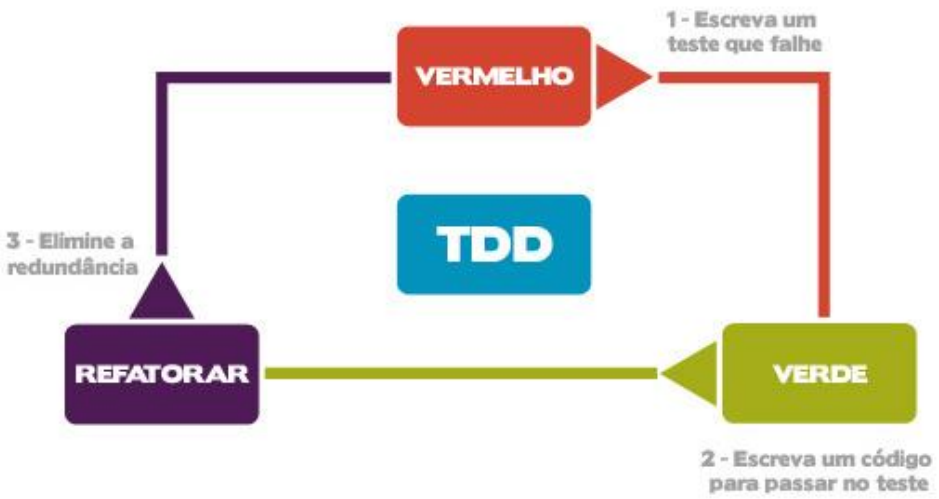
O ciclo do TDD é mostrado na imagem abaixo.

Esquema 54 – Ciclo TDD.

Em resumo, no **TDD**, o **teste é escrito e executado antes da programação ou codificação**.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

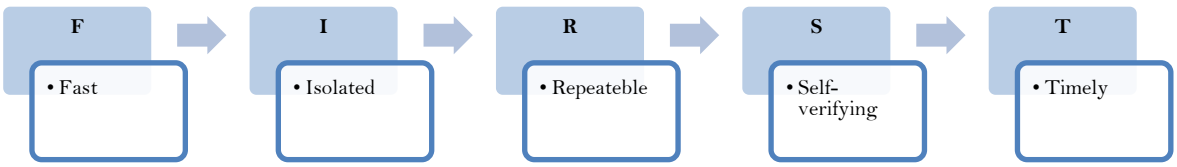
Simplificadamente, são usadas três cores para representar as fases do ciclo TDD:



Esquema 55 – Ciclo TDD Simplificado.

Os testes, inclusive no TDD, devem ser:

- **F (Fast) - Rápidos**: devem ser rápidos, pois testam apenas uma unidade;
- **I (Isolated) - Isolados**: Testes unitários são isolados, testando individualmente as unidades e não sua integração;
- **R (Repeateble) – Repetíveis:** Repetição nos testes, com resultados de comportamento constante;
- **S (Self-verifying) – Auto-verificáveis:** A auto verificação deve verificar se passou ou se deu como falha o teste;
- **T (Timely) – Tempestivos**: o teste deve ser oportuno, sendo um teste por unidade.



Esquema 56 – Características dos testes.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

41- (FCC - 2018 - MPE-PE - Analista Ministerial - Informática) Um Analista está trabalhando em um projeto de desenvolvimento de software utilizando o método Extreme Programming (XP) e executando testes com base no ciclo definido no Test-Driven Development (TDD). Nesse ciclo,

a) o código de programa é escrito e depois é submetido aos testes.

b) o teste escrito inicialmente falhará porque o código ainda não foi desenvolvido.

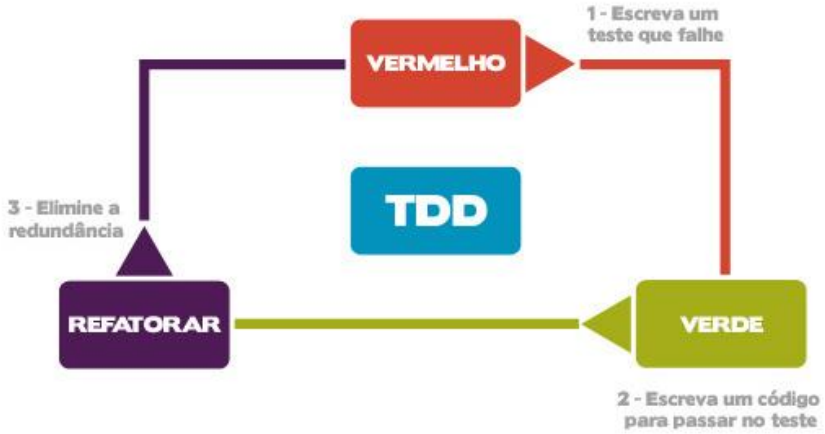
c) primeiro se escreve o teste de aceitação para depois escrever o código correspondente.

d) o código é considerado concluído assim que passa no primeiro teste.

e) cada teste engloba um conjunto de requisitos funcionais e/ou não funcionais.

Resolução:

O ciclo simplificado do TDD é apresentado a seguir:



Vamos analisar cada uma das alternativas:

a) **Incorreto**: os testes vêm antes do código.

b) **Correto**: exato, somente depois de o teste falhar é que será escrito o código para que se passe no teste.

c) **Incorreto**: primeiro se escreve o teste de aceitação **unidade** para depois escrever o código correspondente.

d) **Incorreto**: depois de o código passar nos testes, este ainda deve ser refatorado para atender a padrões de qualidade interna e testado novamente até que passe em todos os testes novamente.

e) **Incorreto**: cada teste engloba um conjunto de requisitos funcionais e/ou não funcionais.

Gabarito: Letra B.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

3.7 ATDD

De maneira análoga ao desenvolvimento orientado a testes, o **ATDD (Acceptance Test Driven Development)** envolve membros da equipe com diferentes perspectivas (cliente, desenvolvimento, teste) colaborando para **escrever testes de aceitação antes de implementar a funcionalidade correspondente**. As discussões colaborativas que ocorrem para gerar o teste de aceitação são frequentemente chamadas de três amigos, representando três perspectivas

- Do cliente (que problema estamos tentando resolver?),
- Desenvolvimento (como podemos resolver esse problema?)
- Teste (sobre o que…).

Esses testes de aceitação representam o ponto de vista do usuário e agem como uma forma de requisitos para descrever como o sistema funcionará, além de servir como uma maneira de verificar se o sistema funciona como pretendido. Em alguns casos, a equipe automatiza os testes de aceitação.

As etapas do ATDD são:

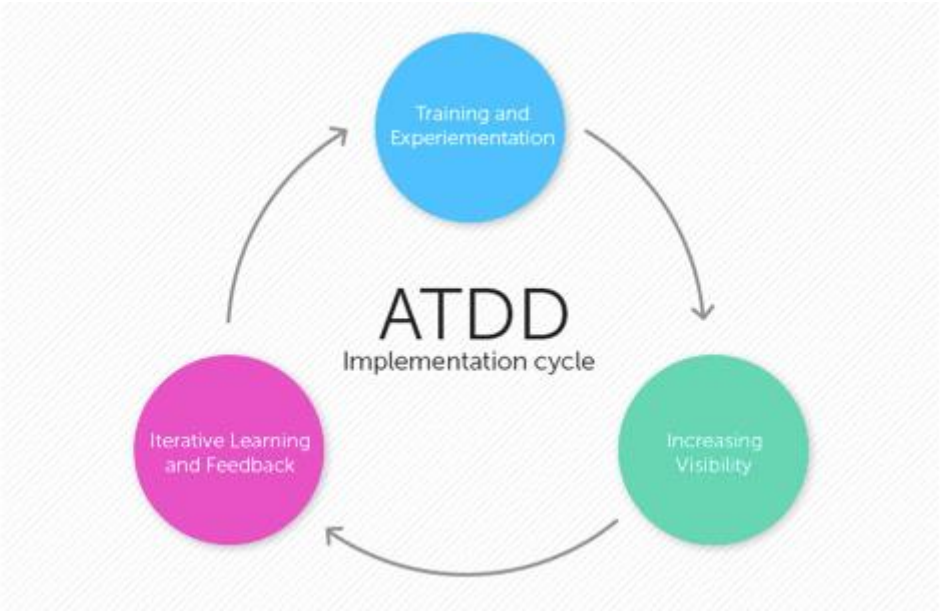
- **Treinamento e experimentação**: para trazer o entendimento comum e compartilhado sobre a equipe e sobre os objetivos da organização, é importante que haja um treinamento apropriado, especialmente quando for relacionado ao ATDD. Neste caso deveriam ser as metas, os objetivos, as expectativas e as informações sobre os resultados que surgirão a partir desta abordagem. Depois do treinamento, é importante começar a implementação em menor escala para testar coisas diferentes e receber um feedback iterativo.

- **Aumentando a visibilidade**: quando o projeto acontece através de múltiplas sprints, é fácil perder o controle dos diferentes processos que devem ser seguidos. Então a chave é criar metas, objetivos e expectativas visíveis para toda a equipe.

- **Feedback e aprendizagem iterativa**: ter um ciclo de feedback constante é crucial para a implementação de qualquer abordagem ágil, incluindo o ATDD. Fazer reuniões retrospectivas semanais ou quinzenais com toda a equipe é um caminho importante para conhecer quais partes do processo estão realmente beneficiando a equipe e quais precisam ser modificados.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

O ciclo de implementação do ATDD pode ser representado pela figura a seguir:



Esquema 57 – Ciclo de Implementação do ATDD.

O desenvolvimento com ATDD pode ser realizado com base em 4 etapas:

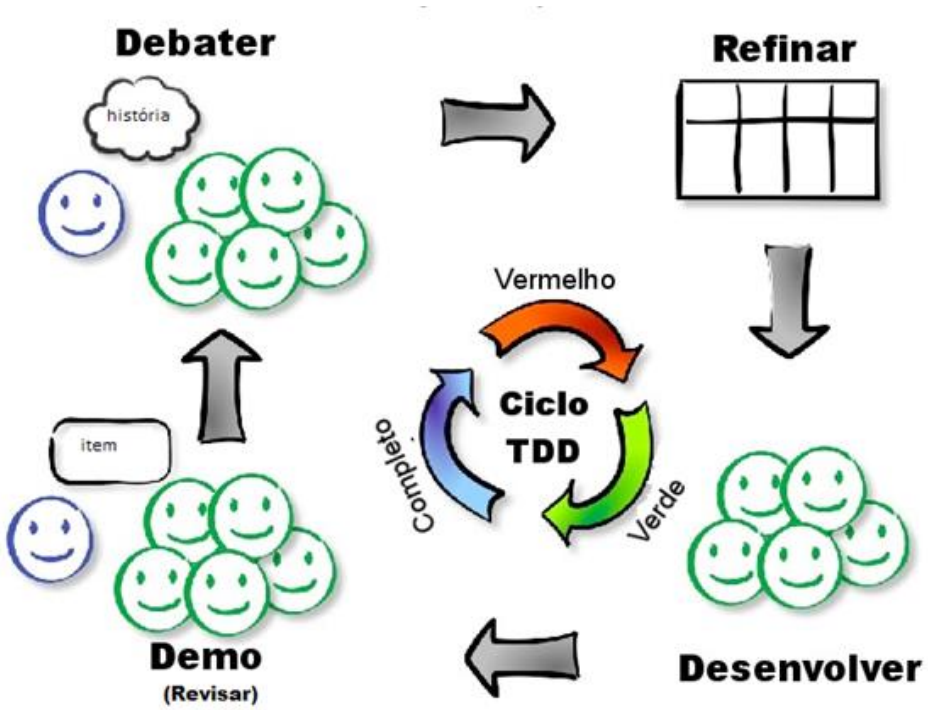
- **Debater os requisitos**: as histórias de usuário (user story) são refinadas em um workshop ou em uma reunião de preparação do backlog do produto (backlog grooming), antes da reunião de planejamento da iteração/sprint.

- **Refinar os testes de aceitação**: organizar os testes de aceitação em um formato requerido pelo framework de automação de testes. Existem diversos frameworks de automação de teste: FIT, FitNesse, Cucumber, Concordian, Robot Framework, RSpec, Jnario, entre outros.

- **Implementar o código com TDD**: implementar a funcionalidade para fazer com que o teste de aceitação passe. E, para tanto, o desenvolvimento deve iniciar pelos testes unitários, incluindo todas as condições propostas para as expectativas existentes.

- **Apresentar os Resultados dos Testes de Aceitação**: após os testes passarem com sucesso, a história é verificada pelo Product Owner, normalmente em uma reunião de Review/Showcase, onde ele poderá aprová-la ou não. O resultado pode levar à criação de uma nova história ou uma alteração nos testes existentes, afim de contemplar novos cenários.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07



Esquema 58 – Etapas do ATDD.

42- (FAURGS - 2018 - BANRISUL - Teste de Software) Considere as afirmações abaixo sobre desenvolvimento ágil orientado a testes de aceitação, em inglês Acceptance Test-Driven Development (ATDD).

I - Em ATDD, testes de aceitação devem ser especificados de forma colaborativa por desenvolvedores, clientes e testadores, de forma a reduzir a diferença entre teste de sistema e teste de aceitação.

II - Em ATDD, testes de aceitação são executados pelo cliente ou usuário apenas na última fase do projeto, sendo o propósito principal verificar se o sistema desenvolvido está funcionando de acordo com o que foi acordado.

III - Em ATDD, testes de aceitação oferecem um contexto para derivar testes para as diferentes unidades do projeto do software em TDD (Test-Driven Design – projeto orientado a teste).

IV - Em ATDD, testes de aceitação não podem ser automatizados.

Quais estão corretas?

a) Apenas I e II.

b) Apenas I e III.

c) Apenas II e III.

d) Apenas I, III e IV.

e) I, II, III e IV.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Resolução:

Vamos analisar cada um dos itens:

I – **Correto**: As discussões colaborativas que ocorrem para gerar o teste de aceitação são frequentemente chamadas de três amigos, representando três perspectivas

- Do cliente (que problema estamos tentando resolver?),
- Desenvolvimento (como podemos resolver esse problema?)
- Teste (sobre o que…).

II - **Incorreto**: os testes no ATDD são realizados antes mesmo da especificação dos requisitos e não apenas no final.

III - **Correto**: Em ATDD, testes de aceitação oferecem um contexto para derivar testes para as diferentes unidades do projeto do software em TDD (Test-Driven Design – projeto orientado a teste).

IV - **Incorreto**: Em ATDD, testes de aceitação ~~não~~ podem ser automatizados. Existem diversos frameworks de automação de teste: FIT, FitNesse, Cucumber, Concordian, Robot Framework, RSpec, Jnario, entre outros.

Gabarito: Letra B.

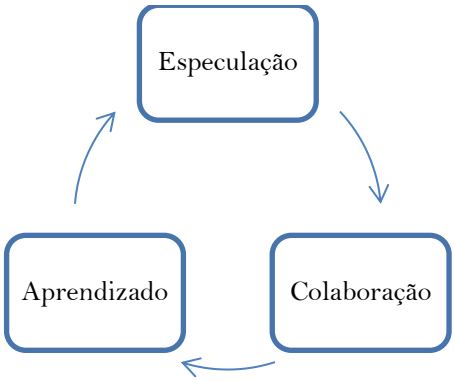
Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

3.8 ASD

O **Desenvolvimento Adaptativo de Sofware (ASD)** foi proposto por Jim Highsmith como uma técnica para **construção de software e sistemas altamente complexos**. Esse modelo se concentra na colaboração e auto-organização das equipes.

O criador do modelo Adaptativo define um ciclo de vida para o modelo baseando-se em três fases: especulação, colaboração e aprendizagem.

- Na fase de **especulação** o projeto é iniciado e tem-se o planejamento de ciclos adaptáveis. Esse planejamento de ciclos adaptáveis usa as informações contidas no início do projeto como: a missão do cliente, restrições do projeto e os requisitos básicos. Os requisitos básicos serão utilizados para definir o conjunto de ciclos da versão, ou seja, os incrementos de software operacional. Vale salientar que esse plano de ciclos sofrerá mudanças. Após completar cada ciclo o plano é revisto e ajustado para que tenhamos o trabalho reajustado à realidade que a equipe ASD está trabalhando.
- A **colaboração** é um tema bastante discutido e enfatizado nos métodos ágeis. A colaboração envolve confiança, críticas sem animosidade, auxílio, trabalho árduo, comunicação dos problemas ou preocupações de forma a conduzir ações efetivas, etc. Dessa forma, a colaboração ajuda bastante no levantamento de necessidades, especificações, etc.
- O ASD também enfatiza que o **aprendizado** é um elemento-chave para que possamos conseguir uma equipe auto-organizada. O criador do método Highsmith argumenta que os desenvolvedores superestimam o seu próprio entendimento quanto à tecnologia, processo ou mesmo quanto ao projeto. Dessa forma, Highsmith enfatiza que o aprendizado irá ajudar a todos os desenvolvedores a aumentar os níveis reais de entendimento. Com base nisso, as equipes ASD aprendem através de três maneiras: grupos focados, revisões técnicas e autópsias de projetos.



Esquema 59 – Ciclo do ASD.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

43- (FCC - 2019 - Prefeitura de Manaus - AM - Assistente Técnico de Tecnologia da Informação - Programador) A Fazenda Municipal, se optar por utilizar o método ágil denominado ASD (Adaptive Software Development) – Desenvolvimento Adaptativo de Software, deve observar o cumprimento das seguintes três fases componentes desse método:

a) especificação, inspeção e aprendizado.

b) planejamento, projeto e codificação.

c) qualificação, modelagem e negociação.

d) especulação, colaboração e aprendizado.

e) hierarquização, contextualização e validação.

Resolução:

O Desenvolvimento Adaptativo de Software (ASD) baseia-se em três fases:

- Na fase de **especulação** o projeto é iniciado e tem-se o planejamento de ciclos adaptáveis. Esse planejamento de ciclos adaptáveis usa as informações contidas no início do projeto como: a missão do cliente, restrições do projeto e os requisitos básicos. Os requisitos básicos serão utilizados para definir o conjunto de ciclos da versão, ou seja, os incrementos de software operacional. Vale salientar que esse plano de ciclos sofrerá mudanças. Após completar cada ciclo o plano é revisto e ajustado para que tenhamos o trabalho reajustado à realidade que a equipe ASD está trabalhando.
- A **colaboração** é um tema bastante discutido e enfatizado nos métodos ágeis. A colaboração envolve confiança, críticas sem animosidade, auxílio, trabalho árduo, comunicação dos problemas ou preocupações de forma a conduzir ações efetivas, etc. Dessa forma, a colaboração ajuda bastante no levantamento de necessidades, especificações, etc.
- O ASD também enfatiza que o **aprendizado** é um elemento-chave para que possamos conseguir uma equipe auto-organizada. O criador do método Highsmith argumenta que os desenvolvedores superestimam o seu próprio entendimento quanto à tecnologia, processo ou mesmo quanto ao projeto. Dessa forma, Highsmith enfatiza que o aprendizado irá ajudar a todos os desenvolvedores a aumentar os níveis reais de entendimento. Com base nisso, as equipes ASD aprendem através de três maneiras: grupos focados, revisões técnicas e autópsias de projetos.

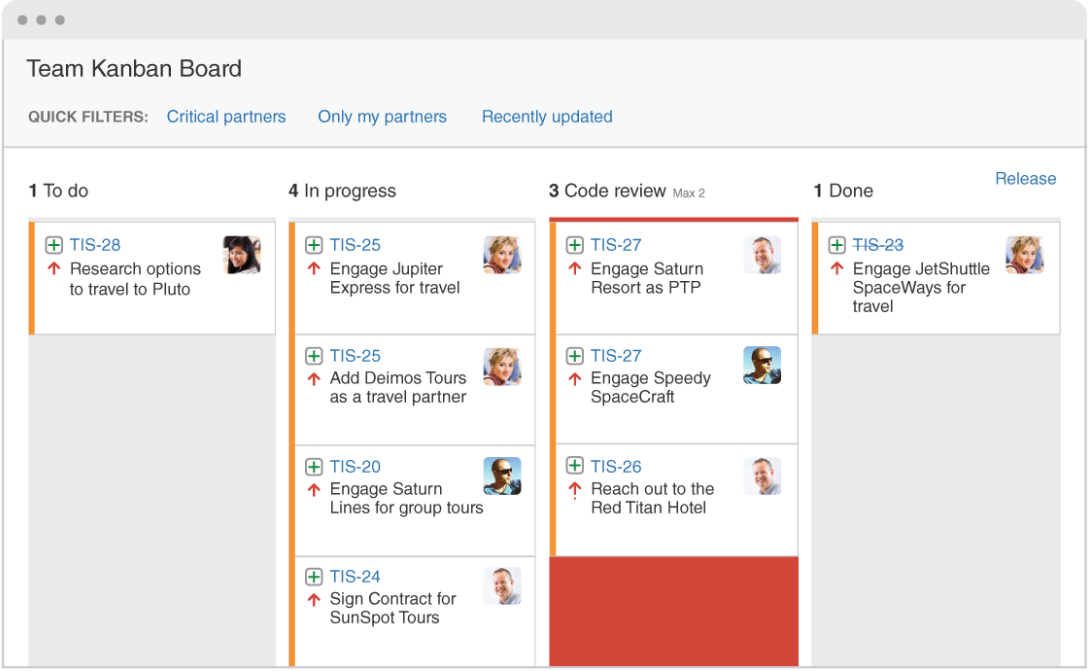
Gabarito: **Letra D**.

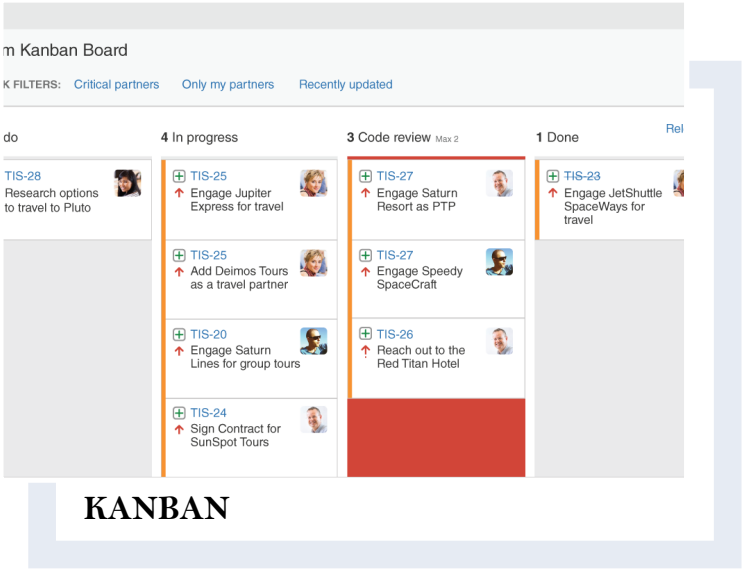
Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

3.9 Kanban

O **Kanban** é uma estrutura popular usada para implementar o desenvolvimento de software ágil. Ele precisa de comunicação de capacidade em tempo real e transparência total de trabalho. Os itens de trabalho ganham **representação visual** em um quadro Kanban, permitindo que os membros da equipe vejam o estado de cada parte do trabalho a qualquer momento.

O **Kanban** facilita a **criação de um fluxo de trabalho** de alta qualidade, capaz de entregar resultados dentro do prazo e livre de falhas. Na metodologia Kanban, as metas são entregues em um fluxo de trabalho pautado pela necessidade do cliente. Em outras palavras, há uma entrega contínua de resultados.





Representação visual do fluxo de trabalho.

Entrega contínua de resultados.

Esquema 60 – Kanban.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

44- **(CESPE - 2018 - BNB - Especialista Técnico - Analista de Sistema)** Julgue o próximo item, a respeito das metodologias ágeis Scrum e Kanban.

A implementação de um Kanban pressupõe a definição de um fluxo de trabalho pela equipe, o qual poderá ser revisto, mediante a inclusão ou a retirada de estágios, à medida que o trabalho evoluir.

Resolução:

O Kanban facilita a criação de um fluxo de trabalho de alta qualidade, capaz de entregar resultados dentro do prazo e livre de falhas. Na metodologia Kanban, as metas são entregues em um fluxo de trabalho pautado pela necessidade do cliente. Em outras palavras, há uma entrega contínua de resultados.

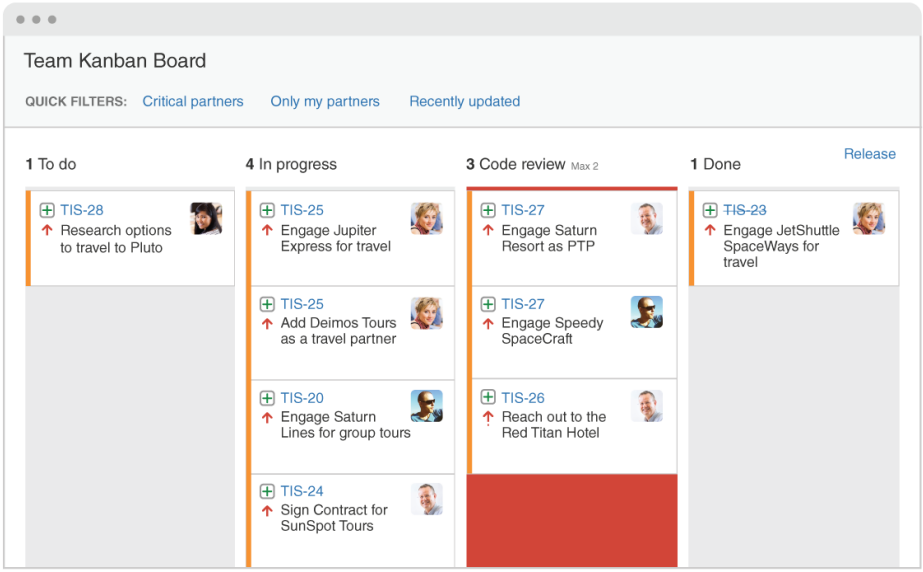
Gabarito: **Certo**.

45- **(CESPE - 2018 - BNB - Especialista Técnico - Analista de Sistema)** Julgue o item a seguir, relativo à gerência de projetos, de acordo com as abordagens do Kanban e do Scrum.

Quando utilizado para atividades de desenvolvimento de sistemas, o método Kanban ajuda a assimilar e a controlar o progresso das tarefas de forma visual.

Resolução:

O **Kanban** é uma estrutura popular usada para implementar o desenvolvimento de software ágil. Ele precisa de comunicação de capacidade em tempo real e transparência total de trabalho. Os itens de trabalho ganham **representação visual** em um quadro Kanban, permitindo que os membros da equipe vejam o estado de cada parte do trabalho a qualquer momento.



Gabarito: **Certo**.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

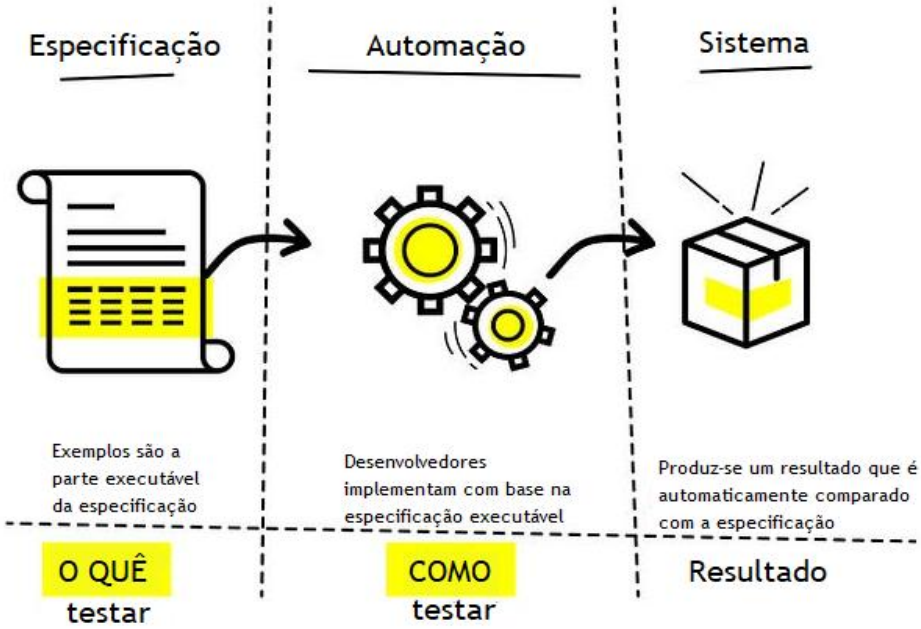
3.10 Especificação por exemplo

A **especificação por exemplo (SbE – Specification by Example)** é um **conjunto de práticas que ajuda a construir o produto certo, focando na comunicação entre pessoas envolvidas no projeto** de forma que todos possam ter o mesmo entendimento e que contribuam para o produto. Ela tem foco no negócio, preconizando uma linguagem comum e uma documentação que seja viva.

A **documentação viva** se refere ao fato de que ela acompanha o desenvolvimento do produto, sendo sempre **atualizada**. Ela pode ser **executável**, sendo utilizada na automatização de testes, garantindo a verificação de todas as mudanças. Ela é **simples de manter**, **colaborativa**, **esclarecedora** e **confiável**.

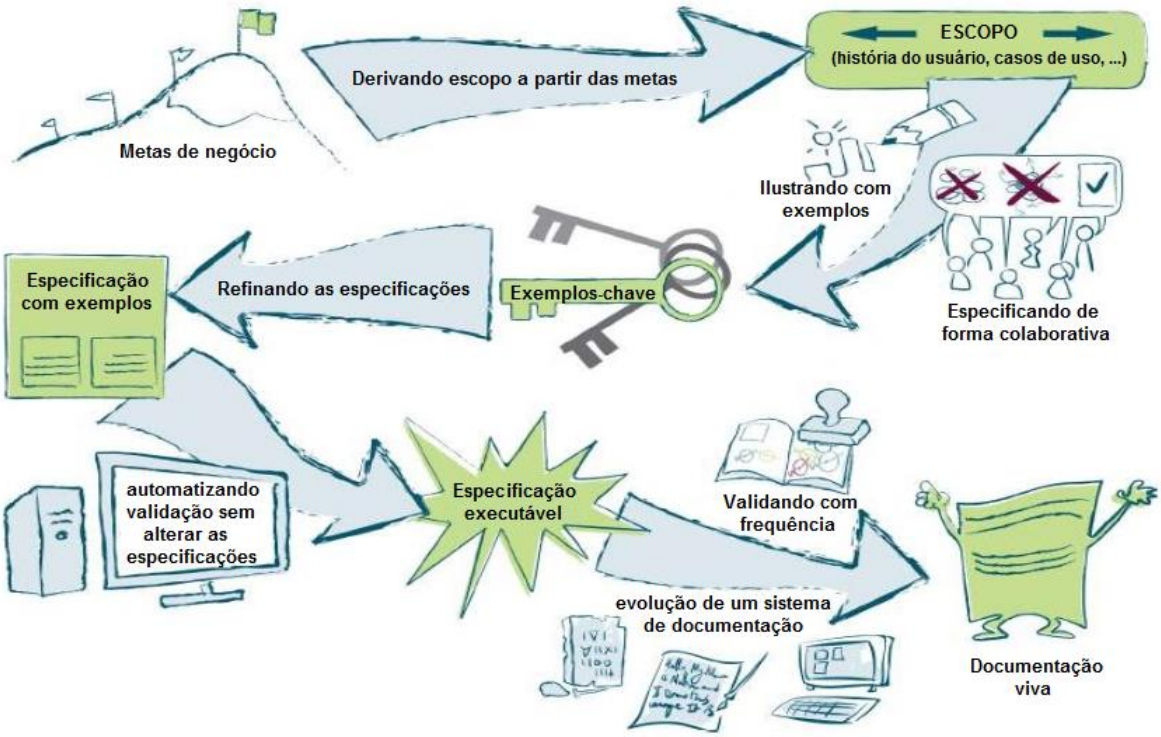
A **especificação por exemplo** foca **no QUE** deve ser feito e **NÃO NO COMO** deve ser feito. Como o próprio nome sugere, a especificação faz uso de **exemplos concretos para definir o comportamento do sistema**. A equipe de desenvolvimento trabalha com as partes interessadas para criar exemplos reais que demonstrem como o sistema deve e comportar em diferentes cenários.

O processo de especificação e desenvolvimento na especificação por exemplo pode ser visualizado de forma simples em 3 partes: especificação, automação e sistema.



Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Em uma visão expandida, a especificação trata de 7 processos, conforme figura a seguir:



Vejamos o que acontece em cada momento:

1. **Derivar o escopo a partir do objetivo**: ajuda a descobrir o que o cliente realmente precisa e dá liberdade para que o time de desenvolvimento possa sugerir a melhor forma de fazer isso. Deve-se entender qual o problema que o cliente quer resolver, ou a meta que ele quer atingir, e a partir daí todo o time colabora para chegar à solução.

2. **Especificar colaborativamente**: para alcançar o sucesso, é importante que todas as pessoas envolvidas colaborem. Assim, o entendimento é uniforme e um conhecimento compartilhado é construído. A colaboração pode ser realizada com workshops, escrita em pares e revisão das especificações pelos desenvolvedores.

3. **Ilustrar com exemplos**: é importante utilizar exemplos concretos do negócio e pensar nas saídas esperadas. Os exemplos ajudam a gerar discussão e eliminar as dúvidas.

4. **Refinar as especificações**: uma boa especificação vira um teste de aceitação; para isso, ela precisa ser: (1) precisa e testável; (2) não ser um script; (3) descrever o negócio e não o design de tela.

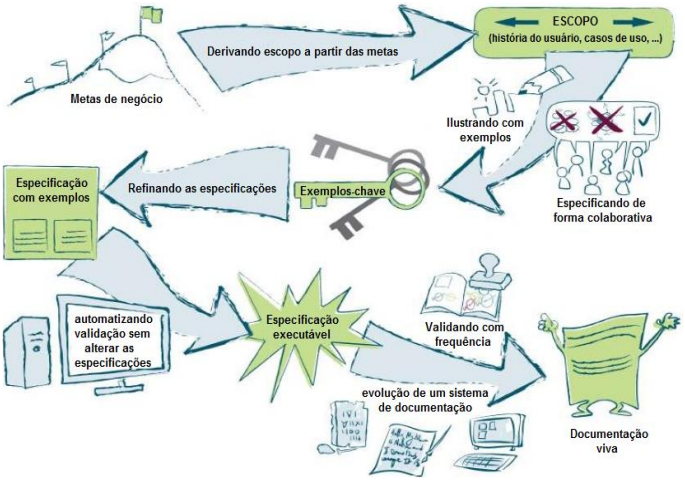
5. **Automatizar as especificações**: os testes automatizados ajudam a garantir que a documentação seja viva.

6. **Validar frequentemente**: utilizar um servidor de integração contínua para executar frequentemente os testes.

7. **Evoluir a documentação viva**: as especificações devem ser organizadas de acordo com um senso lógico e serem fáceis de entender e consistentes.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Em resumo:



Especificação por exemplo

- Práticas que ajuda a construir o produto certo, focando na comunicação entre pessoas envolvidas no projeto.

- Documentação viva.

- Foco no QUE e não no COMO.

- Exemplos concretos para definir o comportamento do sistema.

Esquema 61 – Especificação por exemplo.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

3.11 DDD

O **DDD (Domain Driven Design)** é uma abordagem para o desenvolvimento de software que **centra o desenvolvimento na programação de um modelo de domínio que tenha um rico entendimento dos processos e regras de um domínio**.

O DDD é baseado em três pilares:

- **Linguagem ubíqua**: a ideia central do DDD é que é necessária a construção de uma **linguagem ubíqua que incorpore a terminologia do domínio nos sistemas** de software que construímos. A **linguagem ubíqua** é uma **linguagem compartilhada por todos os membros da equipe de desenvolvimento e partes interessadas do negócio**. A linguagem ubíqua ajuda a garantir que todos estejam falando sobre o sistema usando o mesmo vocabulário e entendendo o mesmo significado. Isso pode ajudar a evitar mal-entendidos e ambiguidades, bem como a melhorar a comunicação e colaboração geral.

- **Contextos delimitados (bounded contexts)**: o DDD enfatiza a **separação de responsabilidades** entre as diferentes partes do sistema. O DDD **divide o sistema em diferentes módulos**, cada um com uma responsabilidade clara e distinta. Isso pode ajudar a simplificar o sistema e torná-lo mais fácil de entender e manter ao longo do tempo.

- **Mapas de contexto (context maps)**: utilizados para **entender como cada contexto delimitado se comunica com os demais contextos delimitados** e, consequentemente, como eles estão funcionando de maneira interdependente.

No DDD um modelo de domínio é a base da comunicação do projeto de software.

Mas o que seria um domínio? O domínio de um software são as atividades desempenhadas pelo usuário e a área de interesse destes. No DDD busca-se então a representação desse domínio em um modelo. Com um modelo é possível:

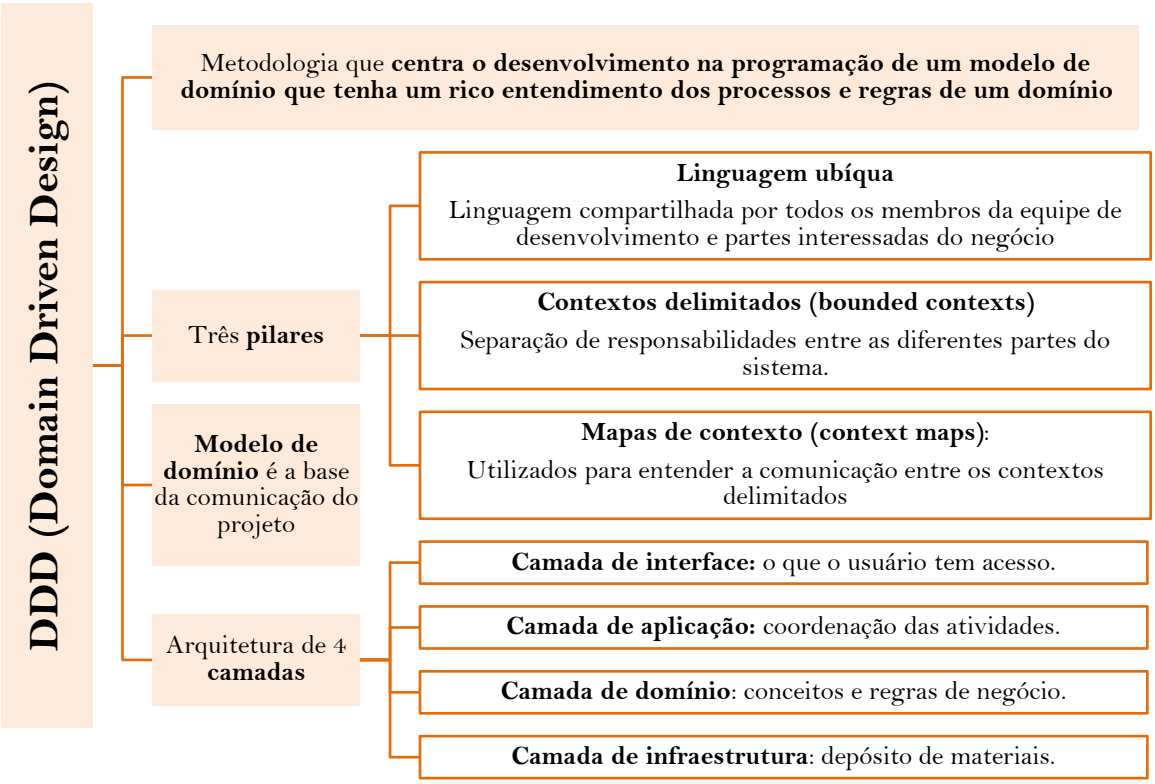
- **Abstrair a complexidade** do negócio através de uma representação simplificada.
- O modelo serve como **algo comum e palpável a todos** os membros do time, que, junto com a linguagem ubíqua, permite que todos possam participar ativamente da sua construção progressiva.
- O modelo (desde que feito corretamente) garante que **aquilo que está sendo especificado é o que está sendo implementado**.
- O modelo é o **meio de comunicação usado pelo time**. Graças ao vínculo entre o modelo e a implementação, os desenvolvedores podem falar na linguagem do software ao comunicarem-se com os especialistas do domínio (sem ter que traduzir a mensagem).

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

- O modelo é o **conhecimento destilado** - é o modo como o time concorda em estruturar o conhecimento extraído do domínio.
- O modelo é **evolutivo**, pois a cada iteração entre especialistas de domínio e a equipe técnica, o modelo se torna mais profundo e expressivo, mais rico, e os desenvolvedores transferem essa fonte de valor para o software.

O DDD se baseia em 4 camadas:

- **Camada de interface**: onde o usuário tem acesso. Nela que estão as informações e os comandos necessários para que o cliente utilize a ferramenta.
- **Camada de aplicação**: as atividades do sistema são coordenadas aqui. É a camada responsável por fazer as validações do que é feito no domínio.
- **Camada de domínio**: representa os conceitos e as regras do negócio, que são transmitidas para todo o sistema.
- **Camada de infraestrutura**: é a última camada da arquitetura e serve como depósito de materiais para as outras. Nela, estão as informações que são utilizadas para conexão de dados, gravação e envio de mensagens, entre outras funções.



Esquema 62 – DDD.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

46- (FGV - 2023 – SEF-MG – Auditor Fiscal da Receita Estadual – Tecnologia da Informação) Em domain-driven design (DDD), a linguagem ubíqua ou linguagem onipresente é um conceito central.

Assinale a opção que indica seu principal objetivo.

a) Documentar o domínio.

b) Providenciar um meio de comunicação com os especialistas do domínio.

c) Facilitar a comunicação entre especialistas e desenvolvedores.

d) Criar uma linguagem compartilhada que é usada durante o processo de desenvolvimento.

e) Criar convenções de nomenclatura.

Resolução:

Embora todos os itens, em maior ou menor grau, representem alguma característica ou facilidade de se utilizar a linguagem ubíqua, a questão pede o principal objetivo, logo o item d) é o que melhor representa esse objetivo.

A ideia central do DDD é que é necessária a construção de uma **linguagem ubíqua que incorpore a terminologia do domínio nos sistemas** de software que construímos. A **linguagem ubíqua** é uma linguagem compartilhada por todos os membros da equipe de desenvolvimento e partes interessadas do negócio. A linguagem ubíqua ajuda a garantir que todos estejam falando sobre o sistema usando o mesmo vocabulário e entendendo o mesmo significado. Isso pode ajudar a evitar mal-entendidos e ambiguidades, bem como a melhorar a comunicação e colaboração geral.

Gabarito: Letra D.

47- (CESPE / CEBRASPE - 2023 – SEPLAN RR – Analista de Planejamento e Orçamento – Tecnologia da Informação) Acerca de metodologias ágeis e linguagens de modelagem, julgue o item que se segue.

Em DDD (Domain-Drive Design), linguagem ubíqua é a linguagem empregada no contexto do projeto, que é entendida pela área de negócio e pelos desenvolvedores.

Resolução:

A ideia central do DDD é que é necessária a construção de uma **linguagem ubíqua que incorpore a terminologia do domínio nos sistemas** de software que construímos. A **linguagem ubíqua** é uma linguagem compartilhada por todos os membros da equipe de desenvolvimento e partes interessadas do negócio. A linguagem ubíqua ajuda a garantir que todos estejam falando sobre o sistema usando o mesmo vocabulário e entendendo o mesmo significado. Isso pode ajudar a evitar mal-entendidos e ambiguidades, bem como a melhorar a comunicação e colaboração geral.

Gabarito: Certo.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

48- (CESPE / CEBRASPE - 2022 – DP DF – Analista de Apoio à Assistência Judiciária – Informática – Banco de Dados) Quanto ao domain driven design (DDD), julgue o item a seguir.

O modelo é estático no DDD; caso ele se torne complexo, caberá aos desenvolvedores transferirem a abstração da complexidade para o software.

Resolução:

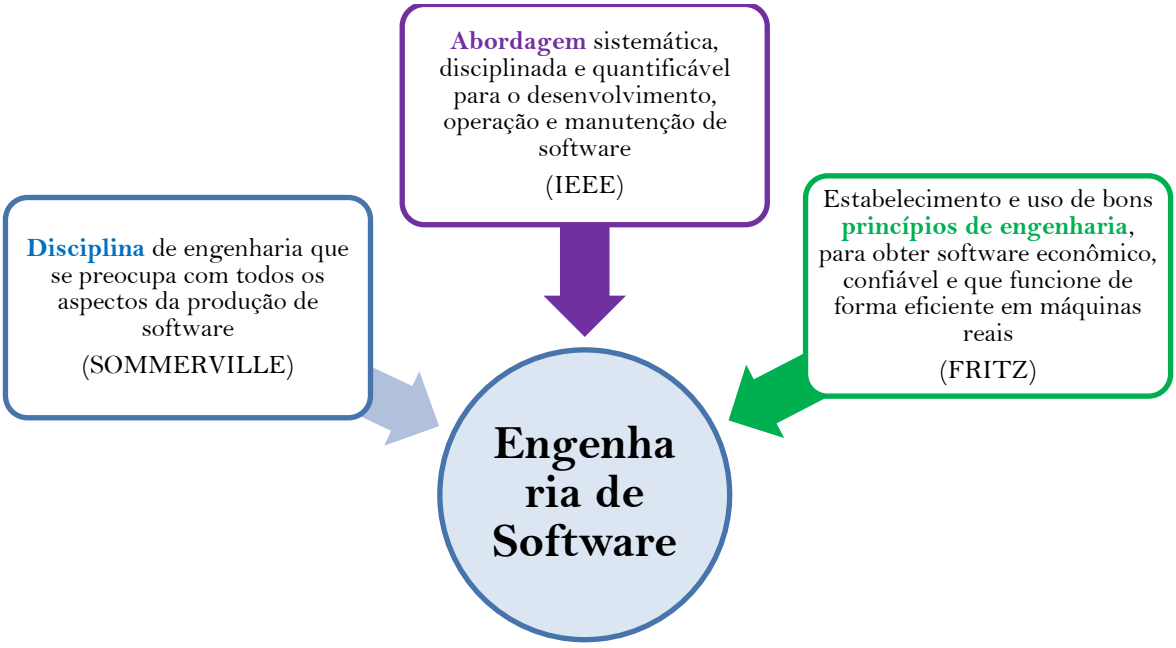
O DDD não é estático, mas sim evolutivo. Além disso, o modelo visa abstrair a complexidade do negócio através de uma representação simplificada.

Gabarito: Errado.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

4. ESQUEMAS DE AULA

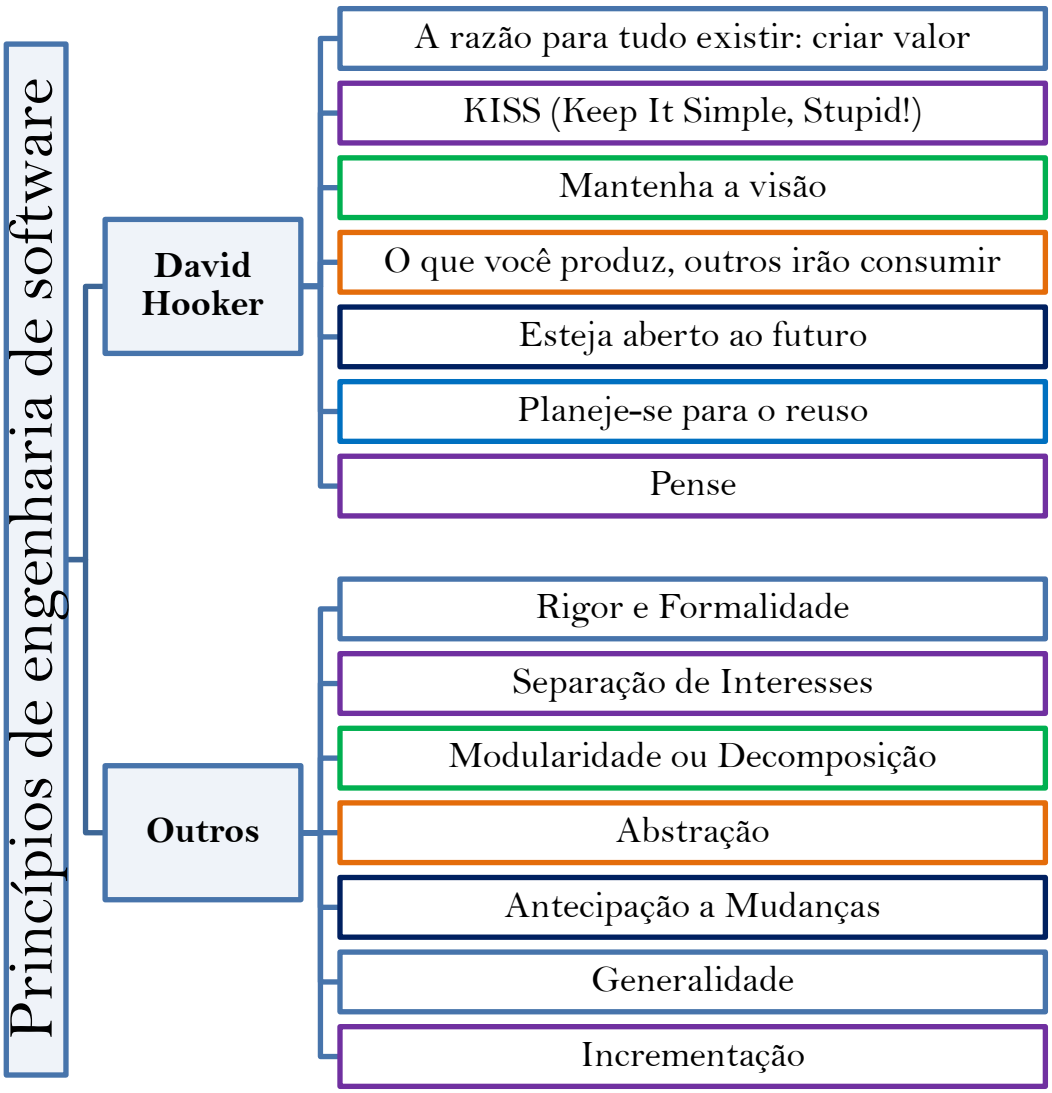
Engenharia de Software.



Engenharia de Software em Camadas

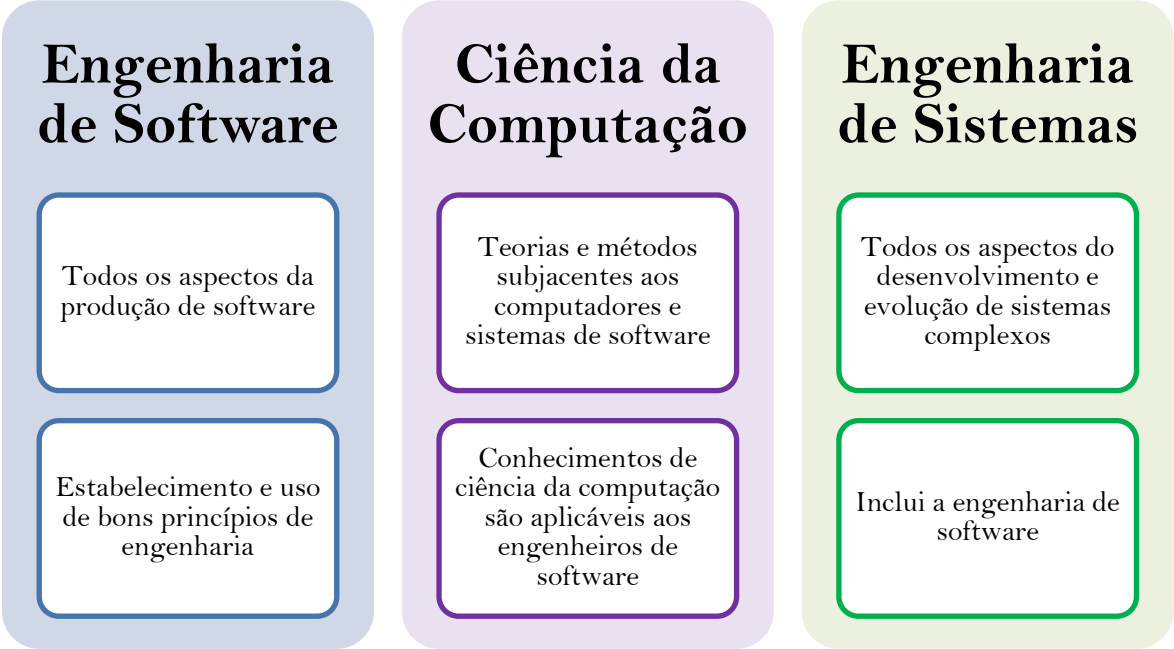
Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Princípios da Engenharia de Software

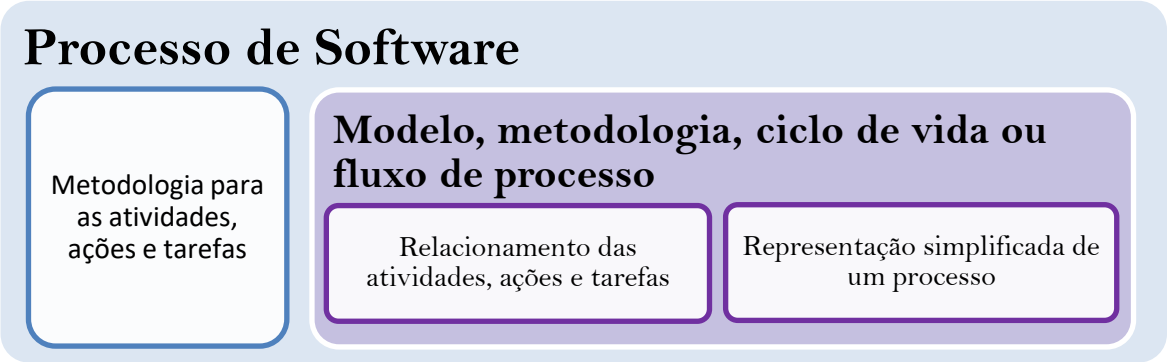


Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

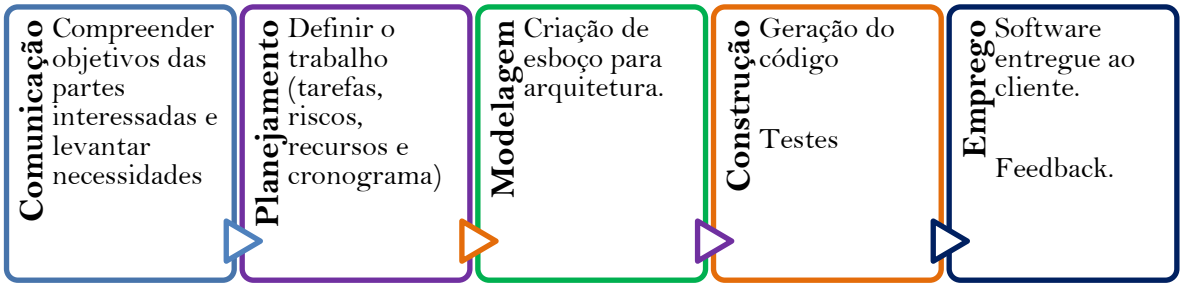
Engenharia de Software e outras disciplinas



Processo de Software

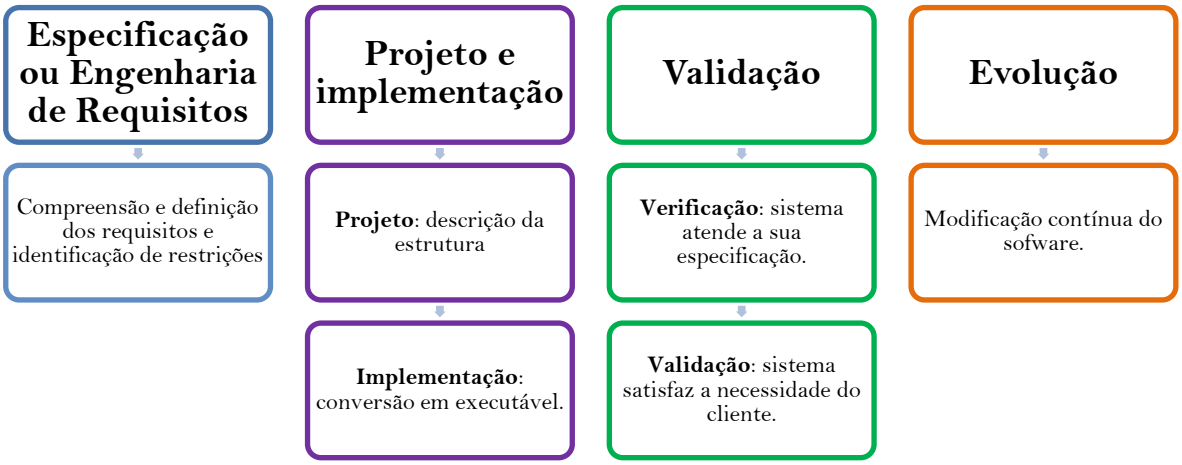


Modelo de Processo Genérico do Pressman



Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Modelo de Processo Genérico de Sommerville

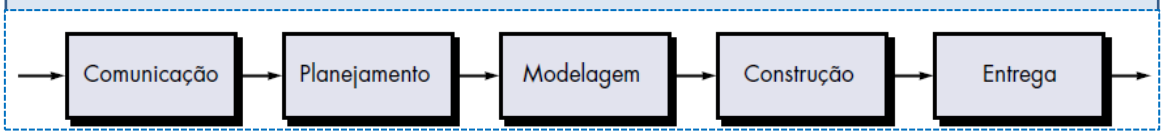


Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Tipos de Fluxos de Processo

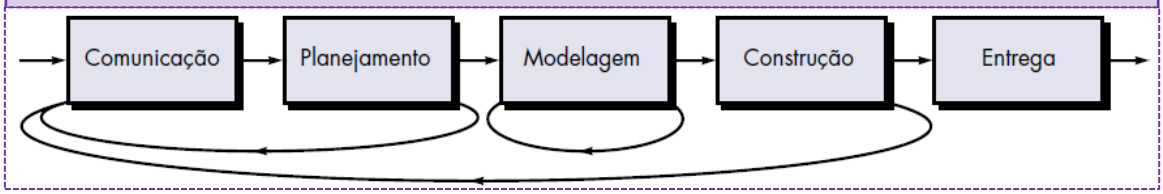
Linear

Executa cada uma das atividades em sequência



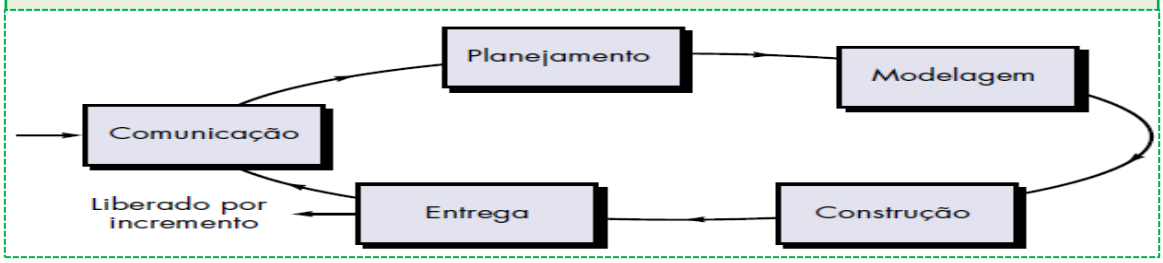
Iterativo

Repete uma ou mais atividades antes de prosseguir.



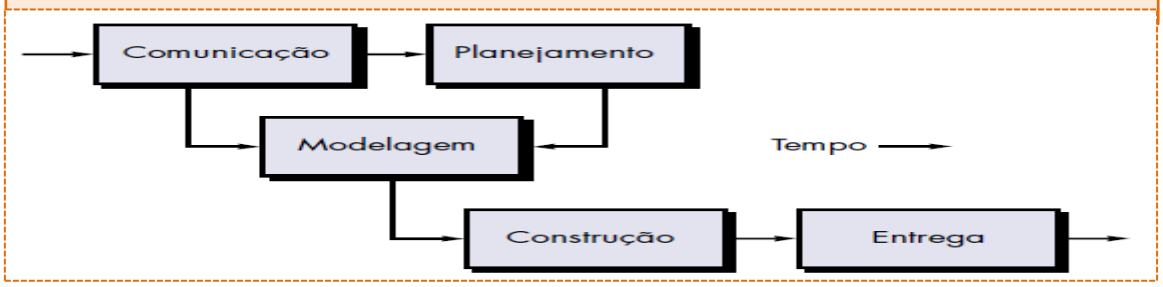
Evolucionário

Executa as atividades de uma forma circular. Cada volta conduz a uma versão mais completa do software.



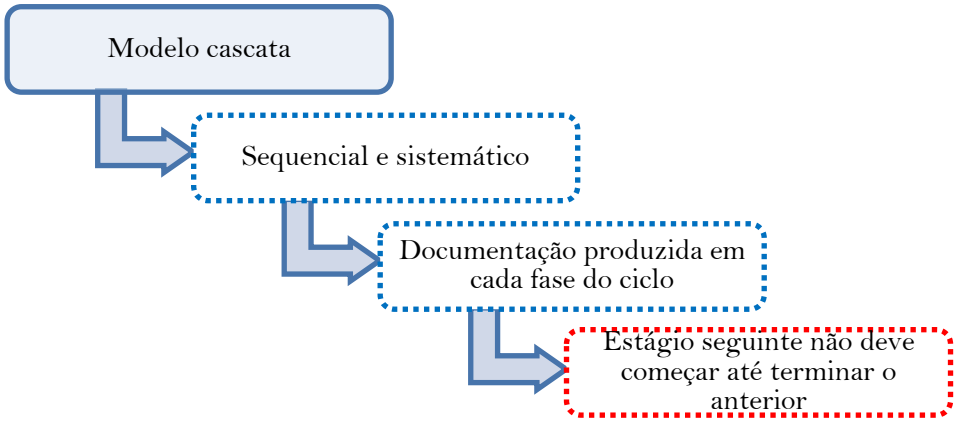
Paralelo

Executa uma ou mais atividades em paralelo



Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Modelo Cascata



Vantagens e desvantagens do Modelo Cascata

Vantagens

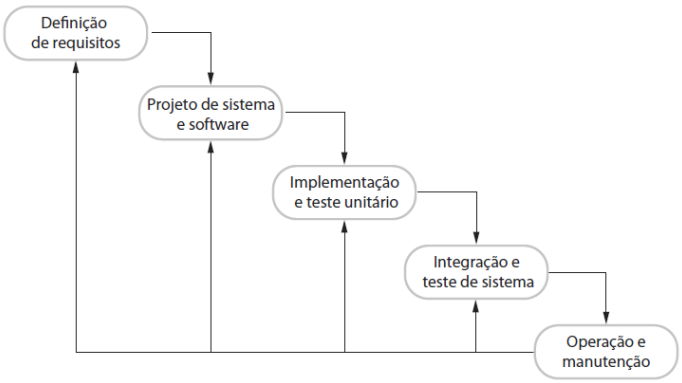
- Processo **simples** de entender.
- Fornece um **modelo de gerenciamento comum**.
- **Documenta** todas as etapas do processo.
- Funciona bem quando os **requisitos são bem compreendidos** e não sofrem grandes modificações.
- Funciona bem para **projetos pequenos**.
- **Aderente a outros modelos**.

Desvantagens

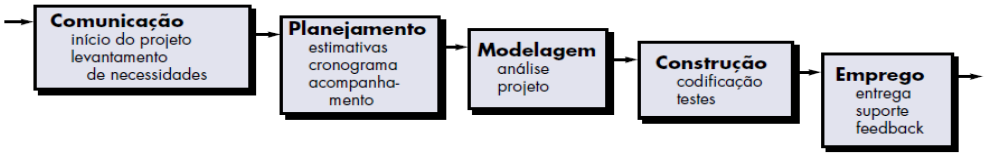
- Divisão **inflexível**.
- **Dificuldade de adaptar**-se a mudanças.
- Projetos **raramente seguem o fluxo** sequencial.
- Dificuldade do cliente **estabelecer todas as necessidades no início** do projeto.
- Versão operacional só fica **disponível ao final**.
- Não funciona bem com **projetos complexos**.
- Conduz a **"estados de bloqueio",** nos quais membros da equipe têm de aguardar outros completarem tarefas.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

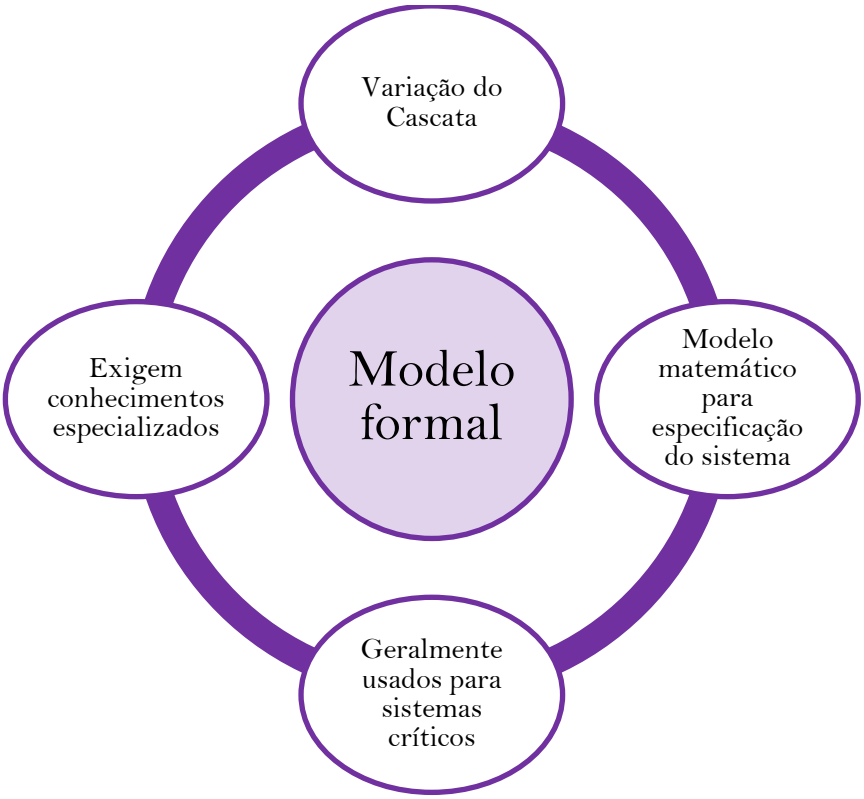
Estágios do modelo Cascata de Sommerville



Fases do modelo Cascata de Pressman

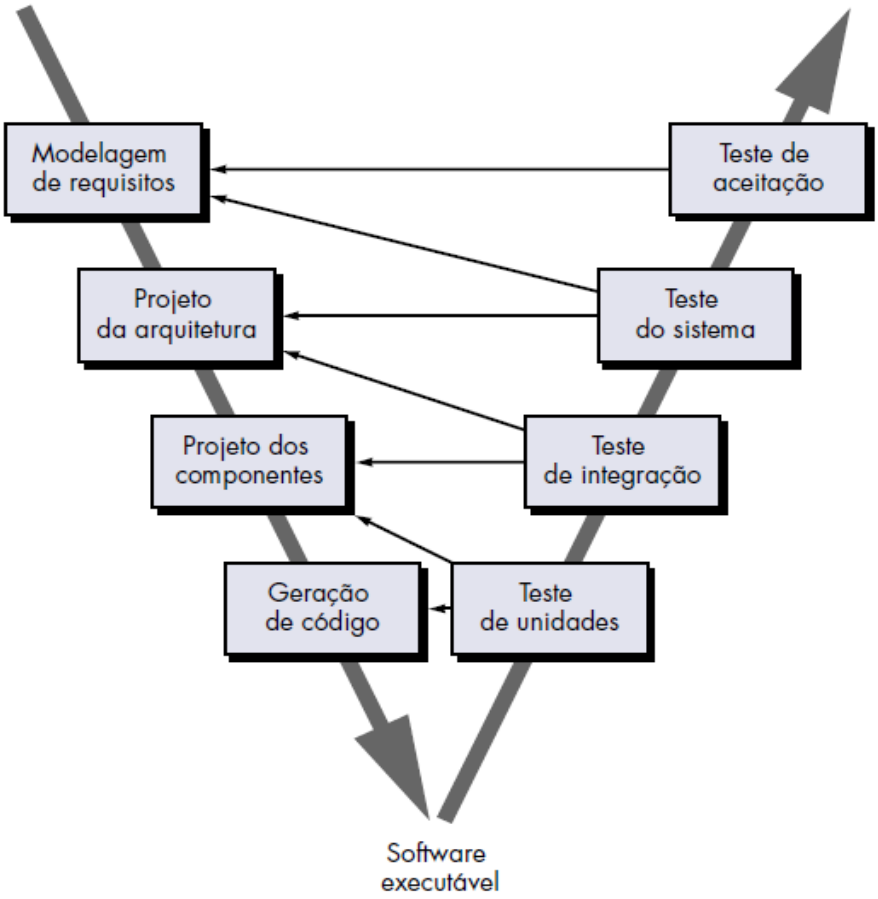


1Modelos formais

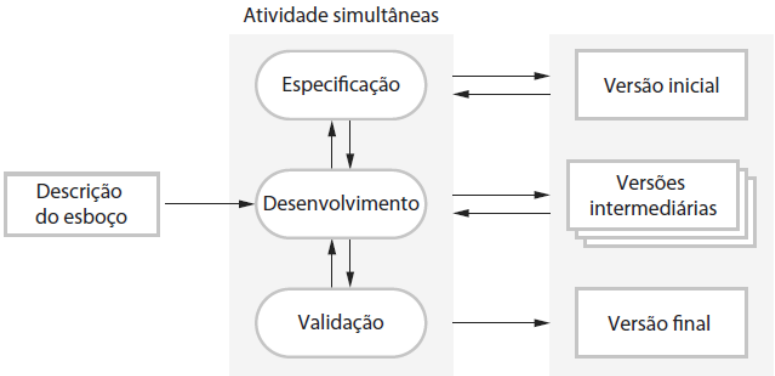


Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Modelo V segundo Pressman

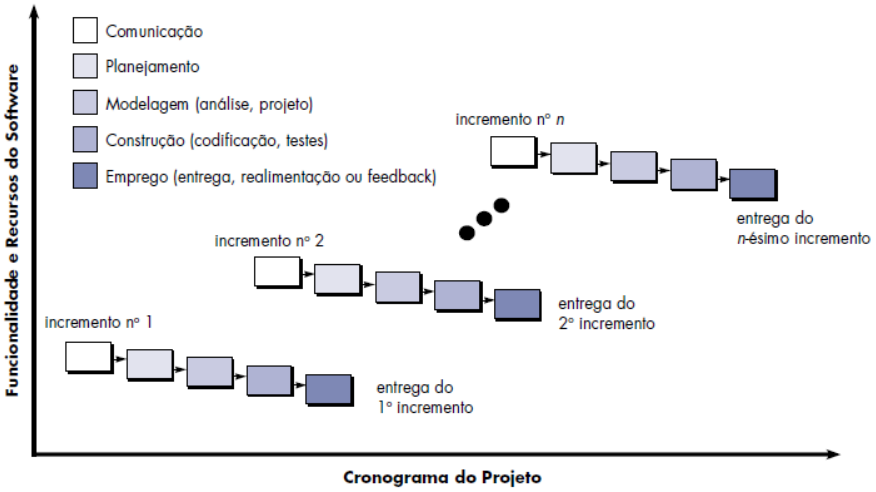


Desenvolvimento Incremental segundo Sommerville



Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Modelo Incremental segundo Pressman



Características do Desenvolvimento Incremental

Desenvolvimento Incremental

- Desenvolver uma implementação inicial, expô-la aos comentários dos usuários e continuar por meio da criação de várias versões
- Cada sequência linear gera "incrementos" (entregáveis/aprovados/liberados) operacionais do software.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Vantagens e desvantagens do desenvolvimento incremental

Vantagens

- Entrega de software operacional já no primeiro incremento
- Feedback do cliente é mais fácil e rápido
- Mais barato e mais fácil para fazer mudanças

Desvantagens

- Incrementos podem degradar a estrutura do sistema
- Processo não é visível

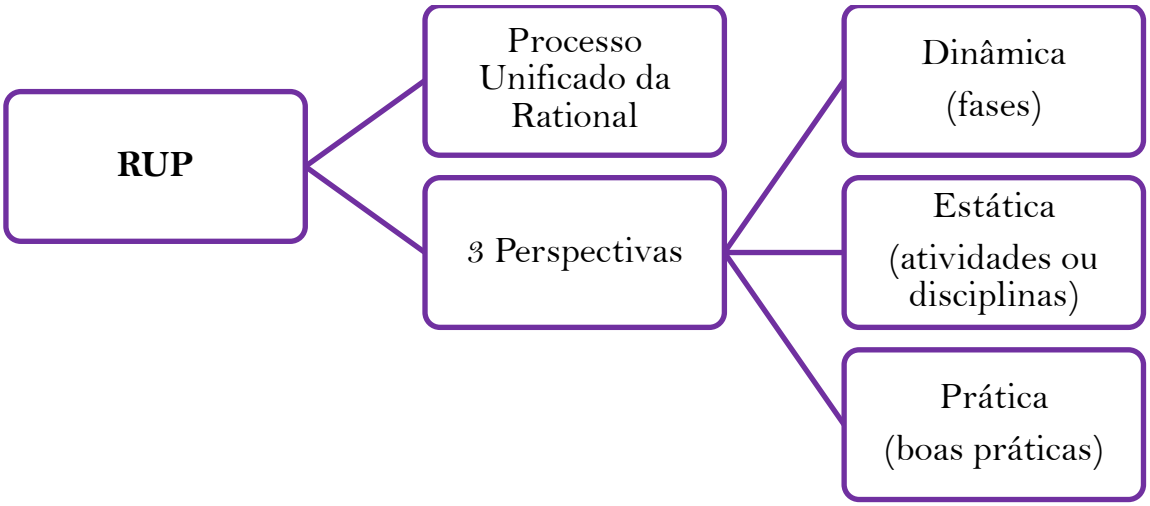
Iterativo x Incremental

Cada iteração gera um novo "pedaço" do produto.

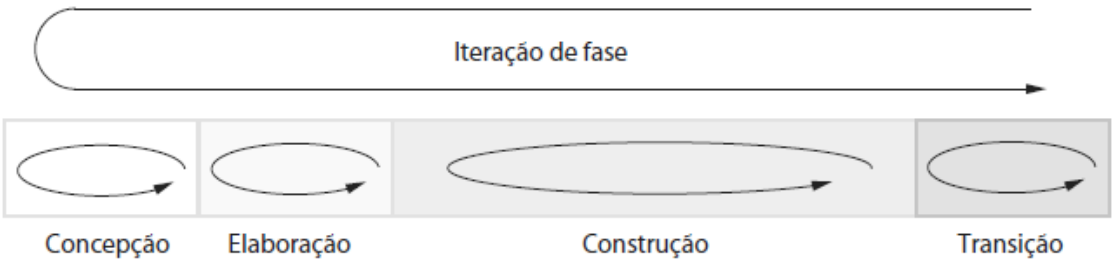
Cada iteração avança no conhecimento do produto.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Rational Unified Processo – RUP



Perspectiva Dinâmica ou fases do RUP



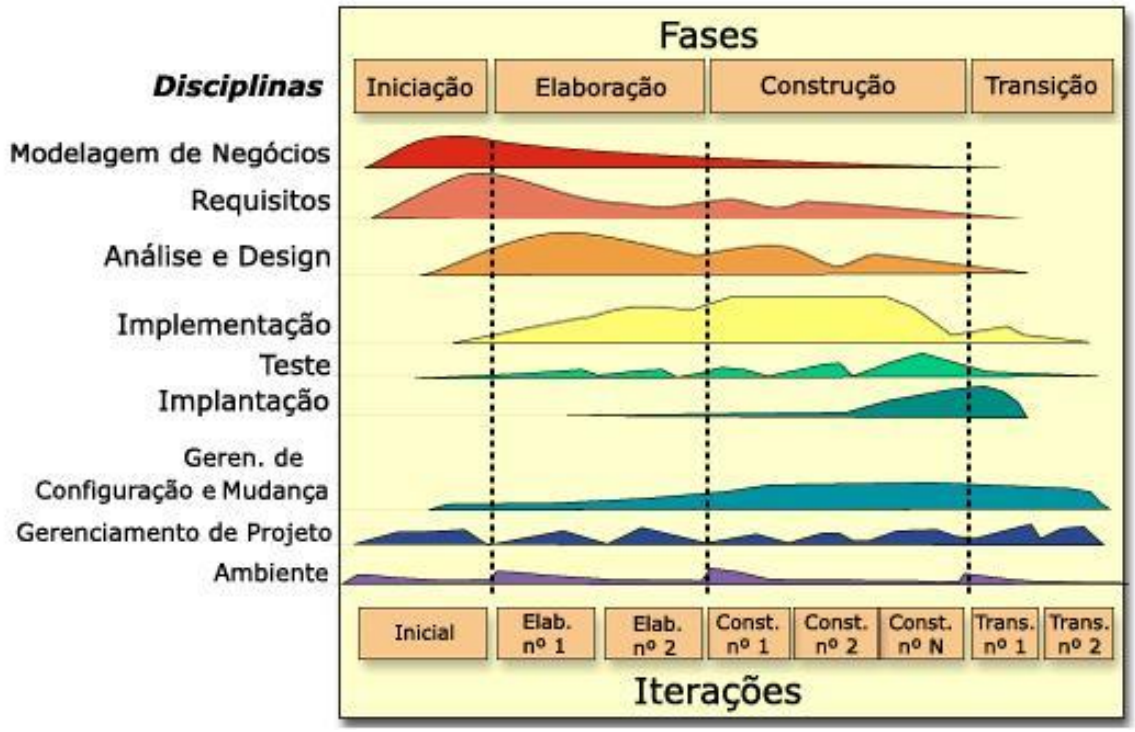
Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Perspectiva estática ou disciplinas do RUP

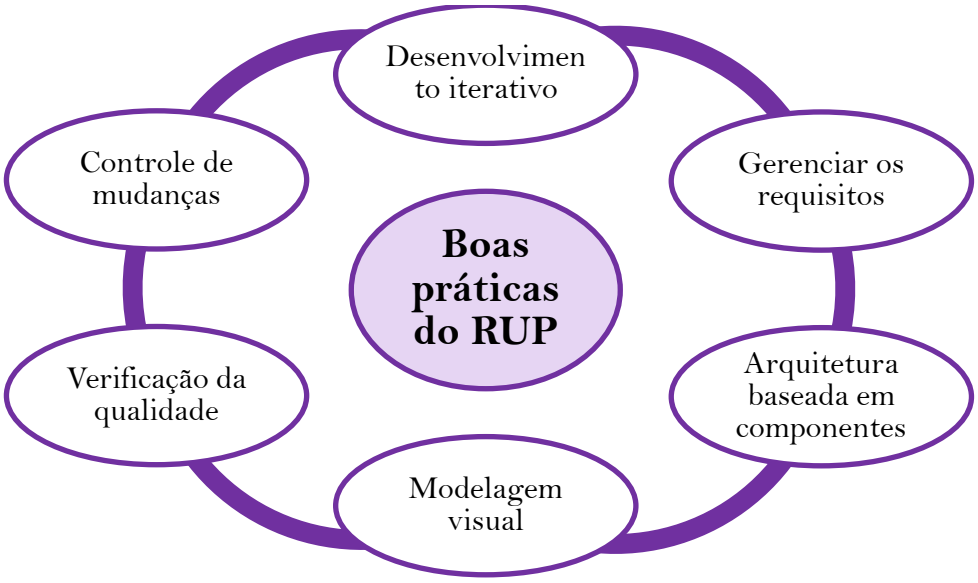
Workflow/Disciplina		Descrição
Disciplinas principais, centrais ou de engenharia de software	**Modelagem de negócios**	Os processos de negócio são modelados por meio de casos de uso de negócios.
	Requisitos	Atores que interagem com o sistema são identificados e casos de uso são desenvolvidos para modelar os requisitos do sistema.
	Análise e projeto	Um modelo de projeto é criado e documentado com modelos de arquitetura, modelos de componentes, modelos de objetos e modelos de sequência.
	Implementação	Os componentes do sistema são implementados e estruturados em subsistemas de implementação. A geração automática de código a partir de modelos de projeto ajuda a acelerar esse processo.
	Teste	O teste é um processo iterativo que é feito em conjunto com a implementação. O teste do sistema segue a conclusão da implementação.
	Implantação	Um release do produto é criado, distribuído aos usuários e instalado em seu local de trabalho.
Disciplinas de apoio/suporte	**Gerenciamento de configuração e mudanças**	Gerencia as mudanças do sistema
	Gerenciamento de projeto	Gerencia o desenvolvimento do sistema
	Meio ambiente	Relacionado com a disponibilização de ferramentas apropriadas para a equipe de desenvolvimento de software.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Fases e Disciplinas do RUP

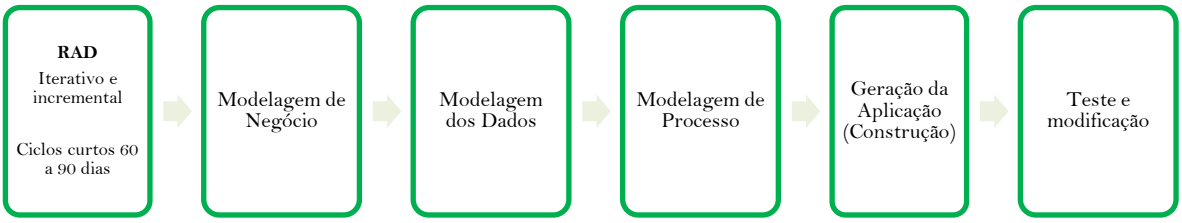


Perspectiva prática ou boas práticas do RUP

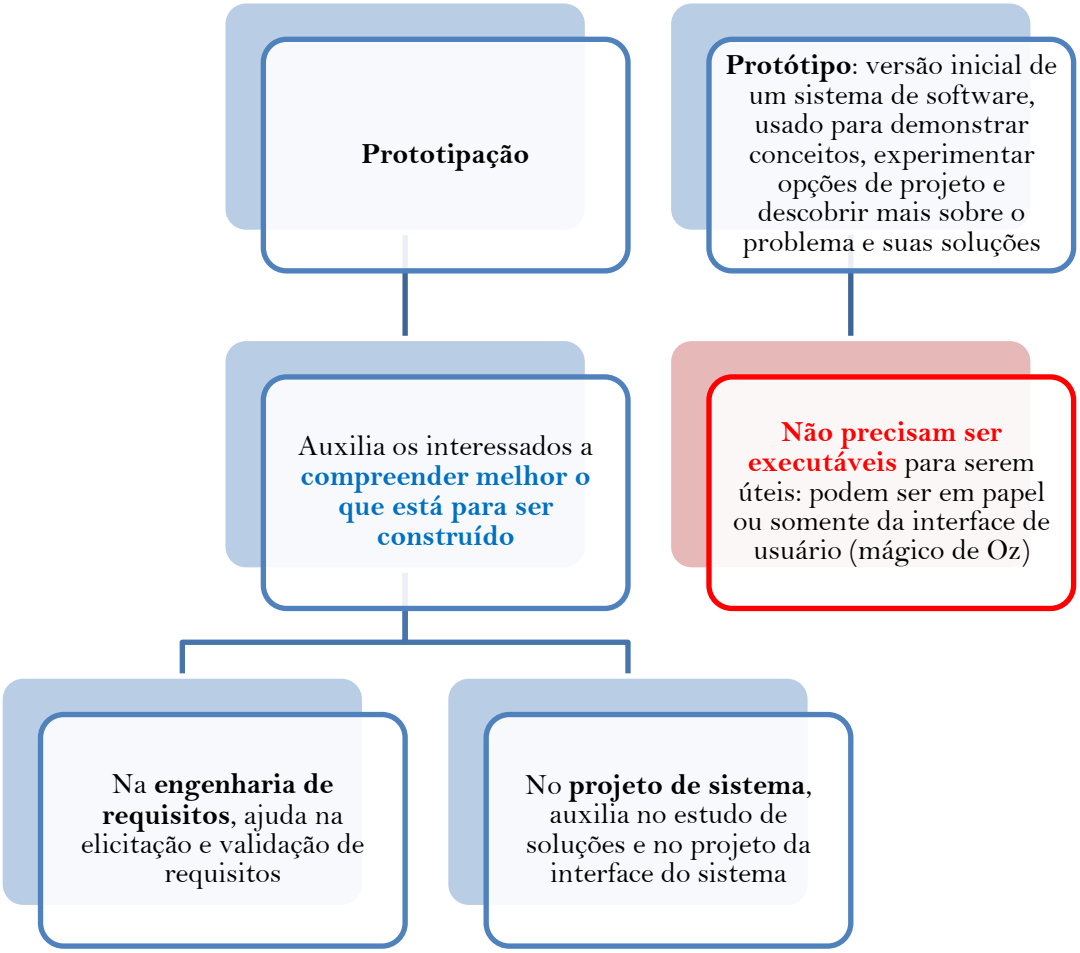


Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Rapid Application Development (RAD)



Prototipação



Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Tipos de prototipação

Prototipação evolucionária ou desenvolvimento exploratório

- Protótipo é refinado até o sistema final
- Modelo de processo
- Começa com requisitos mais compreendidos

Prototipação descartável (throw-away)

- Protótipo é descartado
- Técnica usada em outros modelos
- Começa com requisitos menos compreendidos

Vantagens e desvantagens da prototipação

Vantagens

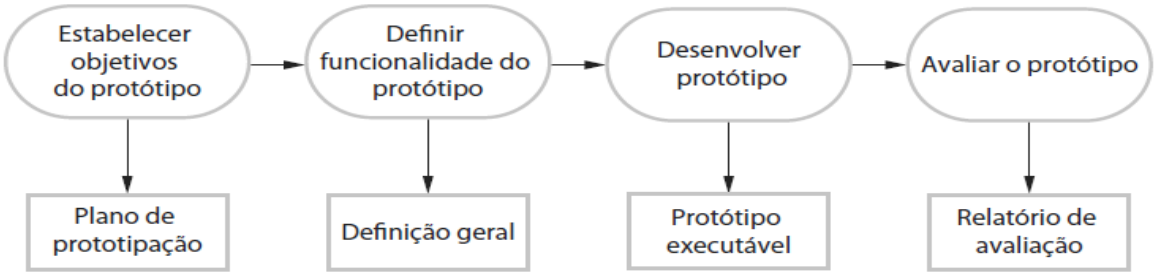
- Auxilia na compreensão dos requisitos
- Permite estudar soluções e interfaces de usuário
- Não precisam ser executáveis
- Pequeno investimento inicial para produzir uma primeira versão do software

Desvantagens

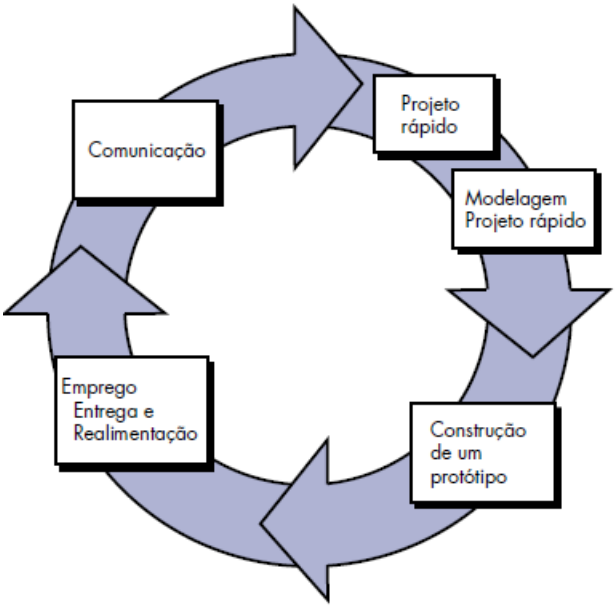
- Dificuldade de implementar requisitos não funcionais
- Não documentado
- Degradação da estrutura do sistema
- Padrões de qualidade relaxados
- Pressão do cliente para entregar protótipo com poucas correções
- Acomodação ou esquecimento de uso de tecnologias ineficientes

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Modelo de Processo para Desenvolvimento de Protótipos de Sommerville



Paradigma de prototipação segundo Pressman



Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Modelo Espiral

Modelo espiral

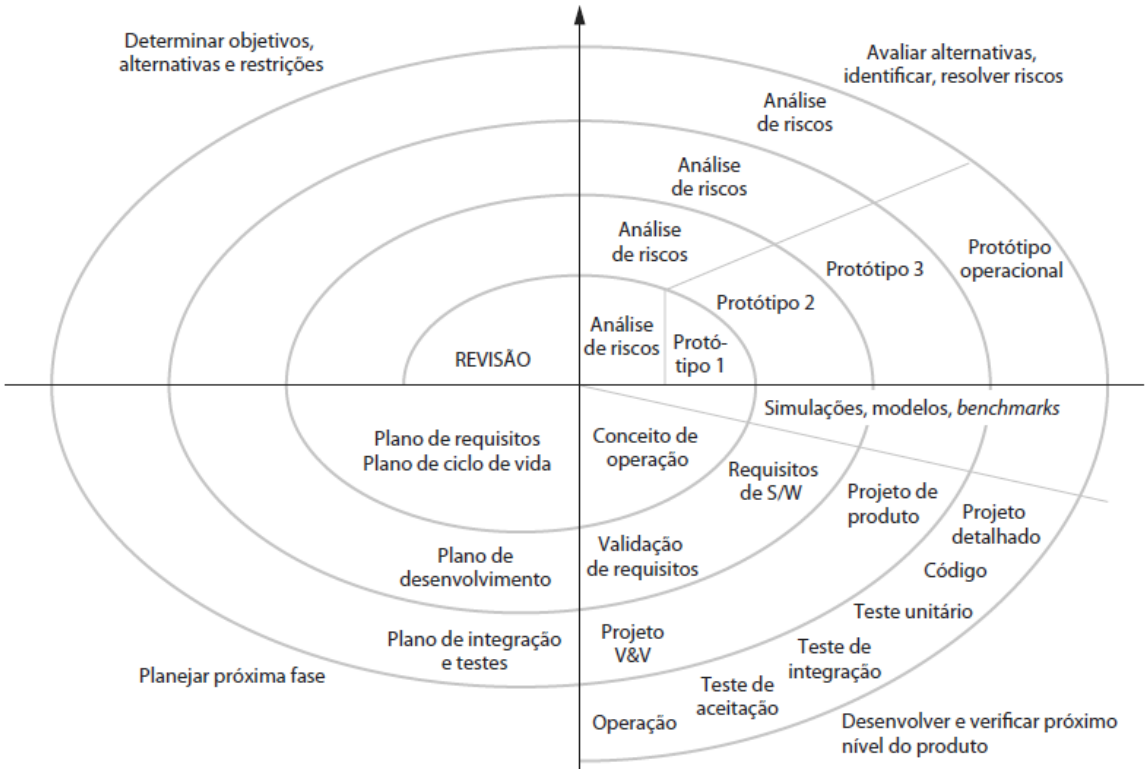
Evolucionário que acopla a natureza iterativa da prototipação com os aspectos sistemáticos e controlados do modelo cascata

Dirigido a riscos (inclui atividades explíticas de gerenciamento de riscos)

Cada volta na espiral representa uma fase do processo

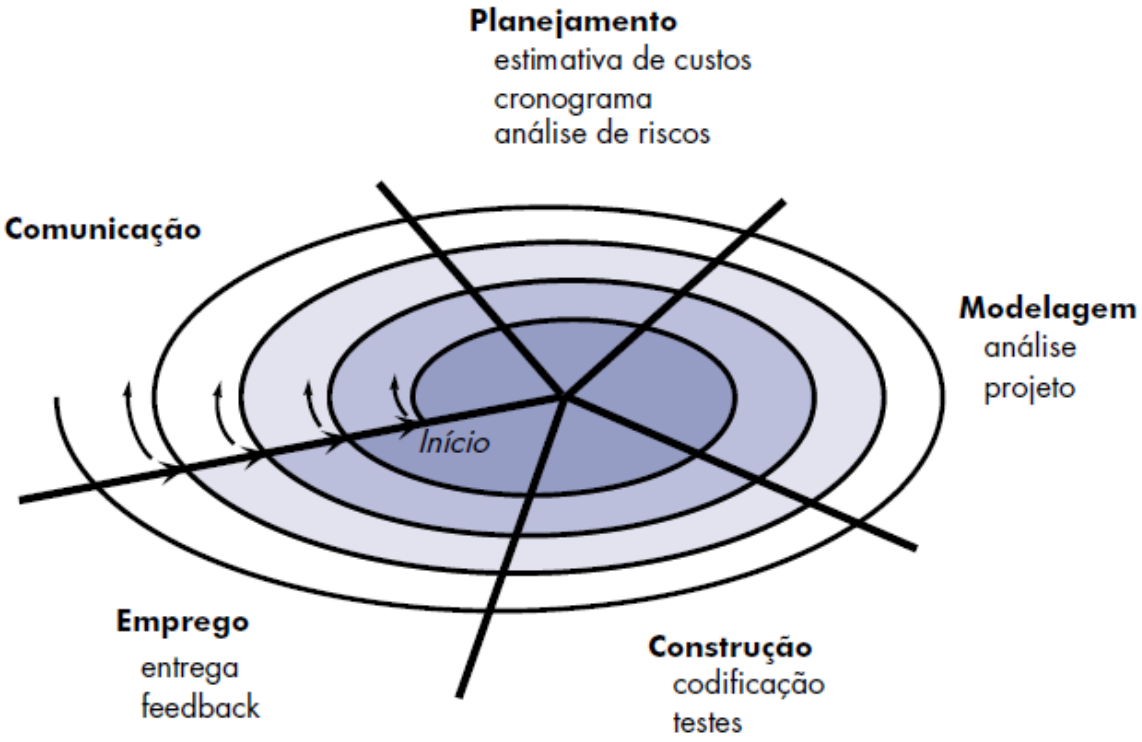
Combina prevenção e tolerância a mudanças

Modelo espiral de Boehm segundo Sommerville



Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Modelo espiral proposto por Pressman

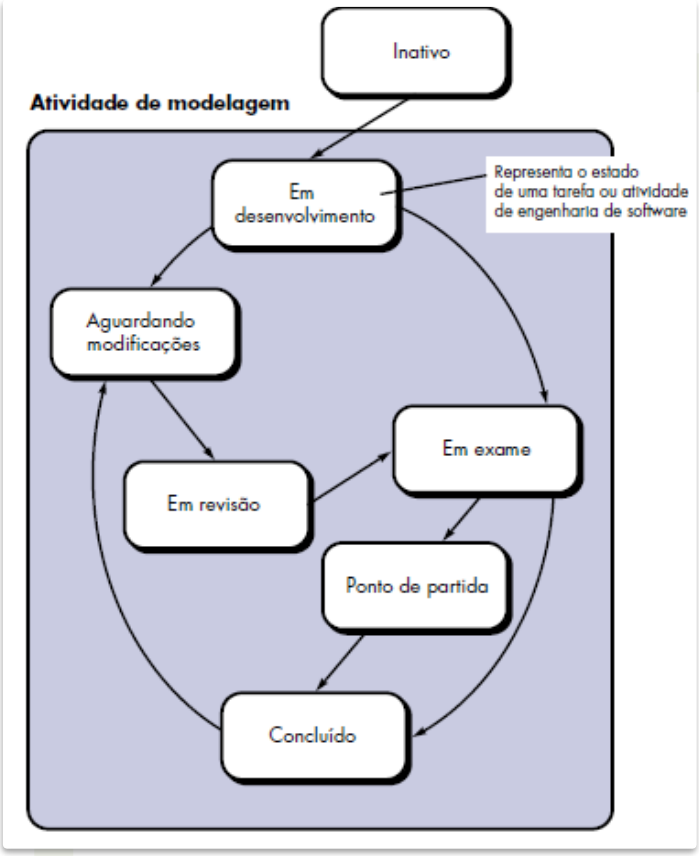


Vantagens e desvantagens do modelo espiral

Vantagens	Desvantagens
Evolucionário, combinando a iteratividade da prototipação com o controle do cascata	Exige especialização em avaliação de riscos
Define uma série de pontos âncora de controle	Um risco não descoberto pode causar grandes impactos
Combina prevenção e tolerância a mudanças	Gerenciamento de processo pode ser mais complexo
Inclui explicitamente atividades de gestão de riscos	

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Modelo de Desenvolvimento Concorrente ou Engenharia Concorrente



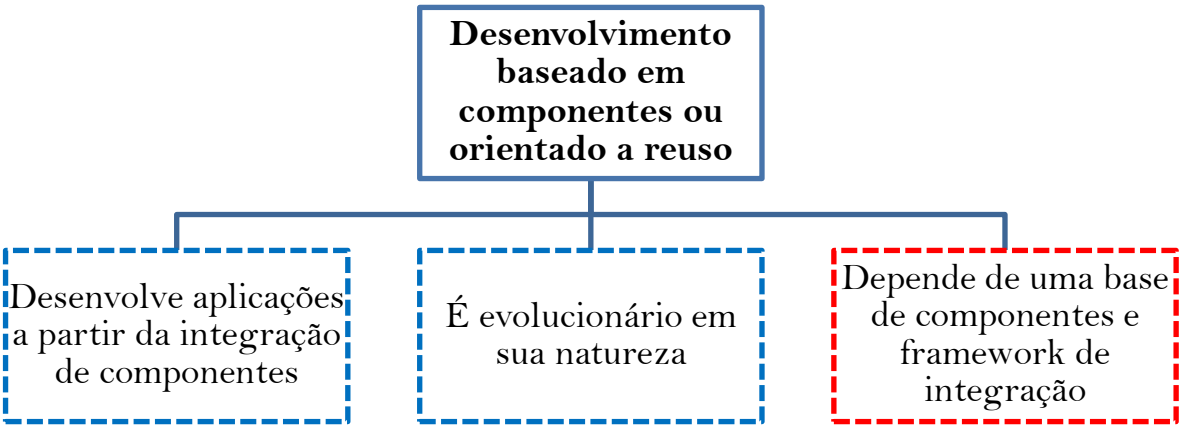
Modelo de Desenvolvimento Concorrente

- Representa elementos concorrentes e iterativos de qualquer modelo de processo.

- Fornece uma imagem precisa do estado do projeto.

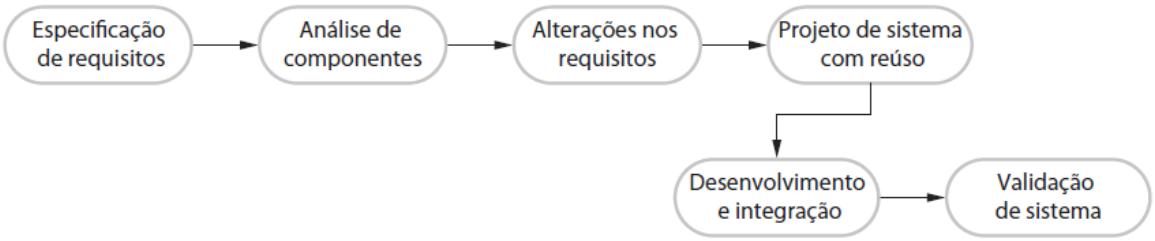
- Define uma rede de processos.

Desenvolvimento baseado em componentes ou orientado a reuso



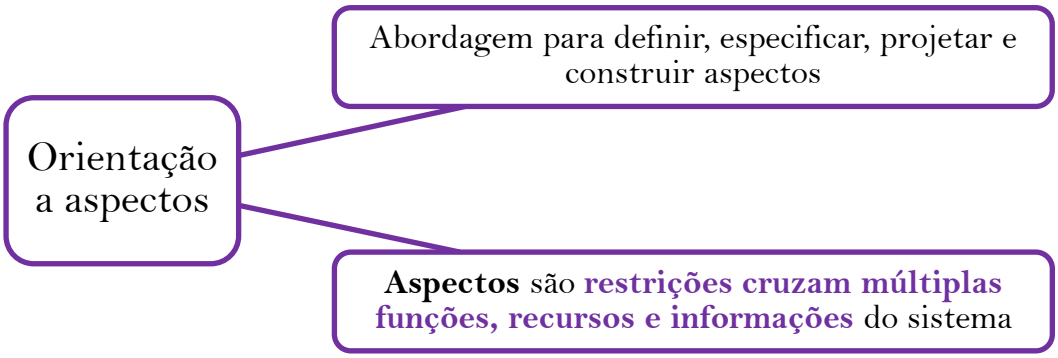
Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Desenvolvimento orientado a reuso de Sommerville



Desenvolvimento baseado em componentes de Pressman

Desenvolvimento orientado a aspectos

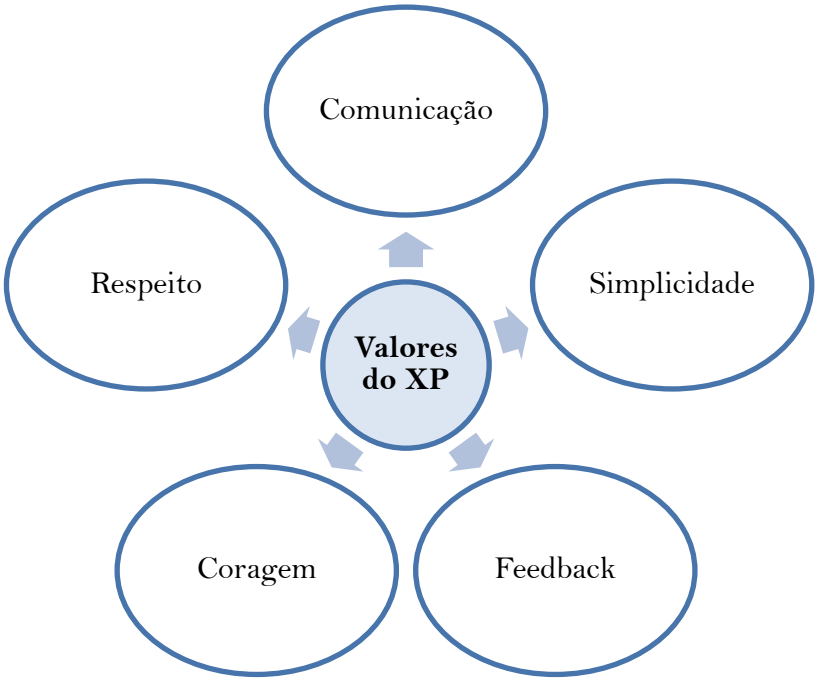


Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Métodos ágeis

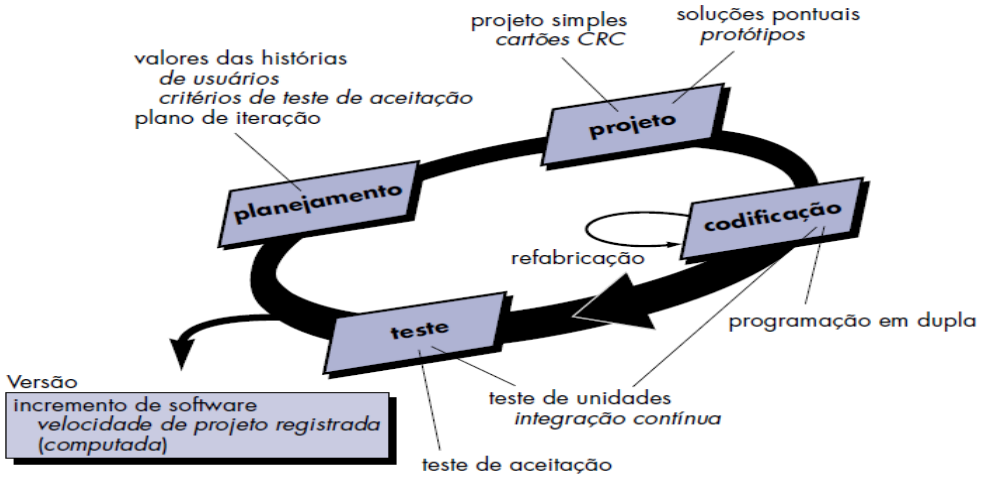
Metódos ágeis	
	Indivíduos e interações acima de **processos e ferramentas**
	Software operacional acima de **documentação completa**
	Colaboração dos clientes acima de **negociação contratual**
	Respostas a mudanças acima de **seguir um plano**

Valores do XP

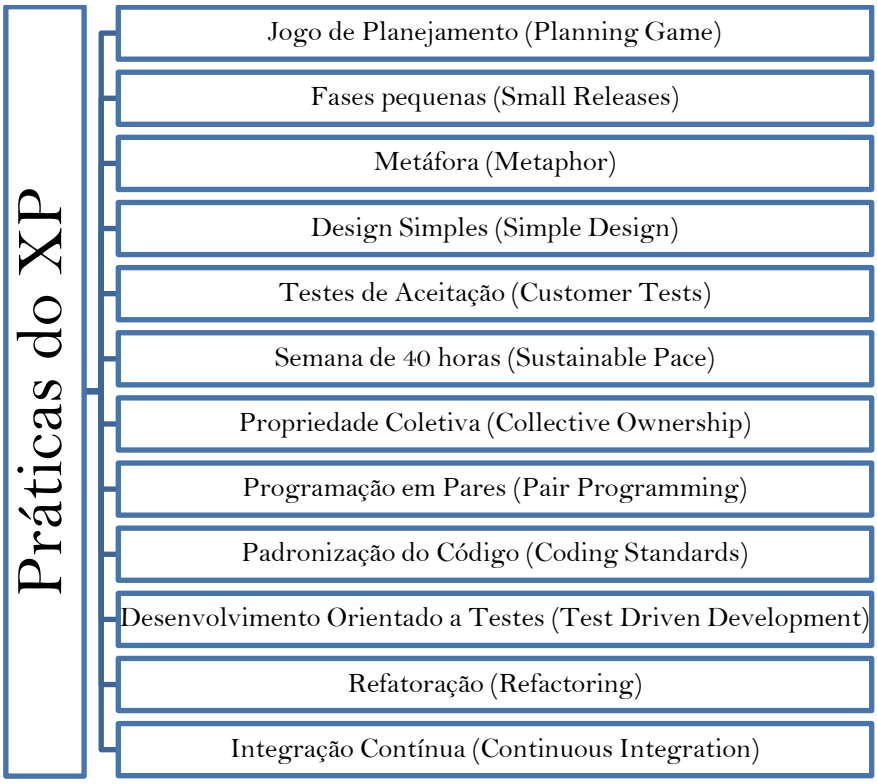


Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Processo de Extremming Programming segundo Pressman



Práticas do XP



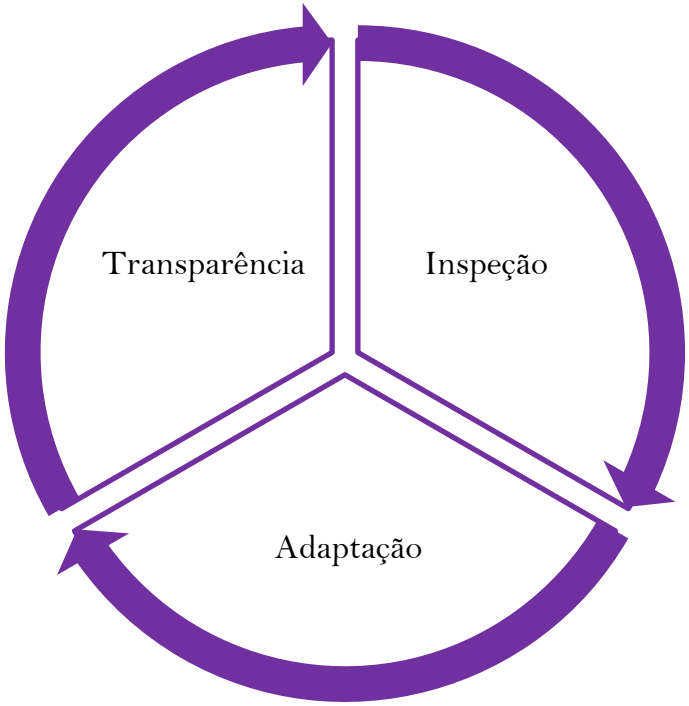
Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Scrum

Scrum

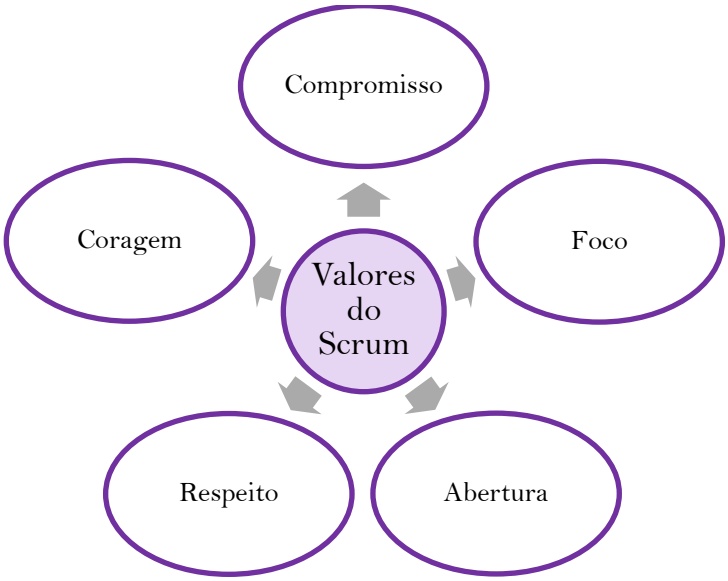
- Método ágil com foco no gerenciamento
- Abordagem iterativa e incremental
- Fundamentado no empirismo
- Times do Scrum associadas a papéis, eventos, artefatos e regras

Pilares do Scrum



Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

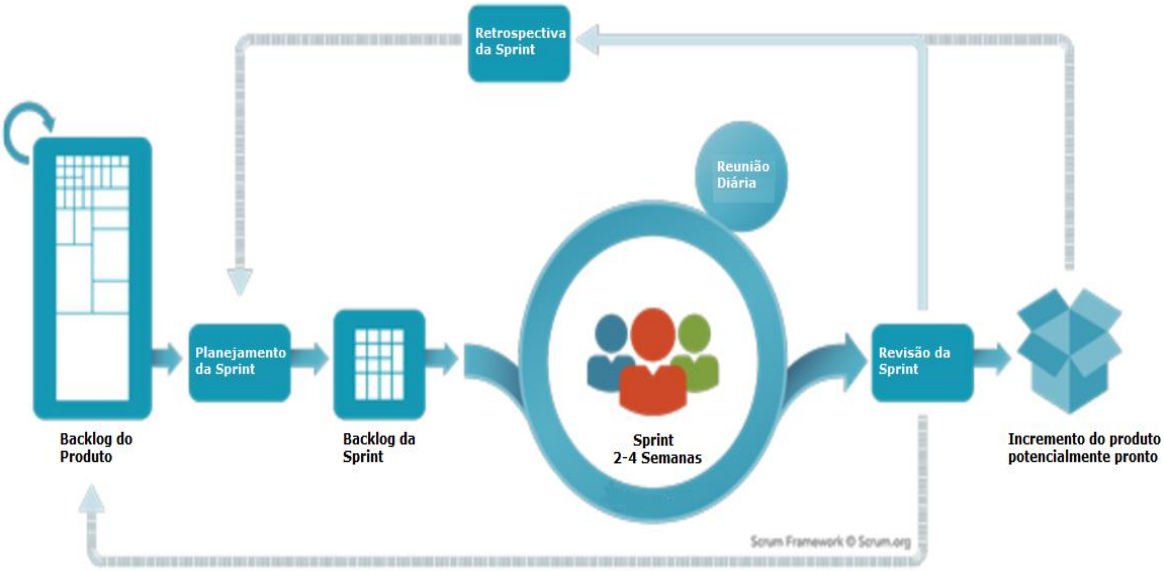
Valores do Scrum



Time Scrum

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Processo do Scrum



Eventos Scrum

Planejamento da Sprint
- Planejar o trabalho a ser executado
- **8 horas** para sprint de 1 mês
- **Responde a**:
  - Por que esta Sprint é valiosa?
  - O que pode ser feito nesta Sprint?
  - Como o trabalho escolhido será realizado?

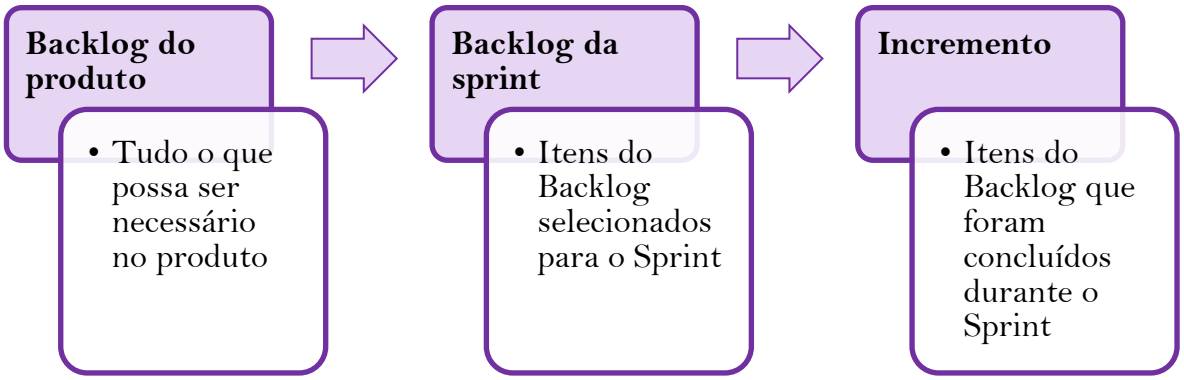
Reunião diária
- Sincronizar atividades e criar um plano para as próximas 24 horas
- **15 min** por dia
- **Responde a**:
  - O que eu fiz ontem?
  - O que eu vou fazer?
  - Quais são os impedimentos?

Revisão da Sprint
- Inspercionar o incremento e adaptdar o product backlog
- **4 horas** para sprint de 1 mês

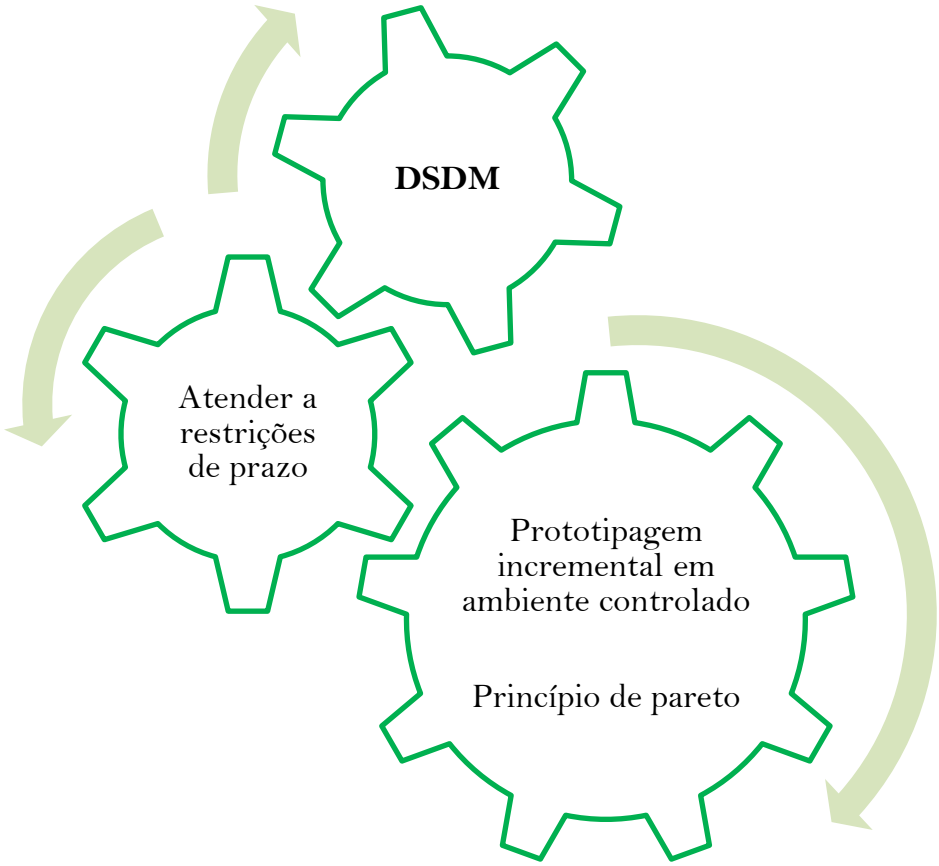
Retrospectiva da Sprint
- Revisar o processo
- **3 horas** para sprint de 1 mês

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Artefatos do Scrum

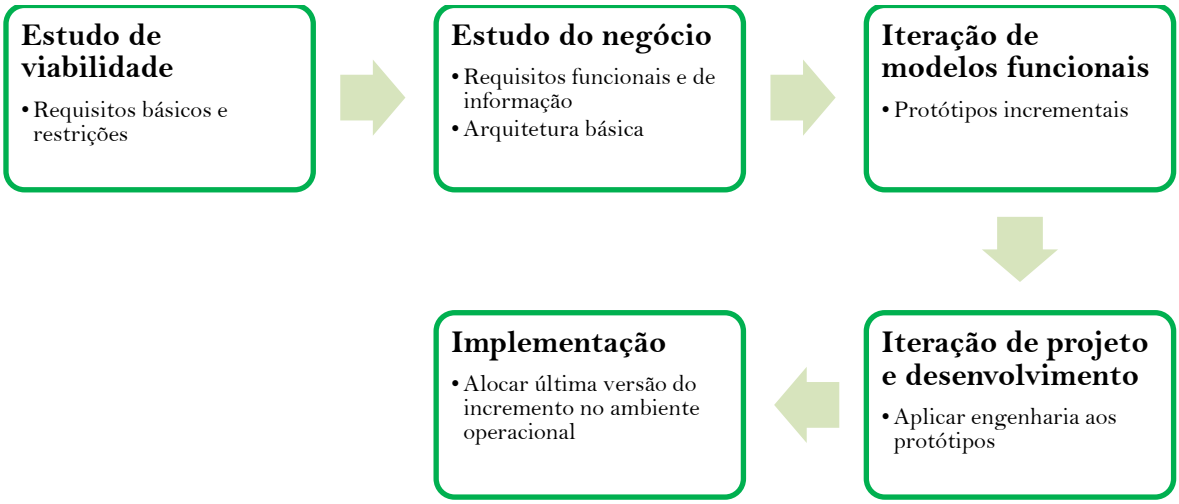


Método de Desenvolvimento de Sistemas Dinâmicos (DSDM)

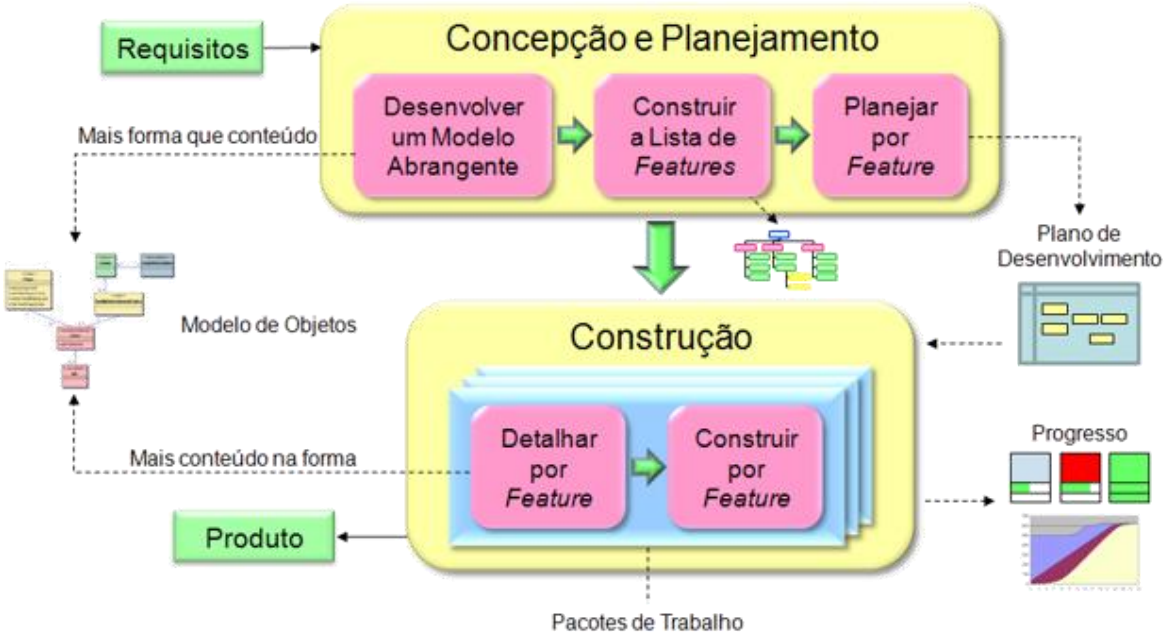


Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Ciclo de vida DSDM



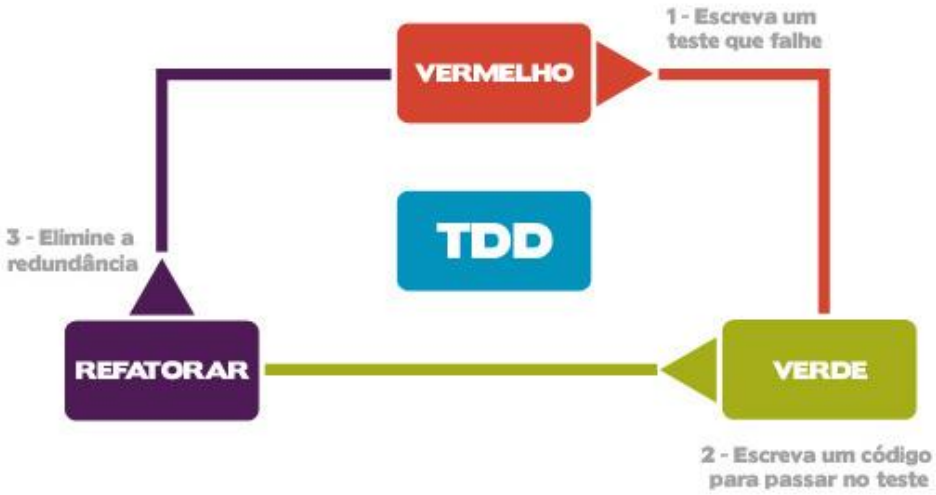
Feature Driven Development – FDD



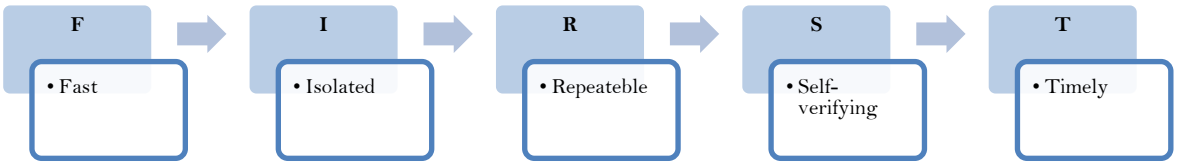
Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Ciclo TDD

Ciclo TDD Simplificado

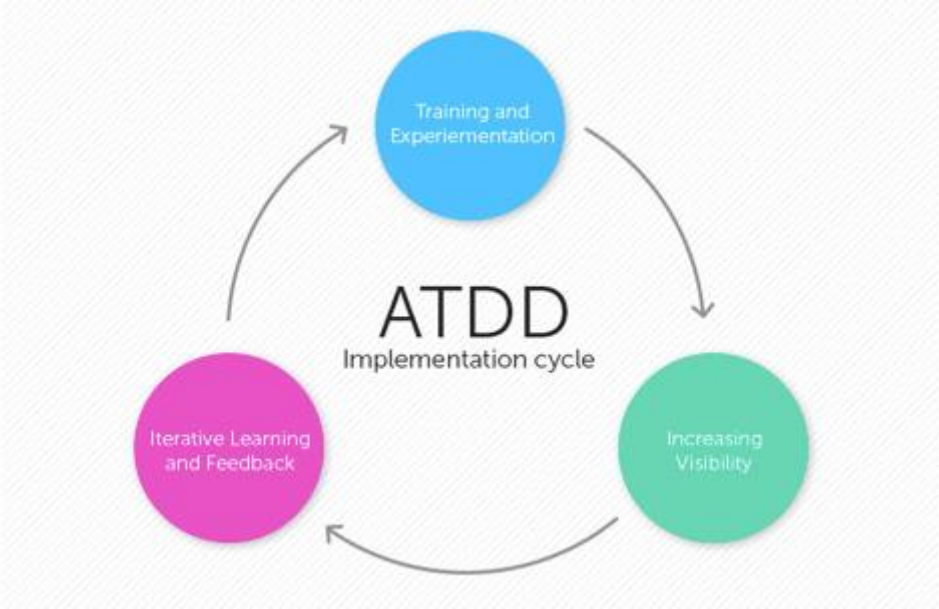


Características dos testes

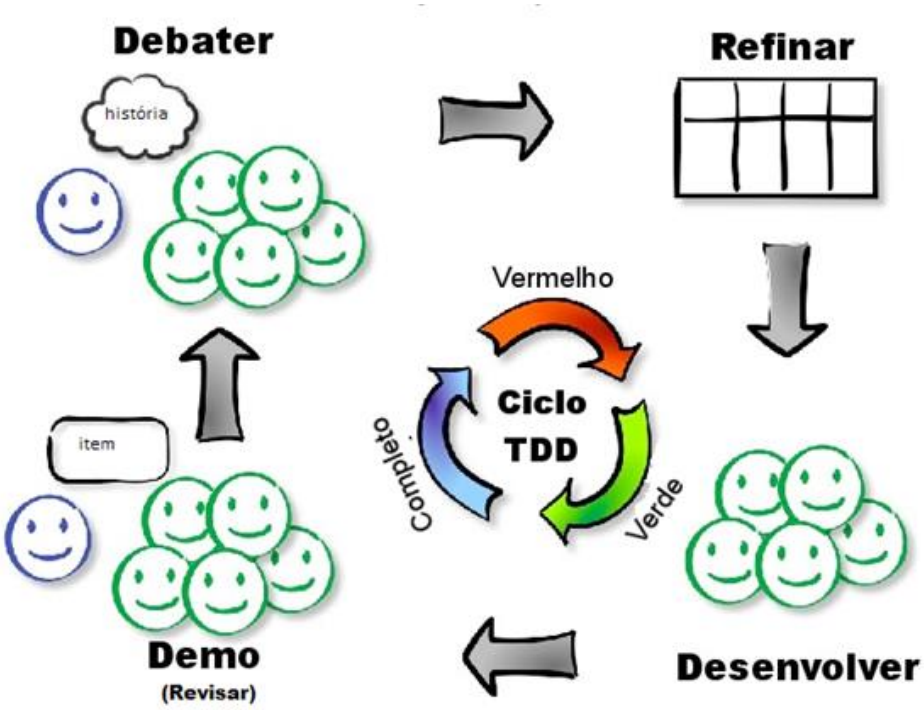


Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Ciclo de Implementação do ATDD

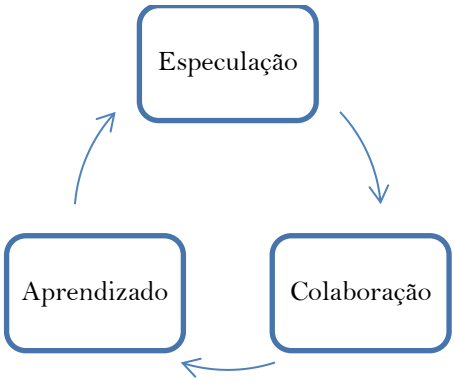


Etapas do ATDD

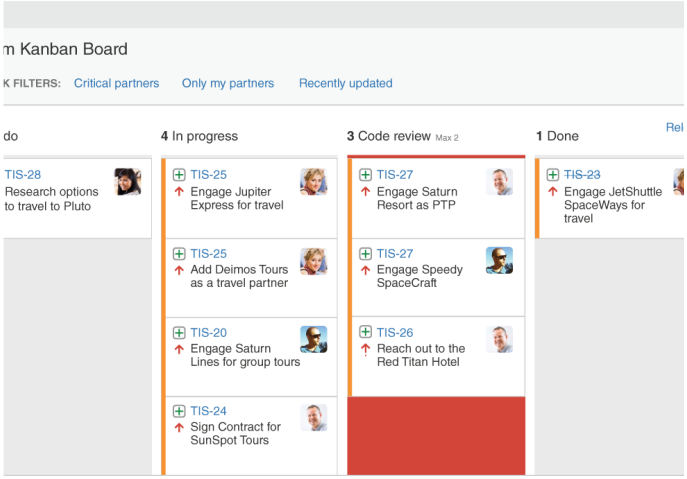


Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Ciclo do ASD



KANBAN



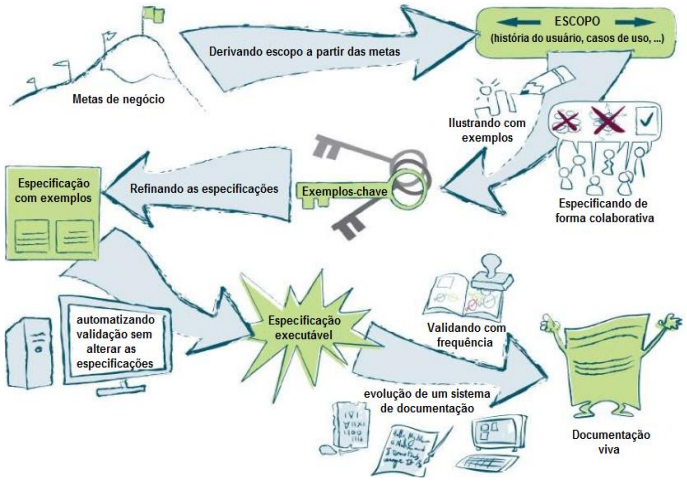
KANBAN

Representação visual do fluxo de trabalho.

Entrega contínua de resultados.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

Especificação por exemplo



Especificação por exemplo

- Práticas que ajuda a construir o produto certo, focando na comunicação entre pessoas envolvidas no projeto.

- Documentação viva.

- Foco no QUE e não no COMO.

- Exemplos concretos para definir o comportamento do sistema.

DDD

DDD (Domain Driven Design)

- Metodologia que **centra o desenvolvimento na programação de um modelo de domínio que tenha um rico entendimento dos processos e regras de um domínio**
- Três **pilares**
  - **Linguagem ubíqua**
    Linguagem compartilhada por todos os membros da equipe de desenvolvimento e partes interessadas do negócio
  - **Contextos delimitados (bounded contexts)**
    Separação de responsabilidades entre as diferentes partes do sistema.
  - **Mapas de contexto (context maps)**:
    Utilizados para entender a comunicação entre os contextos delimitados
- **Modelo de domínio** é a base da comunicação do projeto
- Arquitetura de 4 **camadas**
  - **Camada de interface:** o que o usuário tem acesso.
  - **Camada de aplicação:** coordenação das atividades.
  - **Camada de domínio**: conceitos e regras de negócio.
  - **Camada de infraestrutura**: depósito de materiais.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07

5. REFERÊNCIAS

AGILALLIANCE. **Acceptance Test Driven Development (ATDD)**. Disponível em: <https://www.agilealliance.org/glossary/atdd/>. Acesso em: 15 out. 2019.

DEVMEDIA. **Introdução à Refatoração**. Disponível em: <https://www.devmedia.com.br/introducao-a-refatoracao/21377>. Acesso em: 15 out. 2019.

DEVMEDIA. **TDD: fundamentos do desenvolvimento orientado a testes**. Disponível em: <https://www.devmedia.com.br/tdd-fundamentos-do-desenvolvimento-orientado-a-testes/28151 >. Acesso em: 09 jan. 2019.

DEVMEDIA. **Modelos de Processos Ágeis: conceitos e princípios**. Disponível em: <https://www.devmedia.com.br/modelos-de-processos-ageis-conceitos-e-principios/30059>. Acesso em: 18 dez. 2019.

IMASTERS. Especificação Por Exemplo como ela é – Parte 01. Disponível em: <https://imasters.com.br/desenvolvimento/especificacao-por-exemplo-como-ela-e-parte-01>. Acesso em: 26 abr. 2023.

INFOQ. **Um guia rápido para a implementação do ATDD**. Disponível em: <https://www.infoq.com/br/articles/quick-guide-atdd/>. Acesso em: 15 out. 2019.

INFOQ. **Acceptance Test-Driven Development (ATDD), passo a passo**. Disponível em: < https://www.infoq.com/br/articles/atdd-passo-a-passo/>. Acesso em: 15 out. 2019.

MANIFESTO AGIL. **Manifesto para o desenvolvimento ágil de software**. Disponível em: <http://www.manifestoagil.com.br/> Acesso em: 18 ago. 2018.

MEDIUM. **Entendendo um pouco sobre Especificação por Exemplo**. . Disponível em: <https://medium.com/@ujdlqs/entendendo-um-pouco-sobre-especifica%C3%A7%C3%A3o-por-exemplo-f95727ed3891>. Acesso em: 26 abr. 2023.

MEDIUM. **Domain-Driven Design, do início ao código**. . Disponível em: <https://medium.com/cwi-software/domain-driven-design-do-in%C3%ADcio-ao-c%C3%B3digo-569b23cb3d47>. Acesso em: 26 abr. 2023.

OPUS. **Entrega Contínua – O que é e como aplicar**. Disponível em: <https://www.opus-software.com.br/o-que-e-entrega-continua/>. Acesso em: 15 out. 2019.

PRESSMAN, Roger. **Engenharia de Software**. 7 ed. Porto Alegre : AMGH, 2011.

SCHWABER, Ken; SUTHERLAND, Jeff. **O Guia do SCRUM**. 2020.

SCRUM.ORG. **What is Scrum?**. Disponível em: <https://www.scrum.org/resources/what-is-scrum > Acesso em: 18 ago. 2018.

SOMMERVILLE. Ian. **Engenharia de Software**. 9 ed. Paulo: Pearson Prentice Hall, 2011.

XPEEDUCACAO. Domain-Driven Design (DDD): quais seus princípios e como implementar. Disponível em: <https://blog.xpeducacao.com.br/domain-driven-design-ddd/> Acesso em: 26 abr. 2023.

Licensed to ADEMILSON RODRIGUES LIMA JUNIOR - juninrodv@gmail.com - 160.630.027-07